

MANUAL DE EXPERIMENTOS

LINGUAGEM PYTHON

PARTE III

Ramon Mayor Martins, Ph.D.

IFSC • Câmpus São José
ramon.mayor at ifsc.edu.br

📧 [rmayormartins](#) • [rmayormartins.github.io](#)

Versão 2026/2 • Aprendizado de máquina e profundo

X. ANÁLISE DOS REQUISITOS

25	O processo de desenvolvimento	1
	Centrado no ser humano · Os três tipos de aprendizado · Vocabulário mínimo · Como este volume está organizado	
26	Análise dos requisitos	3
	As perguntas antes do código	

XI. PREPARAÇÃO DE DADOS

27	Preparação de dados	4
-----------	----------------------------	----------

XII. APRENDIZADO DE MÁQUINA: SUPERVISIONADO

28	Treinamento: o contrato do scikit-learn	7
29	Algoritmos clássicos e fronteiras de decisão	5
30	Regressão e séries temporais	5
	Regressão · Séries temporais	
31	Modelos em conjunto	6
32	Avaliação: métricas, limiar e confiança	6
33	Interpretabilidade	5

XIII. APRENDIZADO DE MÁQUINA: NÃO SUPERVISIONADO

34	Dimensionalidade e agrupamento	7
	Análise de componentes principais · Agrupamento	
35	Detecção de anomalias	3
36	Poucos rótulos: semi e autossupervisão	4

XIV. APRENDIZADO POR REFORÇO

37	Aprendizado por reforço	4
	Q-learning tabular	

XV. APRENDIZADO PROFUNDO

38	Rede neural do zero: o treinamento por dentro	5
39	TensorFlow e Keras	9
40	Redes convolucionais e sinais	6
	Convolução em uma dimensão	
41	Arquiteturas e treino avançado	9
	Além da pilha linear · Autoencoders · Redes recorrentes · Treino: o que realmente muda o resultado · Transferência de aprendizado	
42	PyTorch	6
	Tensores e autograd · Um modelo completo · Escolhendo entre os dois	

XVI. PREDIÇÃO E EXPORTAÇÃO

43	Predição, implantação e exportação	7
	Do Keras para o dispositivo · Depois de publicar	
44	Fechando o ciclo: avaliação honesta	6
	Checklist antes de apresentar um modelo	

XVII. APÊNDICES

F	Cartão de consulta e glossário	1
	Glossário · Para continuar	

[0] SOBRE ESTA PARTE III

Da análise dos requisitos à exportação do modelo, na ordem em que o trabalho realmente acontece.

Este volume não está organizado por algoritmo, e sim pelo **processo de desenvolvimento**: seis etapas, da definição do problema à publicação do modelo, com uma volta obrigatória da avaliação para o treinamento. A seção 25 apresenta esse ciclo e a Tabela 25.3 mostra onde cada etapa está no volume.

Dentro das etapas de treinamento e avaliação, o conteúdo segue os três tipos de aprendizado, nesta ordem: **supervisionado** nas seções 28 a 33, **não supervisionado** nas 34 a 36 e **por reforço** na 37. Depois disso o mesmo percurso se repete em **aprendizado profundo**, das seções 38 a 42. As seções 43 e 44 fecham com predição, exportação e a conferência honesta do que foi feito.

A atenção, os transformers e toda a IA generativa moderna estão na **Parte IV**, que continua a numeração a partir da seção 45.

As bibliotecas

O aprendizado de máquina clássico usa o **scikit-learn**, que cobre preparação, treino, avaliação e predição com a mesma interface, e o **joblib** para exportar. O aprendizado profundo usa **TensorFlow** com a interface **Keras**, além do **PyTorch**, e o **TensorFlow Lite** na exportação. Nas etapas numéricas aparecem ainda NumPy, pandas, Matplotlib e SciPy, tratados na Parte II.

O que é preciso saber antes

Esta parte assume a Parte I inteira e, da Parte II, as seções 21 (NumPy) e 22 (Matplotlib). Quase todo exemplo daqui manipula arrays e produz um gráfico.

O que instalar

[>] TERMINAL

```
pip install scikit-learn matplotlib pandas
pip install tensorflow-cpu (seções 38 a 41)
pip install torch (seção 42)
```

No Google Colab nada precisa ser instalado, e ainda há GPU disponível, o que reduz o tempo de treino das seções 38 a 42. Os exemplos aqui foram dimensionados para rodar em CPU comum, em segundos ou poucos minutos.

[!] UMA EXCEÇÃO DECLARADA

Todos os códigos deste manual foram executados na geração do arquivo e mostram a saída real, com uma exceção: a **seção 42, de PyTorch**. O ambiente usado na composição não pôde carregar a biblioteca, que exige bibliotecas CUDA mesmo em máquinas sem GPU. Aqueles exemplos aparecem sem painel de saída e seguem a API oficial da versão 2.x.

=====

`|| BLOCO X`

ANÁLISE DOS REQUISITOS

Etapa 1 do processo: entender o problema antes de escrever a primeira linha.

=====

[25] O PROCESSO DE DESENVOLVIMENTO

Um modelo não nasce de um comando de treino. Ele nasce de um processo com seis etapas, e este volume inteiro está organizado nessa ordem.

Programação tradicional recebe **regras** e **dados** e produz respostas. Aprendizado de máquina recebe **dados** e **respostas** e produz as **regras**. Essa inversão é o que torna o desenvolvimento diferente: não se escreve a solução, conduz-se um processo até ela aparecer.



+-- Figura 25.1: as seis etapas do desenvolvimento de um modelo. A seta de retorno entre avaliação e treinamento é a parte que costuma faltar nos diagramas: avaliar não encerra o trabalho, decide se ele recomeça.

Centrado no ser humano

Cada etapa tem uma pergunta que só uma pessoa responde, e nenhum algoritmo decide por ela. Quem convive com o problema define o que conta como acerto; quem opera o sistema convive com os erros; e quem é afetado pela decisão raramente está na sala. Manter essas três perguntas visíveis é o que separa um modelo útil de um modelo que apenas pontua bem.

ETAPA	A MÁQUINA FAZ	A PESSOA DECIDE
1 requisitos	nada	qual é o problema e o que é sucesso
2 preparação	transforma os dados	o que é ruído, o que é erro e o que fica de fora
3 treinamento	ajusta os parâmetros	qual família de modelo e qual custo minimizar
4 avaliação	calcula as métricas	qual métrica importa e qual erro é aceitável
5 predição	responde	quando confiar e quando levar a decisão a uma pessoa
6 exportação	empacota	onde roda, quem mantém e quando desligar

+-- Tabela 25.1: a divisão de trabalho em cada etapa.

Os três tipos de aprendizado

TIPO	O QUE RECEBE	O QUE APRENDE	EXEMPLO DE TELECOM
Supervisionado	amostras com resposta	a função que leva da entrada à resposta	reconhecer a modulação de um sinal
Não supervisionado	amostras sem resposta	grupos, estrutura ou o que é anômalo	agrupar perfis de tráfego
Por reforço	recompensa do ambiente	a política de ação	alocação dinâmica de canal

+-- Tabela 25.2: os três tipos, e a ordem em que aparecem neste volume.

Vocabulário mínimo

- * **Amostra:** uma linha do conjunto de dados.
- * **Atributo** (*feature*): uma coluna de entrada, chamada de *X*.
- * **Rótulo** (*target*): a resposta desejada, chamada de *y*.
- * **Modelo:** a função com parâmetros ajustáveis.
- * **Treino:** ajustar os parâmetros minimizando uma perda.
- * **Inferência:** usar o modelo já treinado em dados novos.
- * **Época:** uma passagem completa pelo conjunto de treino.
- * **Sobreajuste:** decorar o treino e falhar no teste.

+-- #CÓDIGO 25.1 O vocabulário em código -----

A convenção **X** maiúsculo para a matriz e **y** minúsculo para o vetor vale em toda a literatura e em todas as bibliotecas desta parte.

```
import numpy as np

# X: matriz de atributos (amostras nas linhas, atributos nas colunas)
X = np.array([[2.0, 30.0],      # [distancia_km, elevacao_graus]
              [5.0, 10.0],
              [1.0, 45.0],
              [8.0,  5.0]])

# y: a resposta de cada amostra
y = np.array([1, 0, 1, 0])      # 1 = enlace bom, 0 = ruim

print("amostras:", X.shape[0])
print("atributos:", X.shape[1])
print("classes:", np.unique(y))
print("proporção da classe 1:", y.mean())

--- SAÍDA ---
amostras: 4
atributos: 2
classes: [0 1]
proporção da classe 1: 0.5
```

Como este volume está organizado

ETAPA DO PROCESSO	ONDE ESTÁ
1 análise dos requisitos	seções 25 e 26
2 preparação de dados	seção 27
3 e 4 treinamento e avaliação, supervisionado	seções 28 a 33, com scikit-learn
3 e 4 treinamento e avaliação, não supervisionado	seções 34 a 36
3 e 4 treinamento e avaliação, por reforço	seção 37
3 e 4 treinamento e avaliação, aprendizado profundo	seções 38 a 42, com Keras, TensorFlow e PyTorch
5 e 6 predição e exportação	seções 43 e 44

+-- Tabela 25.3: o mapa deste volume.

[i] AS BIBLIOTECAS DE CADA ETAPA

Etapas 2 a 5 do aprendizado de máquina clássico usam o **scikit-learn**, que cobre preparação, treino, avaliação e predição com a mesma interface. O aprendizado profundo usa **TensorFlow** e sua interface **Keras**, e também **PyTorch**. A etapa 6 usa **joblib**, o formato **.keras** e o **TensorFlow Lite**.

-
- >> EXPERIMENTOS PROPOSTOS
- . Desenhe o processo das seis etapas para um problema do seu curso, preenchendo cada caixa.
 - . Para esse mesmo problema, escreva quem responde a pergunta humana de cada etapa da Tabela 25.1.
 - . Classifique em supervisionado, não supervisionado ou reforço: prever consumo de energia, agrupar clientes, ensinar um robô a andar.
 - . Escreva o que seriam **X** e **y** no seu problema, com as unidades de cada coluna.

[26] ANÁLISE DOS REQUISITOS

A primeira etapa, e a única em que a máquina não faz nada. Errar aqui custa mais caro que qualquer escolha de algoritmo.

As perguntas antes do código

1. **Qual é a decisão?** Todo modelo existe para mudar uma decisão que hoje é tomada de outro jeito. Se a decisão não muda, o modelo não é necessário.
2. **Qual é a unidade de previsão?** Uma janela de sinal, um enlace, um cliente, um dia. Isso define o que é uma linha da tabela.
3. **Qual é a resposta certa?** Alguém precisa conseguir produzir o rótulo. Se ninguém sabe rotular, não há problema supervisionado.
4. **Quanto custa cada tipo de erro?** Alarme falso e falha não detectada quase nunca custam igual.
5. **Qual desempenho torna o sistema útil?** Um número combinado antes de treinar, não depois de ver o resultado.
5. **Que dados existem hoje?** Quantidade, qualidade, histórico e permissão de uso.
7. **Quem é afetado se o modelo errar?** E existe um caminho para a pessoa contestar ou revisar a decisão?

+-- #CÓDIGO 26.1 Traduzindo o custo do erro em números -----

Com os custos na mesa, a métrica deixa de ser opinião. Aqui vale mais detectar muito e tolerar alarme falso, porque a falha não vista custa vinte e oito vezes mais.

```
# Manutencao preditiva de um enlace de radio
CUSTO_ALARME_FALSO = 250.0      # deslocamento de equipe sem falha
CUSTO_FALHA_NAO_VISTA = 7000.0 # enlace fora do ar por horas

proporcao_de_falhas = 0.03
enlaces_por_mes = 400

def custo_mensal(revocacao, precisao):
    """Custo esperado dado o desempenho de um detector."""
    falhas = enlaces_por_mes * proporcao_de_falhas
    detectadas = falhas * revocacao
    nao_vistas = falhas - detectadas
    alarmes = detectadas / precisao if precisao > 0 else 0
    falsos = alarmes - detectadas
    return falsos * CUSTO_ALARME_FALSO + nao_vistas * CUSTO_FALHA_NAO_VISTA

print(f"sem modelo (nada é detectado): "
      f"R$ {custo_mensal(0.0, 1.0):>9,.2f}")
print()
print(f"{'revocação':>10}{'precisão':>10}{'custo mensal':>16}")
for revocacao, precisao in [(0.50, 0.80), (0.80, 0.50),
                           (0.90, 0.30), (0.95, 0.20)]:
    print(f"{'revocacao':>10.2f}{'precisao':>10.2f}"
          f"{'custo_mensal(revocacao, precisao):>16,.2f}"))
```

--- SAÍDA ---

sem modelo (nada é detectado): R\$ 84,000.00

revocação	precisão	custo mensal
0.50	0.80	42,375.00
0.80	0.50	19,200.00
0.90	0.30	14,700.00
0.95	0.20	15,600.00

+-- #CÓDIGO 26.2 A linha de base que o modelo precisa vencer -----

Antes de comemorar 70% de acurácia, verifique quanto acerta o palpite mais bobo possível. Em conjuntos desbalanceados essa comparação é obrigatória.

```
import numpy as np

y_teste = np.array([1, 1, 0, 1, 0, 1, 1, 1, 0, 1])

# 1. responder sempre a classe majoritaria
majoritaria = np.bincount(y_teste).argmax()
acerto = (y_teste == majoritaria).mean()

# 2. responder ao acaso, respeitando a proporcao das classes
rng = np.random.default_rng(0)
sorteio = rng.choice([0, 1], size=(2000, len(y_teste)),
                    p=[1 - y_teste.mean(), y_teste.mean()])
acaso = (sorteio == y_teste).mean()

print("classe majoritária:", majoritaria)
print(f"acurácia respondendo sempre {majoritaria}: {acerto:.0%}")
print(f"acurácia sorteando pela proporção: {acaso:.0%}")
print("\nqualquer modelo precisa superar esses números")
```

```
--- SAÍDA ---
classe majoritária: 1
acurácia respondendo sempre 1: 70%
acurácia sorteando pela proporção: 59%

qualquer modelo precisa superar esses números
```

+-- #CÓDIGO 26.3 Registrando os requisitos -----

Uma página escrita antes de treinar evita a pergunta mais constrangedora de uma banca: por que essa métrica? Guarde o arquivo junto com o código.

```
import json

requisitos = {
    "decisao": "enviar equipe de manutenção ao enlace",
    "unidade": "um enlace observado em uma janela de 24 h",
    "rotulo": "houve indisponibilidade nas 48 h seguintes",
    "tipo": "supervisionado, classificação binária",
    "metrica_principal": "revocação da classe falha",
    "meta": 0.85,
    "restricao": "no máximo 3 alarmes falsos por semana",
    "custo_falso_positivo_reais": 250,
    "custo_falso_negativo_reais": 7000,
    "dados_disponiveis": "18 meses de telemetria, 400 enlaces",
    "linha_de_base": "regra atual por limiar de RSSI",
    "revisao_humana": "técnico confirma antes do deslocamento",
}

with open("requisitos.json", "w", encoding="utf-8") as f:
    json.dump(requisitos, f, indent=2, ensure_ascii=False)

print(json.dumps(requisitos, indent=2, ensure_ascii=False)[:430])
print("...")
```

```
--- SAÍDA ---
{
  "decisao": "enviar equipe de manutenção ao enlace",
  "unidade": "um enlace observado em uma janela de 24 h",
  "rotulo": "houve indisponibilidade nas 48 h seguintes",
  "tipo": "supervisionado, classificação binária",
  "metrica_principal": "revocação da classe falha",
  "meta": 0.85,
  "restricao": "no máximo 3 alarmes falsos por semana",
  "custo_falso_positivo_reais": 250,
  "custo_falso_negativo_reais": 7000,
  "dados"
  ...
}
```

[!] QUANDO NÃO USAR APRENDIZADO DE MÁQUINA

Se a regra é conhecida e determinística, escreva a regra. Um `if potencia < -90: return "fora de cobertura"` é mais rápido, mais barato de manter e mais fácil de justificar do que um modelo treinado. Aprendizado de máquina vale quando a regra existe nos dados mas ninguém sabe escrevê-la, e quando há exemplos suficientes para encontrá-la.

[*] ANTES DE PASSAR PARA A ETAPA 2

Você consegue escrever, em uma frase, qual decisão o modelo vai mudar? Existe um número acordado de desempenho mínimo? Você sabe quanto custa cada tipo de erro? Existe uma linha de base sem aprendizado? Se alguma resposta for não, a preparação de dados vai começar no escuro.

>> EXPERIMENTOS PROPOSTOS

- . Escreva o arquivo de requisitos do seu TCC no formato do CÓDIGO 26.3.
- . Estime os custos dos dois tipos de erro no seu problema e descubra qual métrica eles favorecem.
- . Calcule a linha de base majoritária de um conjunto com 95% de amostras da classe 0 e discuta o que significa 95% de acurácia.
- . Liste três problemas do seu laboratório que **não** precisam de aprendizado de máquina.

=====

|| BLOCO XI

PREPARAÇÃO DE DADOS

Etapa 2 do processo: sem isto, nenhuma etapa seguinte é confiável.

=====

[27] PREPARAÇÃO DE DADOS

Etapa 2 do processo, e a que mais consome tempo em um projeto real: escala, codificação, valores faltantes e a montagem de um fluxo que não vaze informação para a avaliação.

+-- #CÓDIGO 27.1 Escala dos atributos -----

Algoritmos baseados em distância (k vizinhos, SVM) e redes neurais exigem escala comparável. Árvores e florestas não se importam.

```
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler

X = np.array([[2.0, 1500.0],
              [5.0, 1200.0],
              [1.0, 1800.0],
              [8.0, 900.0]])

padrao = StandardScaler().fit_transform(X)
faixa = MinMaxScaler().fit_transform(X)

print("original:\n", X)
print("\npadronizado (média 0, desvio 1):\n", padrao.round(3))
print("\nnormalizado (0 a 1):\n", faixa.round(3))
```

```
--- SAÍDA ---

original:
[[2.0e+00 1.5e+03]
 [5.0e+00 1.2e+03]
 [1.0e+00 1.8e+03]
 [8.0e+00 9.0e+02]]

padronizado (média 0, desvio 1):
[[-0.73  0.447]
 [ 0.365 -0.447]
 [-1.095  1.342]
 [ 1.461 -1.342]]

normalizado (0 a 1):
[[0.143 0.667]
 [0.571 0.333]
 [0.    1.    ]
 [1.    0.    ]]
```

+-- #CÓDIGO 27.2 Atributos categóricos -----

Uma coluna de texto vira uma coluna binária por categoria. Usar apenas números inteiros criaria uma ordem que não existe.

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder, LabelEncoder

modulacoes = np.array(["BPSK"], ["QPSK"], ["16QAM"], ["QPSK"]))

codificador = OneHotEncoder(sparse_output=False)
binario = codificador.fit_transform(modulacoes)

print("categorias:", codificador.categories_[0])
print(binario)

rotulos = LabelEncoder()
print("\nrótulos como inteiros:",
      rotulos.fit_transform(["bom", "ruim", "bom", "médio"]))
print("classes:", rotulos.classes_)
```

--- SAÍDA ---

```
categorias: ['16QAM' 'BPSK' 'QPSK']
[[0.  1.  0.]
 [0.  0.  1.]
 [1.  0.  0.]
 [0.  0.  1.]]

rótulos como inteiros: [0 2 0 1]
classes: ['bom' 'médio' 'ruim']
```

+-- #CÓDIGO 27.3 Valores faltantes -----

Descartar a linha inteira é a solução mais simples, mas custa amostras. Preencher com média ou mediana costuma ser melhor quando faltam poucos valores.

```
import numpy as np
from sklearn.impute import SimpleImputer

X = np.array([[2.0, 1500.0],
              [5.0, np.nan],
              [np.nan, 1800.0],
              [8.0, 900.0]])

print("faltantes por coluna:", np.isnan(X).sum(axis=0))

preenchido = SimpleImputer(strategy="mean").fit_transform(X)
print("preenchido com a média:\n", preenchido.round(2))
```

--- SAÍDA ---

```
faltantes por coluna: [1 1]
preenchido com a média:
[[ 2. 1500.]
 [ 5. 1400.]
 [ 5. 1800.]
 [ 8.  900.]]
```

+-- #CÓDIGO 27.4 Pipeline: o jeito certo de encadear -----

O `Pipeline` ajusta o normalizador só com os dados de treino em cada partição. Normalizar tudo antes de dividir é vazamento de informação e infla o resultado.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y)

sem_escala = SVC().fit(X_tr, y_tr)
fluxo = make_pipeline(StandardScaler(), SVC()).fit(X_tr, y_tr)

print(f"SVM sem escala: {sem_escala.score(X_te, y_te):.3f}")
print(f"SVM com escala: {fluxo.score(X_te, y_te):.3f}")
print("validação cruzada:",
      cross_val_score(fluxo, X, y, cv=5).mean().round(3))
```

--- SAÍDA ---

```
SVM sem escala:  0.593
SVM com escala:  0.981
validação cruzada: 0.983
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Monte um pipeline com imputação, escala e classificador.
- . Compare o resultado com e sem `StandardScaler` em um modelo de k vizinhos.
- . Codifique uma coluna categórica do seu conjunto e confira quantas colunas ela virou.
- . Trate os valores faltantes de um conjunto seu de duas formas diferentes e compare o resultado.

=====

|| BLOCO XII

APRENDIZADO DE MÁQUINA: SUPERVISIONADO

Etapas 3 e 4 com rótulo conhecido: treinar, avaliar e interpretar, tudo com scikit-learn.

=====

[28] TREINAMENTO: O CONTRATO DO SCIKIT-LEARN

Uma única interface para dezenas de algoritmos: `fit` para treinar, `predict` para inferir, `score` para avaliar. Trocar de algoritmo é trocar uma linha.

```
[>] O CONTRATO DO SCIKIT-LEARN
modelo = Algoritmo(parametros)
modelo.fit(X_treino, y_treino)
y_previsto = modelo.predict(X_teste)
modelo.score(X_teste, y_teste)
Trocar de algoritmo é trocar a primeira linha.
```

+--- #CÓDIGO 28.1 Primeiro classificador -----

`stratify=y` mantém a proporção das classes nas duas partes. `random_state` torna a divisão reproduzível.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

modelo = KNeighborsClassifier(n_neighbors=5)
modelo.fit(X_tr, y_tr)

print("amostras de treino:", X_tr.shape[0])
print("amostras de teste:", X_te.shape[0])
print(f"acurácia no teste: {modelo.score(X_te, y_te):.3f}")
print("previsão para uma flor:", modelo.predict([X_te[0]]))
print("classe verdadeira:", y_te[0])
```

--- SAÍDA ---

```
amostras de treino: 105
amostras de teste: 45
acurácia no teste: 0.978
previsão para uma flor: [2]
classe verdadeira: 2
```

+-- #CÓDIGO 28.2 Trocando o algoritmo -----

A mesma interface para todos. Comece sempre pelo modelo mais simples que resolve o problema.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

modelos = {
    "k vizinhos": KNeighborsClassifier(),
    "árvore": DecisionTreeClassifier(random_state=0),
    "regressão logística": LogisticRegression(max_iter=1000),
    "floresta aleatória": RandomForestClassifier(random_state=0),
    "SVM": SVC(),
}

for nome, modelo in modelos.items():
    modelo.fit(X_tr, y_tr)
    print(f"{nome:<22}{modelo.score(X_te, y_te):.3f}")
```

--- SAÍDA ---

k vizinhos	0.978
árvore	0.978
regressão logística	0.933
floresta aleatória	0.889
SVM	0.956

+-- #CÓDIGO 28.3 Matriz de confusão e relatório -----

Acurácia sozinha esconde erro: a matriz mostra **quais** classes o modelo confunde.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix,
                             classification_report)

X, y = load_iris(return_X_y=True)
nomes = load_iris().target_names
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

modelo = RandomForestClassifier(random_state=0).fit(X_tr, y_tr)
previsto = modelo.predict(X_te)

print("matriz de confusão (linha = verdadeiro):")
print(confusion_matrix(y_te, previsto))
print()
print(classification_report(y_te, previsto, target_names=nomes))
```

--- SAÍDA ---

matriz de confusão (linha = verdadeiro):

```
[[15  0  0]
 [ 0 14  1]
 [ 0  4 11]]
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.78	0.93	0.85	15
virginica	0.92	0.73	0.81	15
accuracy			0.89	45
macro avg	0.90	0.89	0.89	45
weighted avg	0.90	0.89	0.89	45

+-- #CÓDIGO 28.4 Precisão, revocação e F1 na prática -----

Em manutenção preditiva, deixar de detectar uma falha costuma custar mais que um alarme falso: nesse caso priorize a revocação.

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

# Detector de falha em enlace: 1 = falhou
real = [0, 0, 0, 0, 0, 0, 1, 1, 1, 1]
previsto = [0, 0, 0, 1, 0, 0, 1, 1, 0, 0]

print(f"acurácia: {accuracy_score(real, previsto):.2f}")
print(f"precisão: {precision_score(real, previsto):.2f}")
print(f"revocação: {recall_score(real, previsto):.2f}")
print(f"F1: {f1_score(real, previsto):.2f}")
```

--- SAÍDA ---

```
acurácia: 0.70
precisão: 0.67 (dos alarmes, quantos eram reais)
revocação: 0.50 (das falhas, quantas foram detectadas)
F1: 0.57 (equilíbrio entre as duas)
```

+-- #CÓDIGO 28.5 Validação cruzada -----

Uma única divisão treino e teste pode dar sorte ou azar. A validação cruzada repete o processo e informa a variação.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

X, y = load_iris(return_X_y=True)
modelo = RandomForestClassifier(random_state=0)

notas = cross_val_score(modelo, X, y, cv=5)

print("acurácia em cada partição:", notas.round(3))
print(f"média {notas.mean():.3f} desvio {notas.std():.3f}")

--- SAÍDA ---
acurácia em cada partição: [0.967 0.967 0.933 0.967 1.    ]
média 0.967 desvio 0.021
```

+-- #CÓDIGO 28.6 Ajuste de hiperparâmetros -----

GridSearchCV testa todas as combinações com validação cruzada e guarda a melhor.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)

grade = {"n_neighbors": [1, 3, 5, 7, 9, 15],
         "weights": ["uniform", "distance"]}

busca = GridSearchCV(KNeighborsClassifier(), grade, cv=5)
busca.fit(X, y)

print("melhor combinação:", busca.best_params_)
print(f"melhor acurácia: {busca.best_score_:.3f}")
print("combinações testadas:", len(busca.cv_results_["params"]))

--- SAÍDA ---
melhor combinação: {'n_neighbors': 7, 'weights': 'uniform'}
melhor acurácia: 0.980
combinações testadas: 12
```

+-- #CÓDIGO 28.7 Sobreajuste em uma imagem -----

O treino continua melhorando enquanto o teste estaciona e piora: essa é a assinatura visual do sobreajuste.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=400, noise=0.30, random_state=7)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=7)

profundidades = range(1, 16)
treino, teste = [], []
for d in profundidades:
    m = DecisionTreeClassifier(max_depth=d, random_state=0)
    m.fit(X_tr, y_tr)
    treino.append(m.score(X_tr, y_tr))
    teste.append(m.score(X_te, y_te))

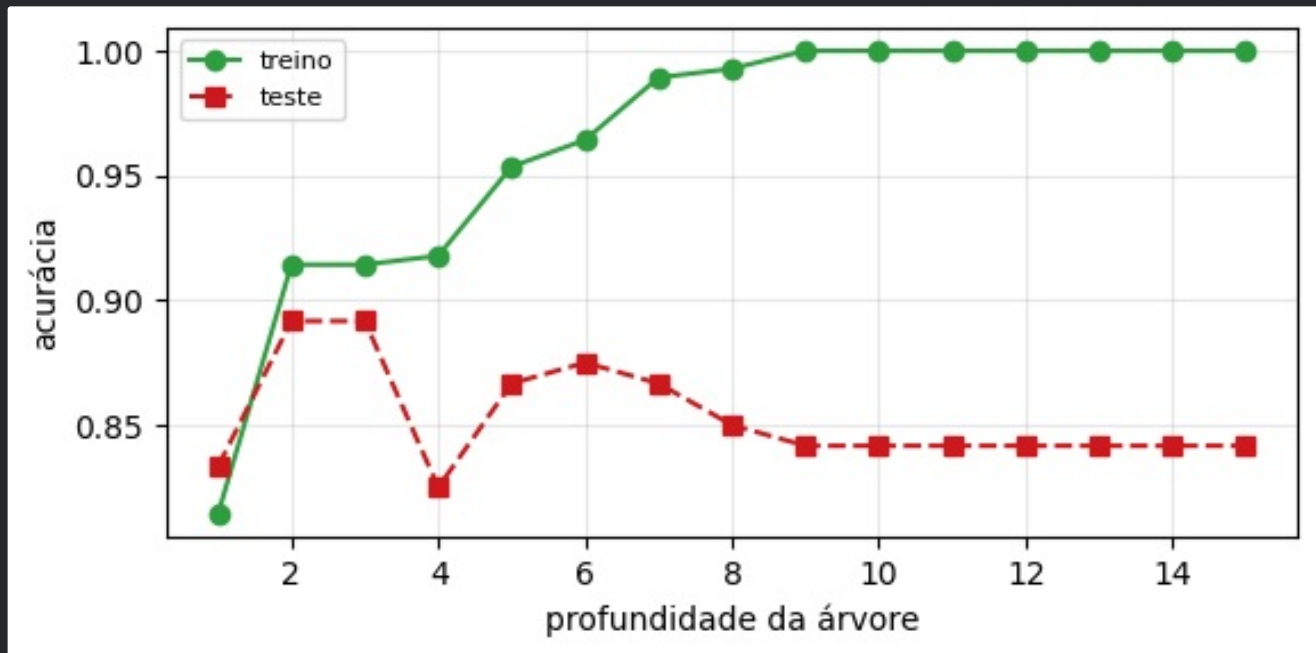
plt.figure(figsize=(6.4, 2.9))
plt.plot(profundidades, treino, "o-", color="#2f9e41",
        label="treino")
plt.plot(profundidades, teste, "s--", color="#cd191e",
        label="teste")
plt.xlabel("profundidade da árvore")
plt.ylabel("acurácia")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("m01.png", dpi=100, bbox_inches="tight")

melhor = profundidades[int(np.argmax(teste))]
print("melhor profundidade no teste:", melhor)
print(f"treino {treino[-1]:.3f} contra teste {teste[-1]:.3f} "
      "na profundidade 15")
```

--- SAÍDA ---

```
melhor profundidade no teste: 2
treino 1.000 contra teste 0.842 na profundidade 15
```

--- FIGURA ---



.....
>> EXPERIMENTOS PROPOSTOS

- . Compare cinco algoritmos no conjunto `load_wine` do scikit-learn.
- . Gere a matriz de confusão do melhor deles e diga quais classes se confundem.
- . Use `GridSearchCV` para ajustar `max_depth` de uma árvore.
- . Explique por que uma acurácia de treino de 100% costuma ser má notícia.

[29] ALGORITMOS CLÁSSICOS E FRONTEIRAS DE DECISÃO

Cada família de algoritmo desenha a separação entre classes de um jeito. Ver a fronteira explica mais do que qualquer tabela de acurácia.

+--- #CÓDIGO 29.1 Naive Bayes -----

Supõe que os atributos são independentes dentro de cada classe. A suposição quase nunca é verdadeira, e mesmo assim o método funciona bem, treina em um passo e serve de linha de base excelente.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
modelo = GaussianNB().fit(X, y)

print("acurácia por validação cruzada:",
      round(cross_val_score(GaussianNB(), X, y, cv=5).mean(), 4))
print("probabilidade das três classes na 1ª amostra:",
      modelo.predict_proba(X[:1]).round(4))
print("média do atributo 0 em cada classe:",
      modelo.theta[:, 0].round(2))
```

--- SAÍDA ---

```
acurácia por validação cruzada: 0.9663
probabilidade das três classes na 1ª amostra: [[1. 0. 0.]]
média do atributo 0 em cada classe: [13.74 12.28 13.15]
```

+--- #CÓDIGO 29.2 Discriminantes linear e quadrático -----

O LDA traça fronteiras retas e o QDA permite fronteiras curvas. Diferente do PCA, o LDA usa os rótulos: ele procura as direções que **separam** as classes.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis)

X, y = load_wine(return_X_y=True)

for nome, modelo in [("LDA", LinearDiscriminantAnalysis()),
                     ("QDA", QuadraticDiscriminantAnalysis())]:
    nota = cross_val_score(modelo, X, y, cv=5).mean()
    print(f"{nome} {nota:.4f}")

# o LDA também reduz dimensão, para no máximo (classes - 1) eixos
reduzido = LinearDiscriminantAnalysis(n_components=2).fit_transform(X, y)
print("\n13 atributos ->", reduzido.shape[1], "eixos discriminantes")
```

--- SAÍDA ---

```
LDA  0.9717
QDA  0.9551
```

```
13 atributos -> 2 eixos discriminantes
```

+--- #CÓDIGO 29.3 SVM e o efeito do kernel -----

O kernel decide o formato da fronteira. O RBF é o padrão razoável; o linear só resolve problemas separáveis por reta.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=600, noise=0.25, random_state=2)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=2, stratify=y)

for kernel in ["linear", "poly", "rbf"]:
    m = make_pipeline(StandardScaler(),
                      SVC(kernel=kernel, degree=3)).fit(X_tr, y_tr)
    print(f"kernel {kernel:<7}{m.score(X_te, y_te):.4f}")

# C controla o quanto o modelo tolera erro no treino
print()
for c in [0.01, 1, 100]:
    m = make_pipeline(StandardScaler(),
                      SVC(C=c)).fit(X_tr, y_tr)
    print(f"C={c:<6}treino {m.score(X_tr, y_tr):.4f}  "
          f"teste {m.score(X_te, y_te):.4f}")
```

--- SAÍDA ---

```
kernel linear 0.8722
kernel poly   0.8444
kernel rbf    0.9333

C=0.01  treino 0.8619  teste 0.8778
C=1      treino 0.9357  teste 0.9333
C=100    treino 0.9405  teste 0.9167
```

+--- #CÓDIGO 29.4 Desenhando as fronteiras -----

A mesma nuvem de pontos, seis separações diferentes. Repare como a árvore corta em degraus retos e o SVM com RBF acompanha a curva.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

X, y = make_moons(n_samples=400, noise=0.25, random_state=2)

modelos = {
    "logística": make_pipeline(StandardScaler(),
                              LogisticRegression()),
    "naive bayes": GaussianNB(),
    "k vizinhos": make_pipeline(StandardScaler(),
                              KNeighborsClassifier(15)),
    "SVM rbf": make_pipeline(StandardScaler(), SVC()),
    "árvore": DecisionTreeClassifier(max_depth=5, random_state=0),
    "floresta": RandomForestClassifier(n_estimators=100,
                                     random_state=0),
}

fig, eixos = plt.subplots(2, 3, figsize=(6.8, 4.0))
for eixo, (nome, modelo) in zip(eixos.flat, modelos.items()):
    modelo.fit(X, y)
    DecisionBoundaryDisplay.from_estimator(
        modelo, X, ax=eixo, alpha=0.55, cmap="RdYlGn",
        response_method="predict")
```

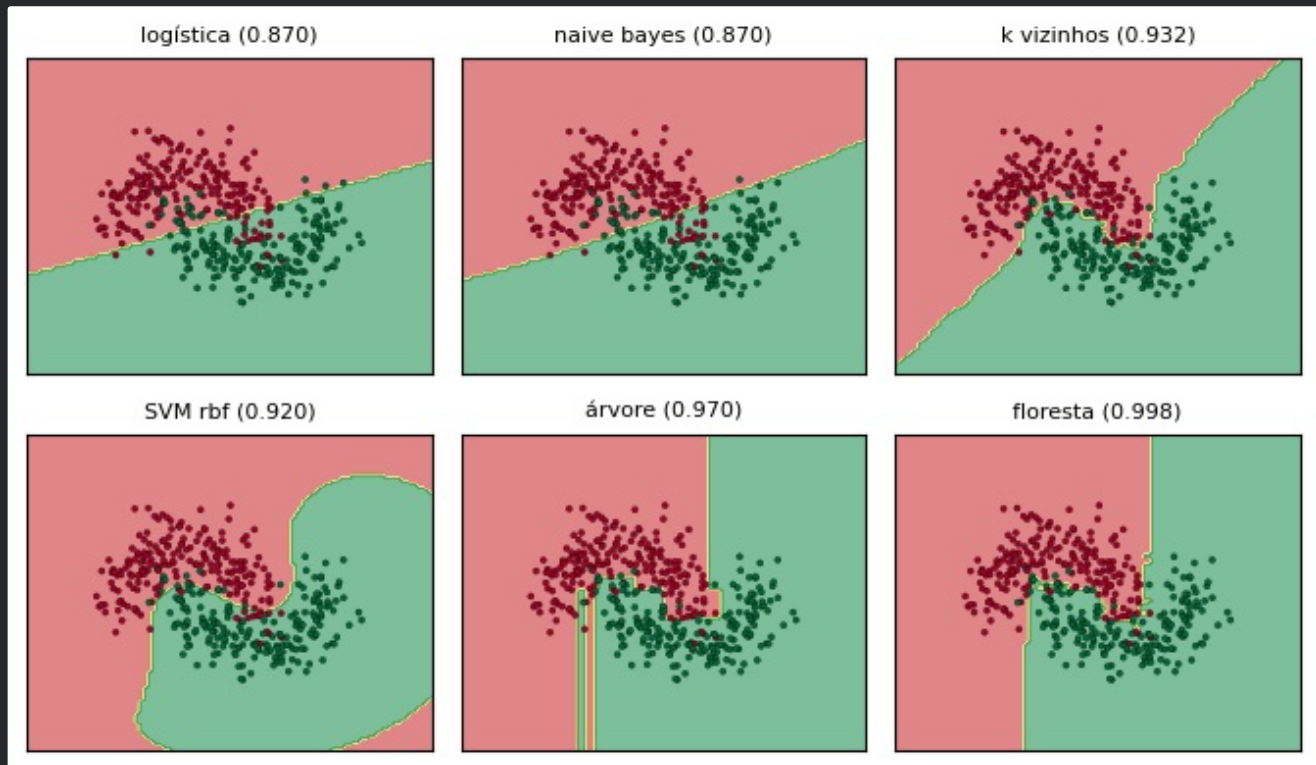
```
eixo.scatter(X[:, 0], X[:, 1], c=y, s=5, cmap="RdYlGn",
             edgecolors="k", linewidths=0.2)
eixo.set_title(f"{nome} ({modelo.score(X, y):.3f})", fontsize=8)
eixo.set_xticks([]); eixo.set_yticks([])

fig.tight_layout()
fig.savefig("c01.png", dpi=100, bbox_inches="tight")
print("seis fronteiras traçadas sobre o mesmo conjunto")
```

--- SAÍDA ---

seis fronteiras traçadas sobre o mesmo conjunto

--- FIGURA ---



+-- #CÓDIGO 29.5 Regressão logística: lendo os coeficientes -----

Em modelo linear com atributos padronizados, o coeficiente é a própria explicação. Essa transparência costuma valer mais que um ponto percentual de acurácia.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

dados = load_breast_cancer()
modelo = make_pipeline(StandardScaler(),
                        LogisticRegression(max_iter=5000))
modelo.fit(dados.data, dados.target)

coef = modelo[-1].coef_[0]
ordem = np.argsort(np.abs(coef))[:, -1][:6]

print("atributos mais influentes (coeficiente padronizado):")
for i in ordem:
    sinal = "aumenta" if coef[i] > 0 else "reduz"
    print(f" {dados.feature_names[i]:<26}{coef[i]:+.3f} "
          f"{sinal} a chance da classe 1")

--- SAÍDA ---
atributos mais influentes (coeficiente padronizado):
worst texture          -1.321  reduz a chance da classe 1
radius error           -1.289  reduz a chance da classe 1
worst radius           -1.027  reduz a chance da classe 1
area error             -0.999  reduz a chance da classe 1
worst area             -0.995  reduz a chance da classe 1
mean concave points    -0.963  reduz a chance da classe 1
```

ALGORITMO	FRONTEIRA	ESCALA IMPORTA	PONTO FORTE
naive bayes	quadrática simples	não	rapidíssimo, poucos dados
logística	reta	sim	coeficientes interpretáveis
LDA / QDA	reta / curva	não	estatisticamente fundamentado
k vizinhos	irregular	sim	sem treino, adapta a qualquer forma
SVM rbf	curva suave	sim	boa em dimensão alta
árvore	degraus retos	não	legível como regras
floresta	degraus suavizados	não	melhor padrão para tabelas

+-- Tabela 29.1: como cada família separa as classes.

-
- >> EXPERIMENTOS PROPOSTOS
- . Trace as fronteiras dos seis modelos em `make_circles` e compare com as luas.
 - . Ajuste `C` e `gamma` do SVM com `GridSearchCV` e observe a fronteira do melhor.
 - . Compare LDA e PCA como redutores antes de um classificador.
 - . Leia os coeficientes de uma logística treinada em dados seus e verifique se o sinal faz sentido físico.

[30] REGRESSÃO E SÉRIES TEMPORAIS

Quando a resposta é um número em vez de uma classe. O contrato do scikit-learn é o mesmo; mudam a métrica e, no caso de série temporal, a forma de separar treino e teste.

Regressão

+-- #CÓDIGO 30.1 Regressão linear -----

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

rng = np.random.default_rng(3)
d = np.linspace(1, 50, 60)
rss_i = -40 - 20 * np.log10(d) + rng.normal(0, 2, d.size)

X = np.log10(d).reshape(-1, 1)      # uma coluna
modelo = LinearRegression().fit(X, rss_i)
previsto = modelo.predict(X)

print(f"modelo: RSSI = {modelo.coef_[0]:.2f}*log10(d) "
      f"{modelo.intercept_:+.2f}")
print(f"expoente de perda n = {-modelo.coef_[0] / 10:.2f}")
print(f"erro absoluto médio: {mean_absolute_error(rss_i, previsto):.2f} dB")
print(f"R²: {r2_score(rss_i, previsto):.4f}")

--- SAÍDA ---
modelo: RSSI = -19.83*log10(d) -40.25
expoente de perda n = 1.98
erro absoluto médio: 1.57 dB
R²: 0.9245
```

+-- #CÓDIGO 30.2 Comparando regressores -----

Para uma relação não linear, o modelo linear não tem chance. Ver a métrica de vários candidatos custa poucas linhas.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(5)
X = rng.uniform(-3, 3, size=(300, 1))
y = np.sin(X).ravel() + rng.normal(0, 0.2, 300)

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=5)

for nome, modelo in [("linear", LinearRegression()),
                     ("ridge", Ridge(alpha=1.0)),
                     ("árvore", DecisionTreeRegressor(max_depth=4,
                                                         random_state=0)),
                     ("floresta", RandomForestRegressor(
                         n_estimators=100, random_state=0))]:
    modelo.fit(X_tr, y_tr)
    erro = mean_squared_error(y_te, modelo.predict(X_te))
    print(f"{nome:<10} EQM {erro:.4f}")

--- SAÍDA ---
linear      EQM 0.2177
ridge       EQM 0.2179
árvore      EQM 0.0521
floresta    EQM 0.0453
```

Séries temporais

Uma série temporal vira um problema de regressão comum assim que se aplica uma janela: os valores passados viram atributos e o próximo valor vira o rótulo. O que não pode mudar é a ordem, e é aí que a maioria dos trabalhos escorrega.

+-- #CÓDIGO 30.3 Transformando série em tabela -----

Esse janelamento transforma previsão em regressão comum, e abre a série para qualquer modelo das seções anteriores.

```
import numpy as np

serie = np.array([10, 12, 13, 15, 14, 16, 18, 17, 19, 21])

def janelar(serie, janela):
    """Cria X com 'janela' valores passados e y com o seguinte."""
    X = np.array([serie[i:i + janela]
                  for i in range(len(serie) - janela)])
    y = np.array(serie[janela:])
    return X, y

X, y = janelar(serie, janela=3)
print("X (três valores anteriores):\n", X)
print("y (próximo valor):", y)
print("formato:", X.shape, y.shape)
```

```
--- SAÍDA ---
X (três valores anteriores):
[[10 12 13]
 [12 13 15]
 [13 15 14]
 [15 14 16]
 [14 16 18]
 [16 18 17]
 [18 17 19]]
y (próximo valor): [15 14 16 18 17 19 21]
formato: (7, 3) (7,)
```

+-----

--- #CÓDIGO 30.4 Previsão contra a linha de base -----

Em série temporal, a linha de base obrigatória é repetir o último valor. Neste caso a floresta ficou **atrás** dela, o que é comum em série suave: sem calcular a persistência, seria fácil apresentar o modelo como bom resultado.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

rng = np.random.default_rng(11)
t = np.arange(400)
serie = (20 + 5 * np.sin(2 * np.pi * t / 50)
         + 0.01 * t + rng.normal(0, 0.6, t.size))

janela = 20
X = np.array([serie[i:i + janela]
              for i in range(len(serie) - janela)])
y = serie[janela:]

corte = int(len(X) * 0.75)          # divisao temporal, sem sortear
X_tr, X_te, y_tr, y_te = X[:corte], X[corte:], y[:corte], y[corte:]

persistencia = X_te[:, -1]          # "amanha e igual a hoje"
modelo = RandomForestRegressor(n_estimators=200,
                              random_state=0).fit(X_tr, y_tr)
previsto = modelo.predict(X_te)

print(f"erro da persistência: {mean_absolute_error(y_te, persistencia):.4f}")
print(f"erro do modelo:      {mean_absolute_error(y_te, previsto):.4f}")

plt.figure(figsize=(6.6, 2.8))
plt.plot(y_te, color="#101214", linewidth=1, label="real")
plt.plot(previsto, color="#2f9e41", linewidth=1, label="previsto")
plt.plot(persistencia, color="#cd191e", linewidth=0.8,
         linestyle=":", label="persistência")
plt.xlabel("amostra do conjunto de teste")
plt.legend(fontsize=8, ncol=3)
plt.grid(alpha=0.3)
plt.savefig("a02.png", dpi=100, bbox_inches="tight")
```

--- SAÍDA ---

```
erro da persistência: 0.7786
erro do modelo:      0.8168
```

--- FIGURA ---



+-- #CÓDIGO 30.5 Validação que respeita o tempo -----

Sortear amostras em série temporal treina com o futuro e testa no passado. `TimeSeriesSplit` mantém a ordem: cada dobra testa logo depois do treino.

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit, cross_val_score
from sklearn.ensemble import RandomForestRegressor

rng = np.random.default_rng(11)
t = np.arange(400)
serie = 20 + 5 * np.sin(2 * np.pi * t / 50) + rng.normal(0, 0.6, 400)
janela = 20
X = np.array([serie[i:i + janela] for i in range(len(serie) - janela)])
y = serie[janela:]

divisor = TimeSeriesSplit(n_splits=4)
for i, (treino, teste) in enumerate(divisor.split(X), 1):
    print(f"dobra {i}: treino [0:{treino[-1] + 1}] "
          f"teste [{teste[0]}:{teste[-1] + 1}]")

notas = cross_val_score(RandomForestRegressor(n_estimators=100,
                                              random_state=0),
                        X, y, cv=divisor,
                        scoring="neg_mean_absolute_error")
print("\nerro médio por dobra:", (-notas).round(4))
```

--- SAÍDA ---

```
dobra 1: treino [0:76] teste [76:152]
dobra 2: treino [0:152] teste [152:228]
dobra 3: treino [0:228] teste [228:304]
dobra 4: treino [0:304] teste [304:380]
```

```
erro médio por dobra: [0.6313 0.4658 0.5963 0.5429]
```

[!] NUNCA SORTEIE UMA SÉRIE TEMPORAL

Separar treino e teste ao acaso coloca o futuro no treino e o passado no teste. O resultado sobe, o modelo parece ótimo e não funciona em operação. Divida sempre por um corte no tempo, e valide com `TimeSeriesSplit`.

.....
>> EXPERIMENTOS PROPOSTOS

- . Ajuste uma regressão para um conjunto de medidas do seu laboratório e informe o R^2 .
- . Compare regressão linear, árvore e floresta no mesmo conjunto e explique a diferença.
- . Faça a previsão de uma série sua e verifique se ela vence a persistência.
- . Mostre, com um exemplo, o quanto a validação sorteada infla o resultado em série temporal.

[31] MODELOS EM CONJUNTO

Combinar vários modelos fracos costuma superar qualquer um deles sozinho. É a família que mais vence competições com dados tabulares.

ESTRATÉGIA	IDEIA	CLASSE TÍPICA
votação	modelos diferentes votam	VotingClassifier
bagging	o mesmo modelo em amostras sorteadas	BaggingClassifier
floresta	bagging de árvores com atributos sorteados	RandomForestClassifier
boosting	cada modelo corrige o erro do anterior	HistGradientBoostingClassifier
empilhamento	um modelo aprende a combinar os outros	StackingClassifier

+-- Tabela 31.1: as cinco estratégias.

+-- #CÓDIGO 31.1 Votação entre modelos diferentes -----

voting="soft" soma as probabilidades, o que costuma superar a votação simples por maioria.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y)

base = [
    ("logística", make_pipeline(StandardScaler(),
                               LogisticRegression(max_iter=2000))),
    ("árvore", DecisionTreeClassifier(random_state=0)),
    ("vizinhos", make_pipeline(StandardScaler(),
                               KNeighborsClassifier())),
]

for nome, modelo in base:
    modelo.fit(X_tr, y_tr)
    print(f"{nome:<12}{modelo.score(X_te, y_te):.4f}")

votacao = VotingClassifier(base, voting="soft").fit(X_tr, y_tr)
print(f"'votação':<12>{votacao.score(X_te, y_te):.4f}")
```

--- SAÍDA ---

```
logística    0.9815
árvore       0.9444
vizinhos     0.9259
votação      0.9815
```

+-- #CÓDIGO 31.2 Bagging: muitas árvores em amostras sorteadas -----

A árvore sozinha varia muito conforme a amostra. Sortear amostras e votar reduz essa variância, que é exatamente o que bagging faz.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
```

```
X, y = load_wine(return_X_y=True)
```

```
modelos = {
    "árvore única": DecisionTreeClassifier(random_state=0),
    "bagging de 100": BaggingClassifier(
        DecisionTreeClassifier(random_state=0),
        n_estimators=100, random_state=0),
    "floresta de 100": RandomForestClassifier(
        n_estimators=100, random_state=0),
}
```

```
for nome, modelo in modelos.items():
    notas = cross_val_score(modelo, X, y, cv=5)
    print(f"{nome:<18}{notas.mean():.4f} "
          f"(desvio {notas.std():.4f})")
```

--- SAÍDA ---

```
árvore única      0.8876 (desvio 0.0395)
bagging de 100    0.9554 (desvio 0.0333)
floresta de 100   0.9833 (desvio 0.0222)
```

+-- #CÓDIGO 31.3 Importância dos atributos -----

A importância interna da floresta favorece atributos com muitos valores distintos. Para decisão séria, confira com a importância por permutação do CÓDIGO 31.6.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier

dados = load_wine()
X, y, nomes = dados.data, dados.target, dados.feature_names

floresta = RandomForestClassifier(n_estimators=300,
                                random_state=0).fit(X, y)
ordem = np.argsort(floresta.feature_importances_)[::-1][:8]

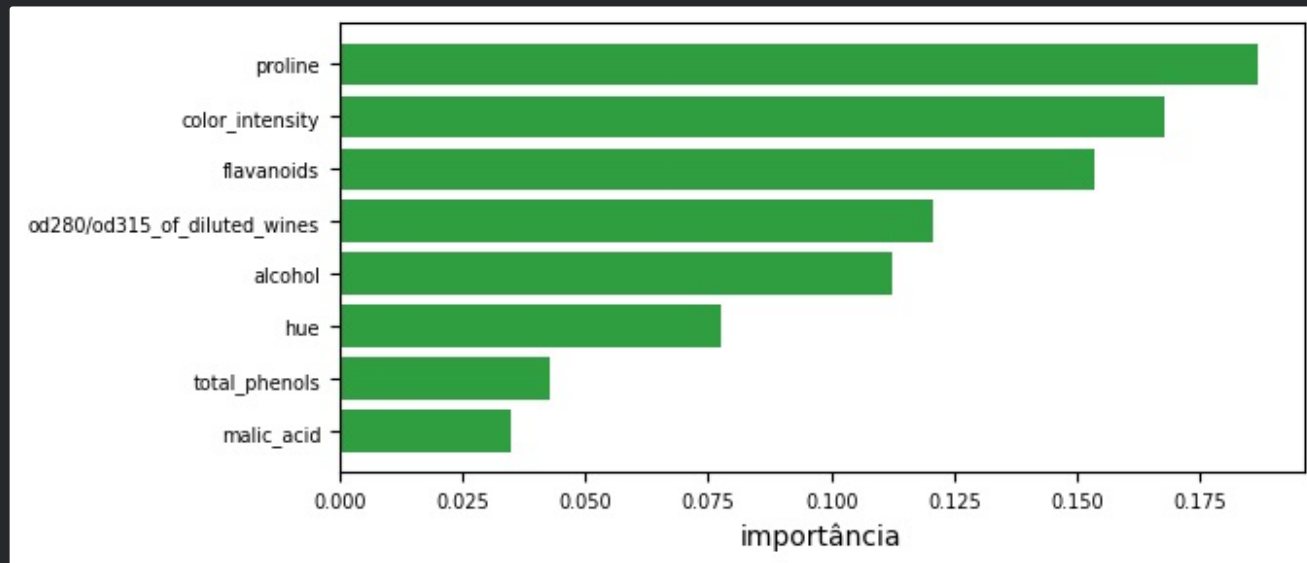
plt.figure(figsize=(6.4, 3.0))
plt.barh([nomes[i] for i in ordem][::-1],
         floresta.feature_importances_[ordem][::-1],
         color="#2f9e41")
plt.xlabel("importância")
plt.tick_params(labelsize=7)
plt.savefig("e01.png", dpi=100, bbox_inches="tight")

for i in ordem[:5]:
    print(f"{nomes[i]:<28}{floresta.feature_importances_[i]:.4f}")
```

--- SAÍDA ---

```
proline           0.1868
color_intensity   0.1677
flavanoids        0.1536
od280/od315_of_diluted_wines0.1208
alcohol           0.1123
```

--- FIGURA ---



+-- #CÓDIGO 31.4 Boosting -----

Bagging treina em paralelo e reduz variância. Boosting treina em sequência, cada modelo focado no que o anterior errou, e reduz viés.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import (RandomForestClassifier,
                              HistGradientBoostingClassifier,
                              AdaBoostClassifier)

X, y = load_wine(return_X_y=True)

modelos = {
    "floresta": RandomForestClassifier(n_estimators=200,
                                     random_state=0),
    "gradiente (hist)": HistGradientBoostingClassifier(
        max_iter=200, random_state=0),
    "adaboost": AdaBoostClassifier(n_estimators=200,
                                   random_state=0),
}

for nome, modelo in modelos.items():
    notas = cross_val_score(modelo, X, y, cv=5)
    print(f"{nome:<20}{notas.mean():.4f}")
```

--- SAÍDA ---

```
floresta          0.9722
gradiente (hist)  0.9665
adaboost          0.9278
```

+-- #CÓDIGO 31.5 Empilhamento -----

O modelo final aprende **quando** confiar em cada um dos modelos de base. Custa mais treino e costuma render os últimos pontos percentuais.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import StackingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)

pilha = StackingClassifier(
    estimators=[
        ("floresta", RandomForestClassifier(n_estimators=100,
                                           random_state=0)),
        ("svm", make_pipeline(StandardScaler(),
                             SVC(probability=True))),
    ],
    final_estimator=LogisticRegression(max_iter=1000),
    cv=5,
)

print("empilhamento:",
      round(cross_val_score(pilha, X, y, cv=5).mean(), 4))
```

--- SAÍDA ---

```
empilhamento: 0.9833
```

+-- #CÓDIGO 31.6 Importância por permutação -----

Embaralhar uma coluna e medir quanto o desempenho cai funciona para qualquer modelo, inclusive redes neurais, e é medido no conjunto de teste.

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance

dados = load_wine()
X_tr, X_te, y_tr, y_te = train_test_split(
    dados.data, dados.target, test_size=0.3, random_state=1,
    stratify=dados.target)

modelo = RandomForestClassifier(n_estimators=200,
                               random_state=0).fit(X_tr, y_tr)
resultado = permutation_importance(modelo, X_te, y_te,
                                   n_repeats=20, random_state=0)

ordem = resultado.importances_mean.argsort()[::-1][:5]
print("queda de acurácia ao embaralhar cada atributo:")
for i in ordem:
    print(f" {dados.feature_names[i]:<28}"
          f"{resultado.importances_mean[i]:.4f} "
          f"± {resultado.importances_std[i]:.4f}")
```

--- SAÍDA ---

```
queda de acurácia ao embaralhar cada atributo:
proline                0.0944 ± 0.0356
color_intensity        0.0917 ± 0.0271
flavonoids             0.0556 ± 0.0255
alcohol               0.0463 ± 0.0180
malic_acid            0.0065 ± 0.0088
```

>> EXPERIMENTOS PROPOSTOS

- . Compare floresta, boosting e empilhamento em um conjunto seu e registre o tempo de treino de cada um.
- . Verifique se os cinco atributos mais importantes pela floresta coincidem com os da permutação.
- . Treine um modelo usando só os cinco atributos mais importantes e compare com o modelo completo.
- . Explique por que bagging ajuda pouco em modelos já estáveis, como a regressão logística.

[32] AVALIAÇÃO: MÉTRICAS, LIMIAR E CONFIANÇA

Um classificador não devolve uma classe, devolve uma probabilidade. Escolher o ponto de corte é uma decisão de engenharia, não do algoritmo.

+++ #CÓDIGO 32.1 Curva ROC e AUC -----

A AUC resume o desempenho em todos os limiares possíveis. 0.5 é o acaso, 1.0 é a separação perfeita.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_curve, roc_auc_score

X, y = make_classification(n_samples=2000, n_informative=6,
                          weights=[0.85, 0.15], random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=3, stratify=y)

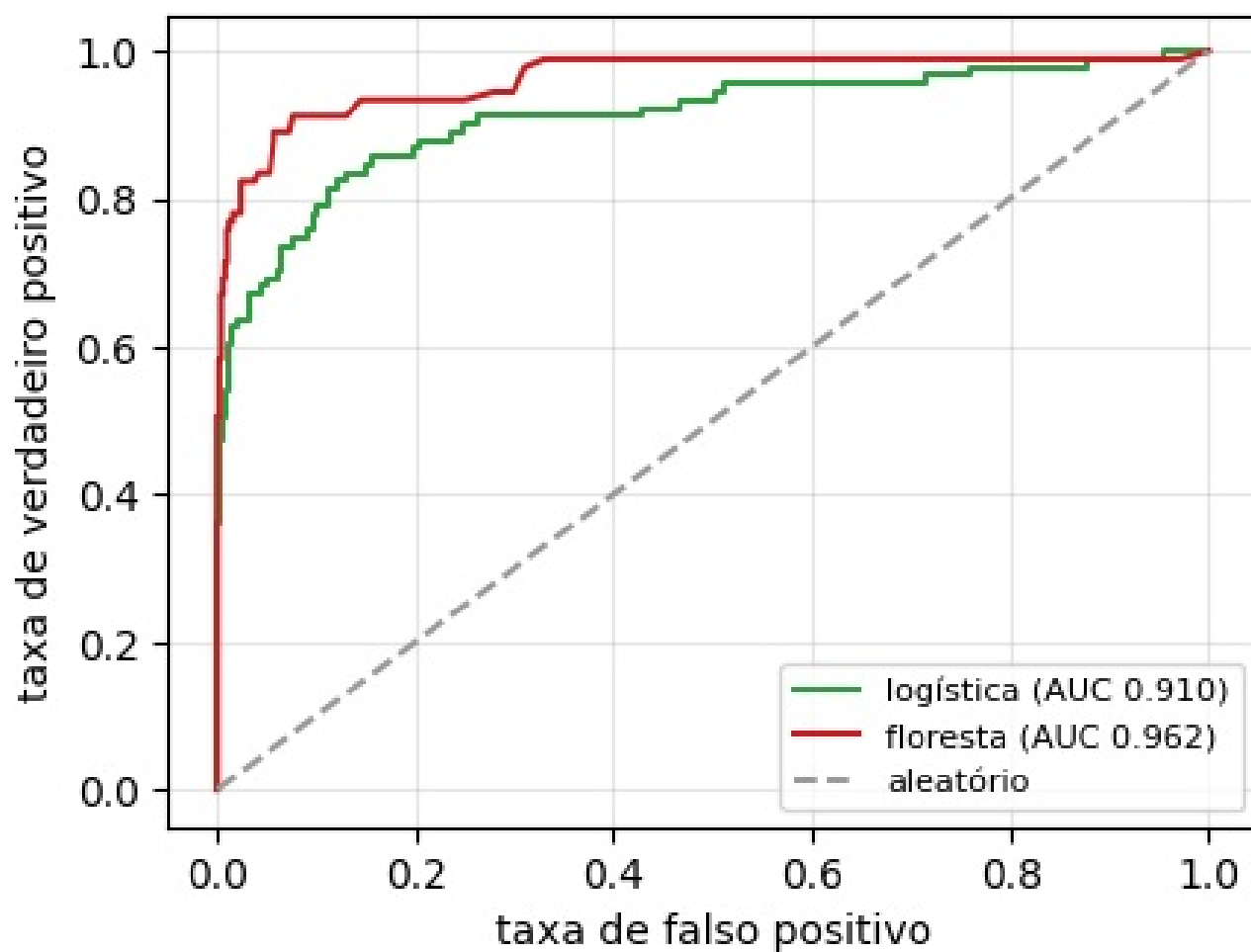
plt.figure(figsize=(4.8, 3.6))
for nome, modelo, cor in [
    ("logística", LogisticRegression(max_iter=2000), "#2f9e41"),
    ("floresta", RandomForestClassifier(n_estimators=200,
                                      random_state=0),
     "#cd191e")]:
    modelo.fit(X_tr, y_tr)
    prob = modelo.predict_proba(X_te)[: , 1]
    fpr, tpr, _ = roc_curve(y_te, prob)
    auc = roc_auc_score(y_te, prob)
    plt.plot(fpr, tpr, color=cor, label=f"{nome} (AUC {auc:.3f})")
    print(f"{nome}<12}AUC {auc:.4f}")

plt.plot([0, 1], [0, 1], "--", color="#999", label="aleatório")
plt.xlabel("taxa de falso positivo")
plt.ylabel("taxa de verdadeiro positivo")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("r01.png", dpi=100, bbox_inches="tight")
```

--- SAÍDA ---

```
logística   AUC 0.9097
floresta    AUC 0.9620
```

--- FIGURA ---



+--- #CÓDIGO 32.2 Curva precisão contra revocação +-----

Com classes muito desbalanceadas, esta curva é mais informativa que a ROC: ela mostra o preço de aumentar a revocação.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_curve, average_precision_score

X, y = make_classification(n_samples=2000, n_informative=6,
                          weights=[0.95, 0.05], random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=3, stratify=y)

modelo = LogisticRegression(max_iter=2000).fit(X_tr, y_tr)
prob = modelo.predict_proba(X_te)[:, 1]
precisao, revocacao, limiares = precision_recall_curve(y_te, prob)

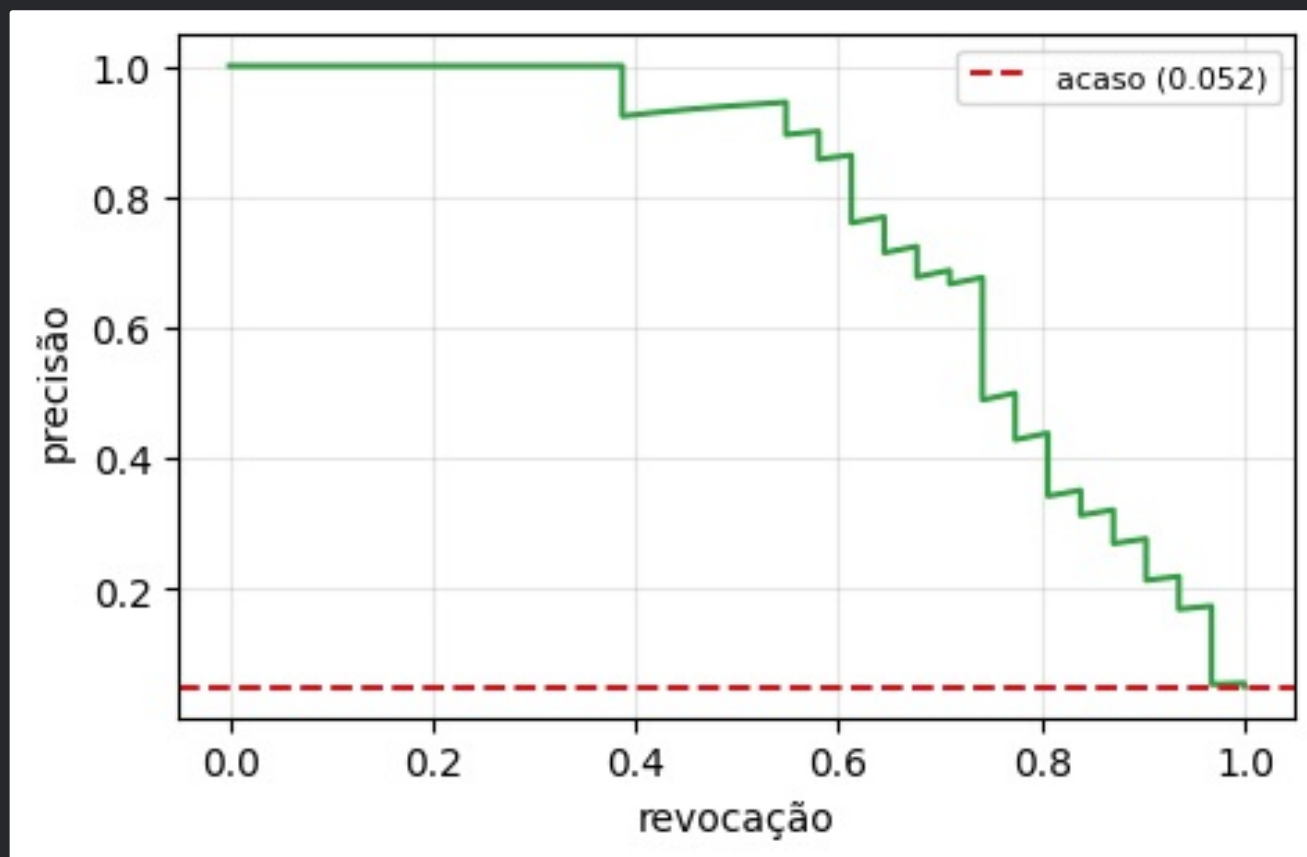
plt.figure(figsize=(5.2, 3.2))
plt.plot(revocacao, precisao, color="#2f9e41")
plt.axhline(y_te.mean(), color="#cd191e", linestyle="--",
            label=f"acaso ({y_te.mean():.3f})")
plt.xlabel("revocação"); plt.ylabel("precisão")
plt.legend(fontsize=8); plt.grid(alpha=0.3)
plt.savefig("r02.png", dpi=100, bbox_inches="tight")

print("proporção da classe rara:", round(y_te.mean(), 4))
print("precisão média:", round(average_precision_score(y_te, prob), 4))
```

--- SAÍDA ---

proporção da classe rara: 0.0517
precisão média: 0.7627

--- FIGURA ---



+- #CÓDIGO 32.3 Escolhendo o limiar pelo custo -----

O limiar 0.5 é apenas uma convenção. Com os custos reais na mesa, a escolha vira uma conta simples.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

X, y = make_classification(n_samples=2000, n_informative=6,
                           weights=[0.9, 0.1], random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=3, stratify=y)

modelo = LogisticRegression(max_iter=2000).fit(X_tr, y_tr)
prob = modelo.predict_proba(X_te)[:, 1]

CUSTO_FALSO_ALARME = 1      # deslocamento de equipe a toa
CUSTO_FALHA_NAO_VISTA = 20 # enlace fora do ar

print(f"{'limiar':>7}{ 'falso+':>8}{ 'falso-':>8}{ 'custo':>8}")
melhor = (None, 1e9)
for limiar in np.arange(0.05, 0.96, 0.10):
    previsto = (prob >= limiar).astype(int)
    vn, fp, fn, vp = confusion_matrix(y_te, previsto).ravel()
    custo = fp * CUSTO_FALSO_ALARME + fn * CUSTO_FALHA_NAO_VISTA
    print(f"{'limiar':>7.2f}{ 'fp':>8}{ 'fn':>8}{ 'custo':>8}")
    if custo < melhor[1]:
        melhor = (limiar, custo)

print(f"\nmelhor limiar: {melhor[0]:.2f} (custo {melhor[1]})")
print("o padrão 0.50 quase nunca é o ótimo")
```

--- SAÍDA ---

limiar	falso+	falso-	custo
0.05	138	5	238
0.15	58	9	238
0.25	33	14	313
0.35	15	18	375
0.45	10	21	430
0.55	6	23	466
0.65	2	30	602
0.75	0	31	620
0.85	0	35	700
0.95	0	44	880

melhor limiar: 0.05 (custo 238)
o padrão 0.50 quase nunca é o ótimo

+- #CÓDIGO 32.4 As probabilidades são confiáveis? -----

Um modelo bem calibrado que diz 70% acerta 70% das vezes. Florestas costumam ser mal calibradas, e isso importa quando a probabilidade vira decisão.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV, calibration_curve

X, y = make_classification(n_samples=3000, n_informative=6,
                           random_state=5)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=5, stratify=y)

cru = RandomForestClassifier(n_estimators=100,
                             random_state=0).fit(X_tr, y_tr)

calibrado = CalibratedClassifierCV(
    RandomForestClassifier(n_estimators=100, random_state=0),
    cv=5, method="isotonic").fit(X_tr, y_tr)
```

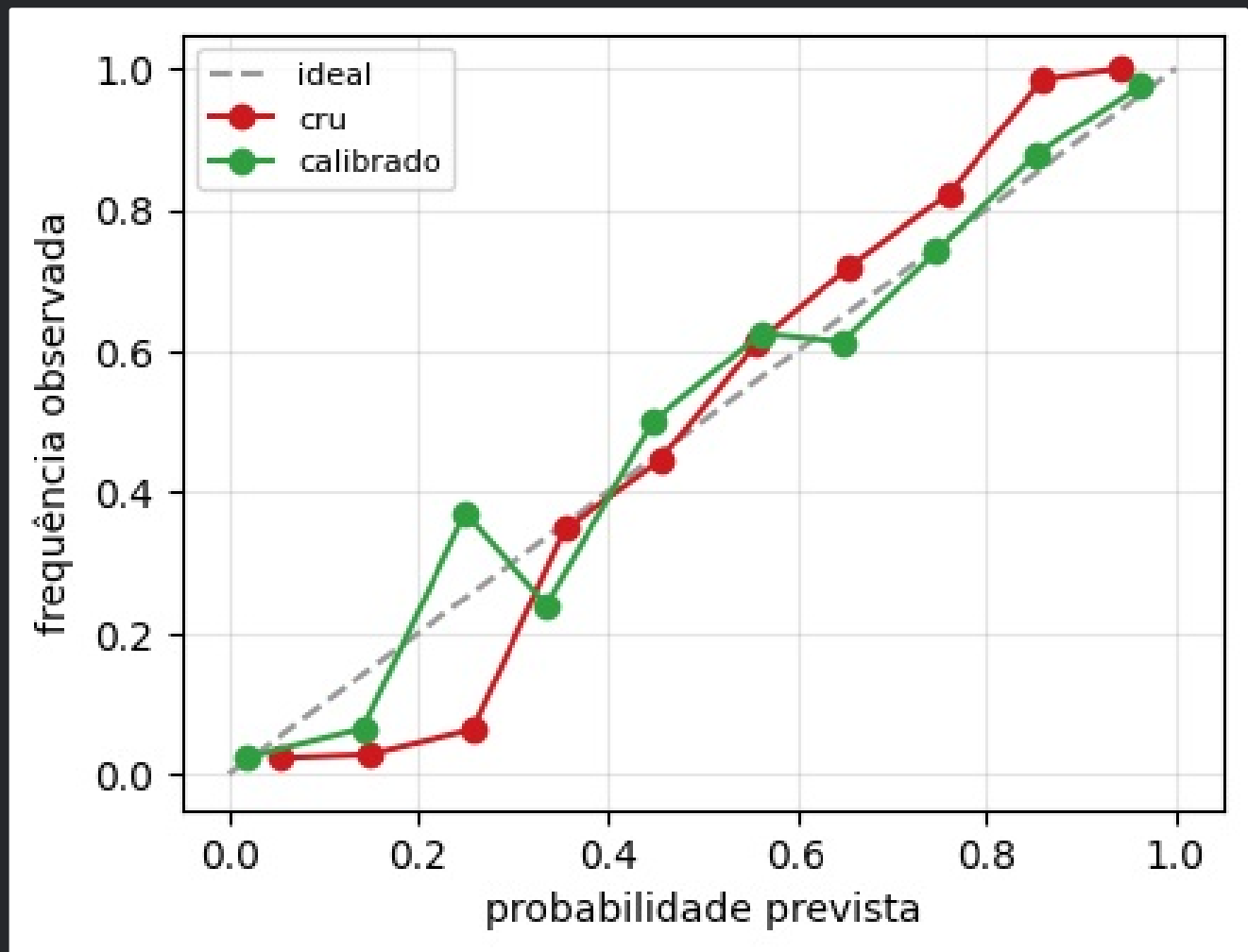
```
plt.figure(figsize=(4.8, 3.6))
plt.plot([0, 1], [0, 1], "--", color="#999", label="ideal")
for nome, modelo, cor in [("cru", cru, "#cd191e"),
                          ("calibrado", calibrado, "#2f9e41")]:
    prob = modelo.predict_proba(X_te)[: , 1]
    real, previsto = calibration_curve(y_te, prob, n_bins=10)
    plt.plot(previsto, real, "o-", color=cor, label=nome)
    erro = np.abs(real - previsto).mean()
    print(f"{nome:<12}erro médio de calibração {erro:.4f}")

plt.xlabel("probabilidade prevista")
plt.ylabel("frequência observada")
plt.legend(fontsize=8); plt.grid(alpha=0.3)
plt.savefig("r03.png", dpi=100, bbox_inches="tight")
```

--- SAÍDA ---

```
cru          erro médio de calibração 0.0727
calibrado    erro médio de calibração 0.0495
```

--- FIGURA ---



+-- #CÓDIGO 32.5 Intervalo de confiança por reamostragem -----

Relatar 0.94 sem intervalo esconde que, com 54 amostras de teste, o número real pode estar bem longe disso.

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

modelo = RandomForestClassifier(n_estimators=200,
                               random_state=0).fit(X_tr, y_tr)
previsto = modelo.predict(X_te)

rng = np.random.default_rng(0)
amostras = []
for _ in range(2000):
    indices = rng.integers(0, len(y_te), len(y_te))
    amostras.append(accuracy_score(y_te[indices], previsto[indices]))

baixo, alto = np.percentile(amostras, [2.5, 97.5])
print(f"acurácia pontual: {accuracy_score(y_te, previsto):.4f}")
print(f"intervalo de 95%: [{baixo:.4f}, {alto:.4f}]")
print(f"amostras de teste: {len(y_te)}")

--- SAÍDA ---
acurácia pontual: 1.0000
intervalo de 95%: [1.0000, 1.0000]
amostras de teste: 54
```

+-- #CÓDIGO 32.6 Dois modelos são mesmo diferentes? -----

Duas médias diferentes podem ser apenas ruído de amostragem. Repetir a validação e comparar com um teste pareado transforma a impressão em evidência.

```
import numpy as np
from scipy import stats
from sklearn.datasets import load_wine
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
divisor = RepeatedStratifiedKFold(n_splits=5, n_repeats=6,
                                  random_state=0)

a = cross_val_score(RandomForestClassifier(n_estimators=100,
                                          random_state=0),
                    X, y, cv=divisor)
b = cross_val_score(make_pipeline(StandardScaler(),
                                  LogisticRegression(max_iter=2000)),
                    X, y, cv=divisor)

print(f"floresta: {a.mean():.4f} ± {a.std():.4f}")
print(f"logística: {b.mean():.4f} ± {b.std():.4f}")

t, p = stats.ttest_rel(a, b)
print(f"nteste t pareado: t={t:.3f} p={p:.4f}")
print(f"diferença significativa? , "sim" if p < 0.05 else "não")

--- SAÍDA ---
floresta: 0.9803 ± 0.0148
logística: 0.9832 ± 0.0171

teste t pareado: t=-0.790 p=0.4360
diferença significativa? não
```

.....

>> EXPERIMENTOS PROPOSTOS

- . Trace a curva ROC do seu modelo e informe a AUC.
- . Defina os custos do seu problema e escolha o limiar pelo custo total.
- . Verifique se as probabilidades do seu modelo estão calibradas.
- . Calcule o intervalo de confiança da acurácia do seu último trabalho.

[33] INTERPRETABILIDADE

Um modelo que acerta sem explicar é difícil de defender em banca e arriscado em operação. Estas ferramentas abrem a caixa preta sem trocar de algoritmo.

Existem dois caminhos: usar um modelo transparente por construção, como a regressão linear e a árvore rasa, ou aplicar uma técnica **agnóstica**, que explica qualquer modelo olhando apenas as entradas e as saídas. As desta seção são do segundo tipo, com exceção da primeira.

+-- #CÓDIGO 33.1 Árvore como conjunto de regras -----

Uma árvore rasa vira texto que qualquer pessoa da área consegue conferir. Serve como documentação do que o modelo aprendeu.

```
from sklearn.datasets import load_wine
from sklearn.tree import DecisionTreeClassifier, export_text

dados = load_wine()
arvore = DecisionTreeClassifier(max_depth=2,
                               random_state=0).fit(dados.data,
                                                    dados.target)

print(export_text(arvore, feature_names=list(dados.feature_names),
              class_names=list(dados.target_names)))
print("acurácia com apenas duas perguntas:",
      round(arvore.score(dados.data, dados.target), 4))
```

--- SAÍDA ---

```
|--- proline <= 755.00
| |--- od280/od315_of_diluted_wines <= 2.11
| | |--- class: class_2
| |--- od280/od315_of_diluted_wines > 2.11
| | |--- class: class_1
|--- proline > 755.00
| |--- flavanoids <= 2.17
| | |--- class: class_2
| |--- flavanoids > 2.17
| | |--- class: class_0
```

acurácia com apenas duas perguntas: 0.9213

+-----

+-- #CÓDIGO 33.2 Dependência parcial -----

A curva responde: mantendo o resto igual, o que acontece com a previsão quando este atributo varia? É a leitura mais usada em artigo aplicado.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.inspection import PartialDependenceDisplay

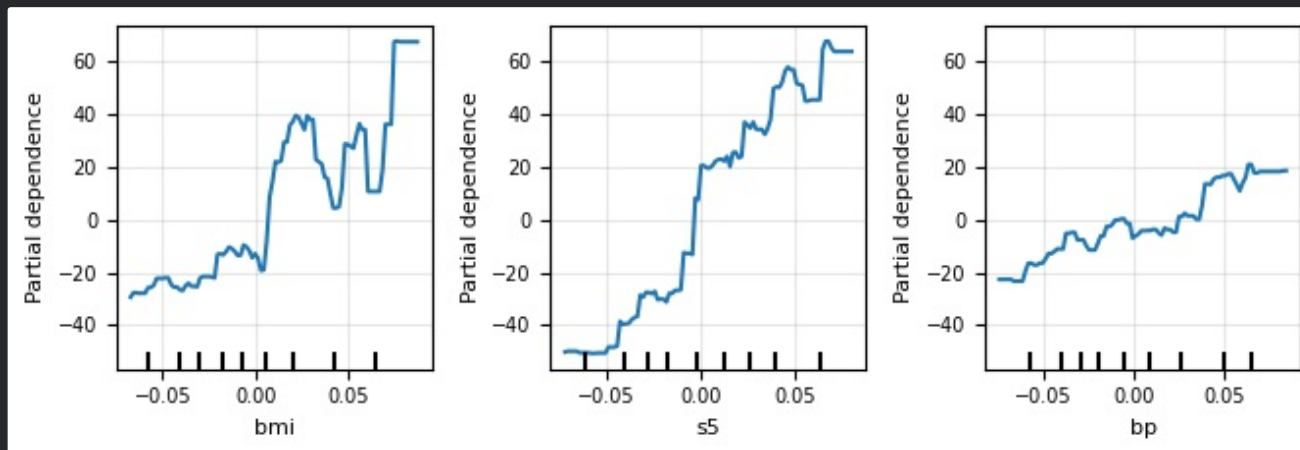
dados = load_diabetes()
modelo = HistGradientBoostingRegressor(
    max_iter=200, random_state=0).fit(dados.data, dados.target)

alvos = [2, 8, 3]          # imc, medida sanguinea, pressao
fig, eixos = plt.subplots(1, 3, figsize=(6.8, 2.4))
PartialDependenceDisplay.from_estimator(
    modelo, dados.data, alvos, ax=eixos,
    feature_names=list(dados.feature_names))
for eixo in eixos:
    eixo.tick_params(labelsize=7)
    eixo.set_xlabel(eixo.get_xlabel(), fontsize=8)
    eixo.set_ylabel(eixo.get_ylabel(), fontsize=8)
    eixo.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("i01.png", dpi=100, bbox_inches="tight")
print("curvas de dependência parcial geradas")
print("cada curva mostra o efeito médio do atributo na previsão")
```

--- SAÍDA ---

curvas de dependência parcial geradas
cada curva mostra o efeito médio do atributo na previsão

--- FIGURA ---



+-- #CÓDIGO 33.3 Efeito em cada amostra -----

A curva média pode esconder que o atributo ajuda um grupo e atrapalha outro. As linhas individuais revelam isso.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.inspection import PartialDependenceDisplay

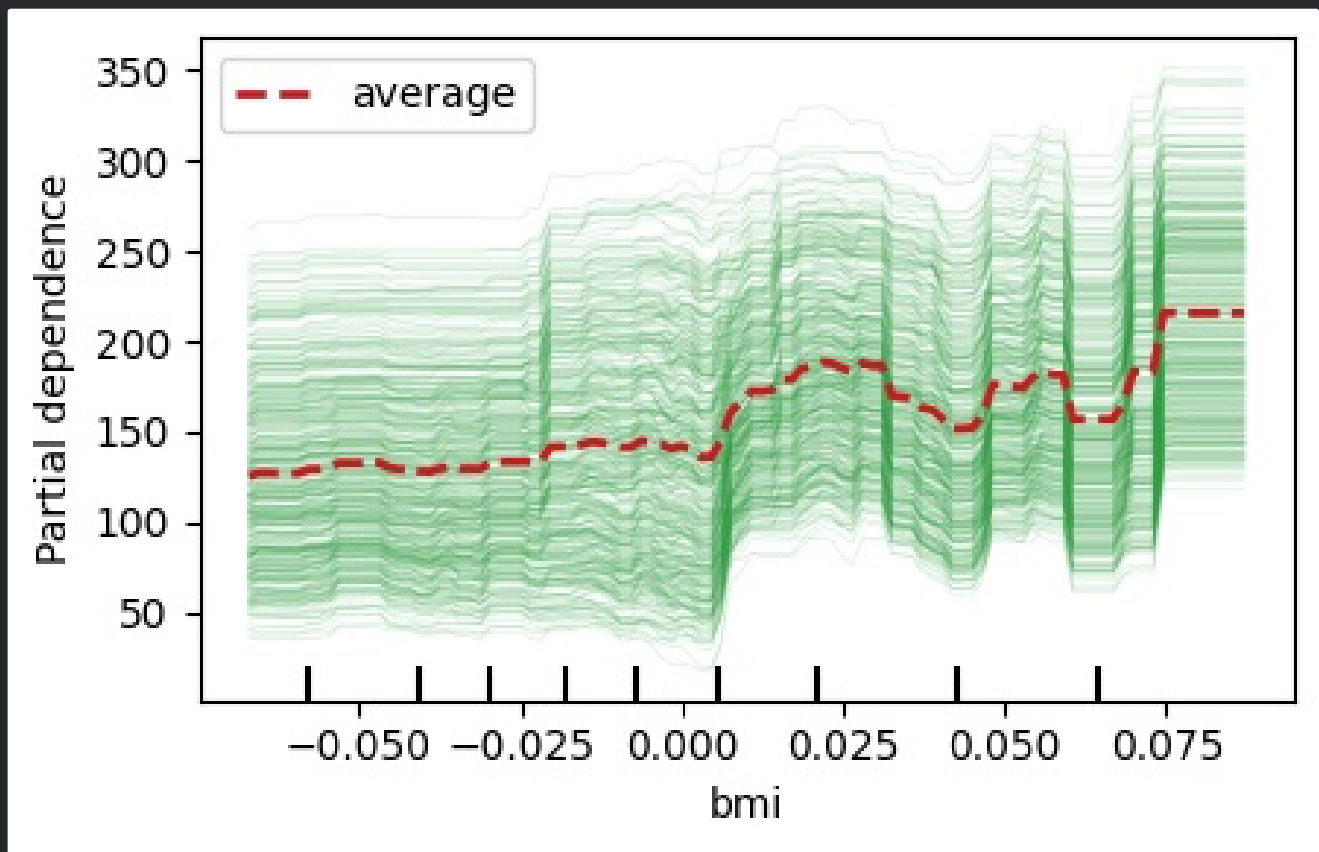
dados = load_diabetes()
modelo = HistGradientBoostingRegressor(
    max_iter=200, random_state=0).fit(dados.data, dados.target)

fig, eixo = plt.subplots(figsize=(4.6, 3.0))
PartialDependenceDisplay.from_estimator(
    modelo, dados.data, [2], ax=eixo, kind="both",
    feature_names=list(dados.feature_names),
    ice_lines_kw={"alpha": 0.15, "color": "#2f9e41"},
    pd_line_kw={"color": "#cd191e", "linewidth": 2})
eixo.grid(alpha=0.3)
eixo.tick_params(labelsize=7)
fig.tight_layout()
fig.savefig("i02.png", dpi=100, bbox_inches="tight")
print("cada linha verde é uma amostra, a vermelha é a média")
print("linhas com inclinações opostas indicam interação escondida")
```

--- SAÍDA ---

cada linha verde é uma amostra, a vermelha é a média
linhas com inclinações opostas indicam interação escondida

--- FIGURA ---



+-- #CÓDIGO 33.4 Contribuição por amostra -----

Uma aproximação simples da ideia por trás do SHAP: substituir o valor pelo típico e medir o quanto a previsão se move. Explica **aquele** decisão, não o modelo inteiro.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

dados = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    dados.data, dados.target, test_size=0.3, random_state=0,
    stratify=dados.target)
modelo = RandomForestClassifier(n_estimators=200,
                               random_state=0).fit(X_tr, y_tr)

def contribuicoes(modelo, amostra, base, n=5):
    """Quanto a previsão muda ao trocar cada atributo pelo valor médio."""
    p0 = modelo.predict_proba(amostra.reshape(1, -1))[0, 1]
    efeitos = []
    for j in range(len(amostra)):
        alterada = amostra.copy()
        alterada[j] = base[j]
        p = modelo.predict_proba(alterada.reshape(1, -1))[0, 1]
        efeitos.append(p0 - p)
    ordem = np.argsort(np.abs(efeitos))[:-1][::-1]
    return p0, [(dados.feature_names[j], efeitos[j]) for j in ordem]

media = X_tr.mean(axis=0)
for i in [0, 1]:
    prob, itens = contribuicoes(modelo, X_te[i], media)
    print(f"\namostra {i}: probabilidade prevista {prob:.3f}, "
          f"classe real {y_te[i]}")
    for nome, efeito in itens:
        seta = "+" if efeito > 0 else "-"
        print(f"  {seta} {nome:<26}{efeito:+.4f}")

--- SAÍDA ---
amostra 0: probabilidade prevista 0.005, classe real 0
- worst radius          -0.0750
- worst perimeter       -0.0550
- mean concave points  -0.0550
- mean concavity        -0.0250
- mean perimeter       -0.0200

amostra 1: probabilidade prevista 1.000, classe real 0
+ worst area            +0.1300
+ worst perimeter       +0.0750
+ worst radius          +0.0450
+ area error            +0.0300
+ mean perimeter       +0.0150
```

+-- #CÓDIGO 33.5 Modelo substituto -----

Treinar um modelo simples para imitar o complexo dá uma explicação global aproximada. Só é honesto quando a concordância é alta: informe sempre esse número.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split

dados = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    dados.data, dados.target, test_size=0.3, random_state=0,
    stratify=dados.target)

floresta = RandomForestClassifier(n_estimators=200,
                                random_state=0).fit(X_tr, y_tr)

# a arvore aprende a imitar a floresta, nao os rotulos verdadeiros
substituto = DecisionTreeClassifier(max_depth=3, random_state=0)
substituto.fit(X_tr, floresta.predict(X_tr))

concorda = (substituto.predict(X_te) == floresta.predict(X_te)).mean()
print(f"á árvore reproduz {concorda:.1%} das decisões da floresta")
print(f"acurácia da floresta: {floresta.score(X_te, y_te):.4f}")
print(f"acurácia do substituto:{substituto.score(X_te, y_te):.4f}\n")
print(export_text(substituto,
                  feature_names=list(dados.feature_names)[:520]))
```

--- SAÍDA ---

a árvore reproduz 92.4% das decisões da floresta
acurácia da floresta: 0.9532
acurácia do substituto:0.9006

```
|--- worst perimeter <= 106.10
| |--- worst concave points <= 0.16
| | |--- area error <= 48.98
| | | |--- class: 1
| | |--- area error > 48.98
| | | |--- class: 0
| |--- worst concave points > 0.16
| | |--- worst concavity <= 1.05
| | | |--- class: 0
| | |--- worst concavity > 1.05
| | | |--- class: 1
|--- worst perimeter > 106.10
| |--- worst perimeter <= 115.35
| | |--- mean texture <= 19.48
| | | |--- class: 1
| | |--- mean texture > 19.48
| |
```

[!] CORRELAÇÃO NÃO É MECANISMO

Todas estas ferramentas descrevem **o que o modelo faz**, não como o mundo funciona. Se dois atributos são correlacionados, o modelo pode ter escolhido um deles por acaso e a explicação apontará para o escolhido. Conclusão causal exige experimento, não gráfico de dependência.

>> EXPERIMENTOS PROPOSTOS

- . Gere as curvas de dependência parcial dos três atributos mais importantes do seu modelo.
- . Treine uma árvore substituta e informe a concordância com o modelo principal.
- . Explique duas previsões individuais, uma certa e uma errada.
- . Compare a importância por permutação com os coeficientes de uma regressão logística no mesmo conjunto.

=====

|| BLOCO XIII

APRENDIZADO DE MÁQUINA: NÃO SUPERVISIONADO

Etapas 3 e 4 sem rótulo: estrutura, grupos, anomalias e o caso de poucos rótulos.

=====

[34] DIMENSIONALIDADE E AGRUPAMENTO

Reduzir o número de atributos para visualizar, acelerar e remover ruído; e encontrar grupos quando não existe rótulo nenhum.

Análise de componentes principais

--- #CÓDIGO 34.1 Quanta informação cabe em poucos eixos -----

Se poucos componentes explicam quase toda a variância, os atributos originais eram redundantes.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)
pca = PCA().fit(X / 16.0)

acumulada = np.cumsum(pca.explained_variance_ratio_)

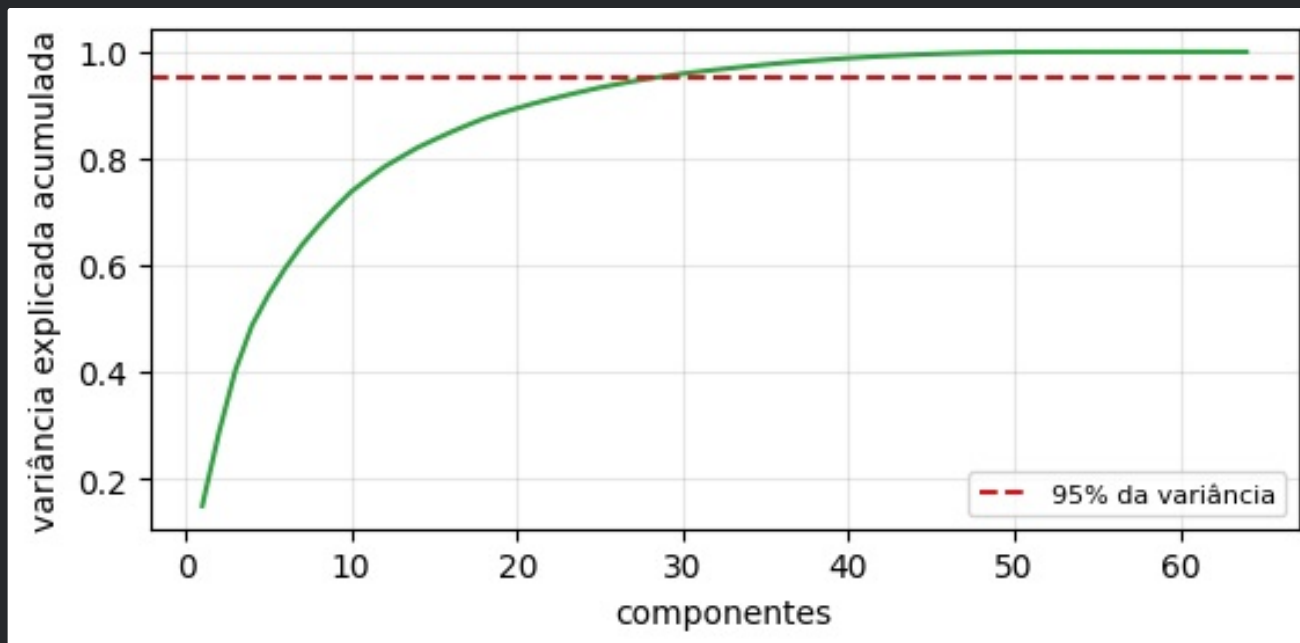
plt.figure(figsize=(6.4, 2.8))
plt.plot(range(1, len(acumulada) + 1), acumulada, color="#2f9e41")
plt.axhline(0.95, color="#cd191e", linestyle="--",
            label="95% da variância")
plt.xlabel("componentes")
plt.ylabel("variância explicada acumulada")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("p01.png", dpi=100, bbox_inches="tight")

print("atributos originais:", X.shape[1])
print("componentes para 90%:", int(np.argmax(acumulada >= 0.90)) + 1)
print("componentes para 95%:", int(np.argmax(acumulada >= 0.95)) + 1)
print("primeiros três:", pca.explained_variance_ratio_[:3].round(4))
```

--- SAÍDA ---

```
atributos originais: 64
componentes para 90%: 21
componentes para 95%: 29
primeiros três: [0.1489 0.1362 0.1179]
```

--- FIGURA ---



+-- #CÓDIGO 34.2 Visualizando em duas dimensões -----

Sessenta e quatro dimensões viram duas. Os dígitos já se separam parcialmente, o que indica que o problema é aprendível.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)
reduzido = PCA(n_components=2, random_state=0).fit_transform(X / 16.0)

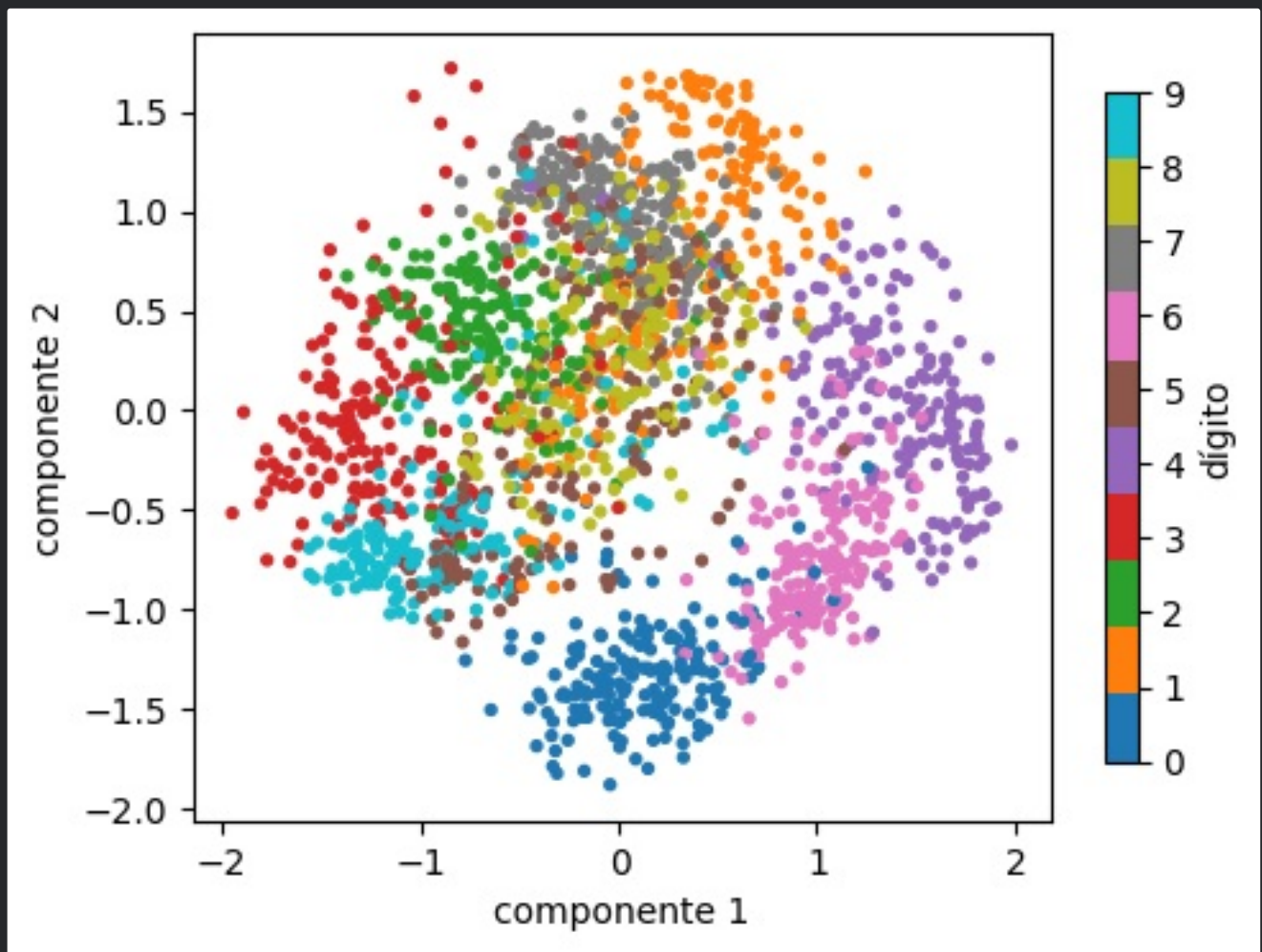
plt.figure(figsize=(5.4, 4.0))
dispersao = plt.scatter(reduzido[:, 0], reduzido[:, 1], c=y, s=8,
                        cmap="tab10")
plt.colorbar(dispersao, label="dígito", shrink=0.85)
plt.xlabel("componente 1")
plt.ylabel("componente 2")
plt.savefig("p02.png", dpi=100, bbox_inches="tight")

print("formato antes:", X.shape, "depois:", reduzido.shape)
```

--- SAÍDA ---

formato antes: (1797, 64) depois: (1797, 2)

--- FIGURA ---



+-- #CÓDIGO 34.3 PCA dentro do pipeline -----

Reduzir dimensão acelera o treino e às vezes melhora o resultado, ao descartar direções que eram só ruído.

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)

for n in [2, 10, 20, 40, None]:
    etapas = [StandardScaler()]
    if n:
        etapas.append(PCA(n_components=n, random_state=0))
    etapas.append(SVC())
    nota = cross_val_score(make_pipeline(*etapas), X, y, cv=5).mean()
    rotulo = f"{n} componentes" if n else "sem PCA (64)"
    print(f"{rotulo:<18}{nota:.4f}")
```

--- SAÍDA ---

2 componentes	0.5431
10 componentes	0.9015
20 componentes	0.9410
40 componentes	0.9455
sem PCA (64)	0.9460

+-- #CÓDIGO 34.4 t-SNE para visualização -----

0 t-SNE separa melhor que o PCA, mas serve só para visualizar: as distâncias no gráfico não têm significado global e o resultado muda com a semente.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True)
X, y = X[:600] / 16.0, y[:600]

mapa = TSNE(n_components=2, perplexity=30, init="pca",
            random_state=0).fit_transform(X)

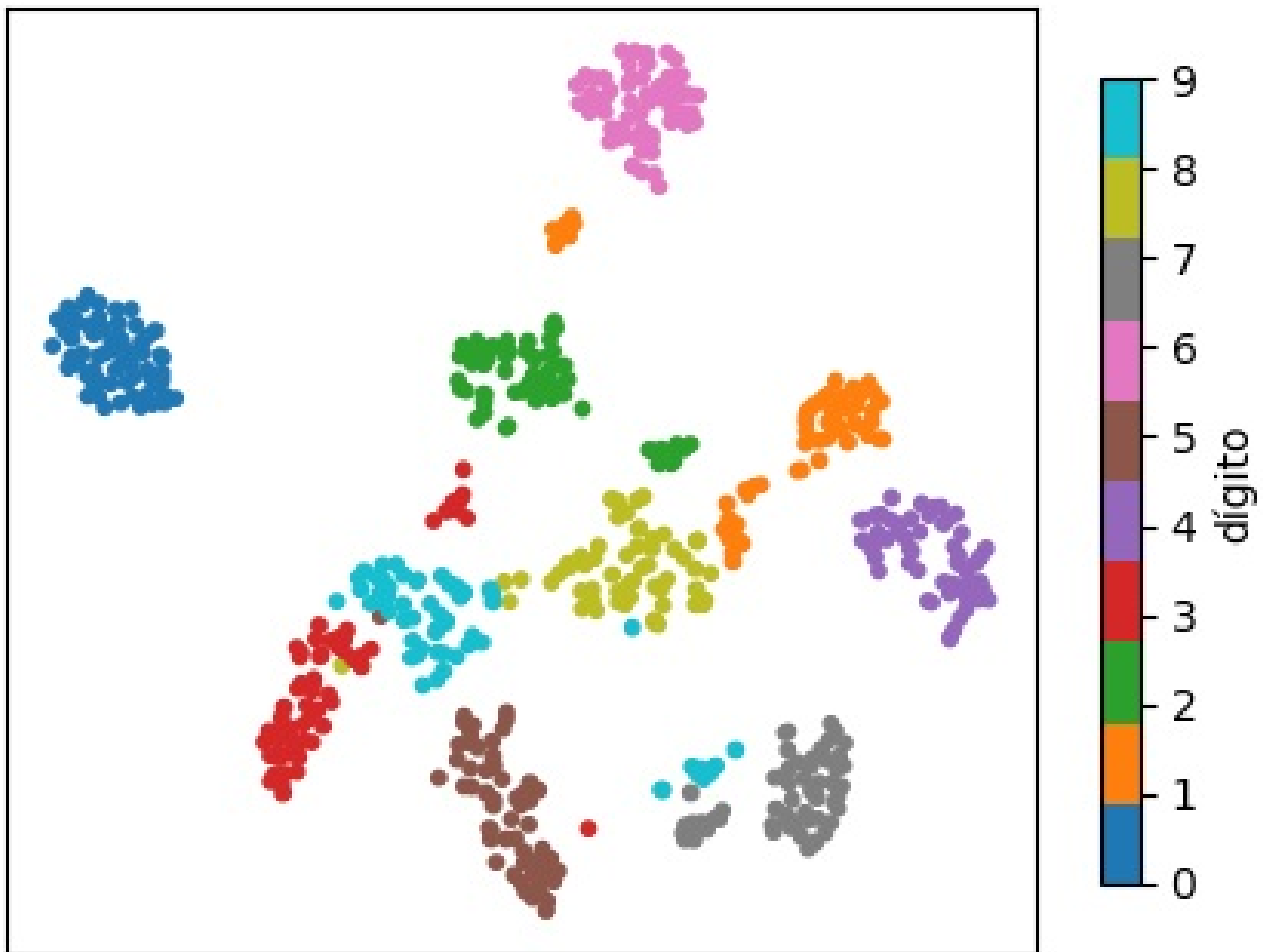
plt.figure(figsize=(5.4, 4.0))
d = plt.scatter(mapa[:, 0], mapa[:, 1], c=y, s=10, cmap="tab10")
plt.colorbar(d, label="dígito", shrink=0.85)
plt.xticks([]); plt.yticks([])
plt.savefig("p03.png", dpi=100, bbox_inches="tight")

print("t-SNE aplicado a", X.shape[0], "amostras")
```

--- SAÍDA ---

t-SNE aplicado a 600 amostras

--- FIGURA ---



Agrupamento

+-- #CÓDIGO 34.5 KMeans -----

O algoritmo não conhece os rótulos: ele apenas minimiza a distância de cada ponto ao centro do seu grupo.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, verdadeiro = make_blobs(n_samples=300, centers=3,
                           cluster_std=1.1, random_state=8)
kmeans = KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)

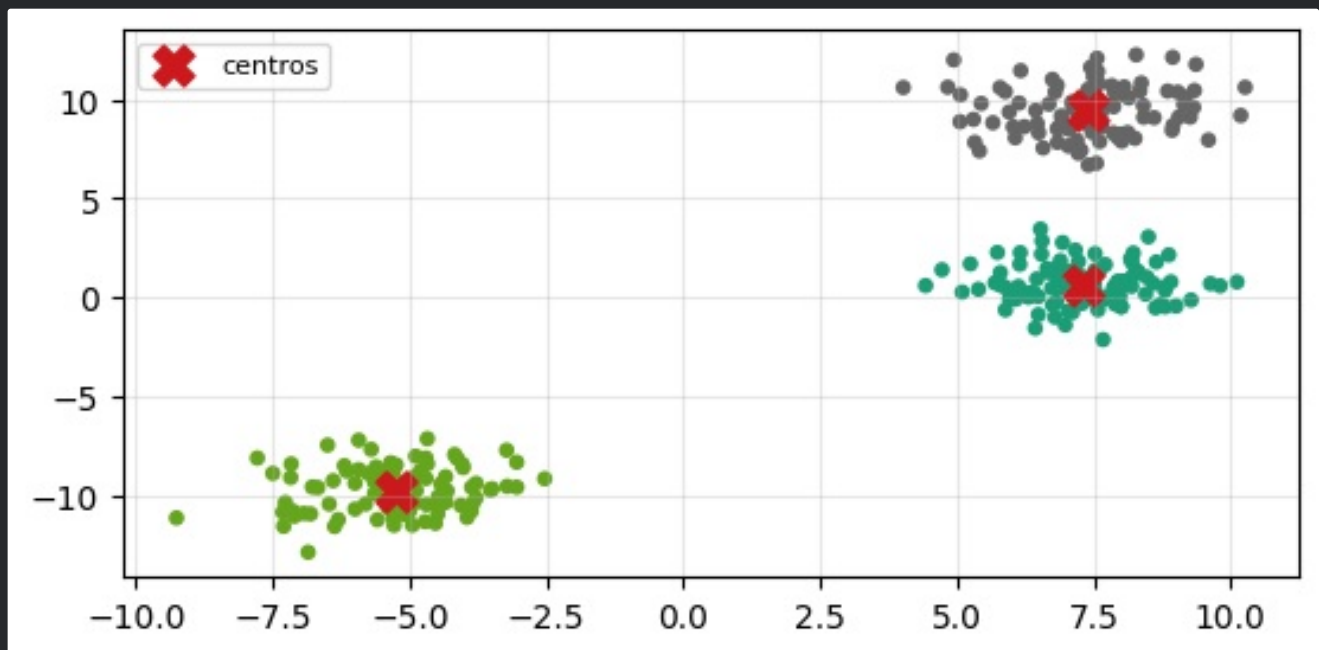
plt.figure(figsize=(6.4, 3.0))
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, s=14, cmap="Dark2")
plt.scatter(*kmeans.cluster_centers_.T, marker="X", s=150,
            color="#cd191e", label="centros")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("p04.png", dpi=100, bbox_inches="tight")

print("centros:\n", kmeans.cluster_centers_.round(2))
print("inércia:", round(kmeans.inertia_, 2))
```

--- SAÍDA ---

```
centros:
[[ 7.3  0.65]
 [-5.24 -9.76]
 [ 7.4  9.45]]
inércia: 793.32
```

--- FIGURA ---



--- #CÓDIGO 34.6 Quantos grupos: cotovelo e silhueta -----

O cotovelo é visual e subjetivo; a silhueta dá um número. Quando os dois concordam, a escolha fica fácil.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=400, centers=4, cluster_std=1.0,
                  random_state=5)

ks = range(2, 9)
inercias, silhuetas = [], []
for k in ks:
    modelo = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    inercias.append(modelo.inertia_)
    silhuetas.append(silhouette_score(X, modelo.labels_))

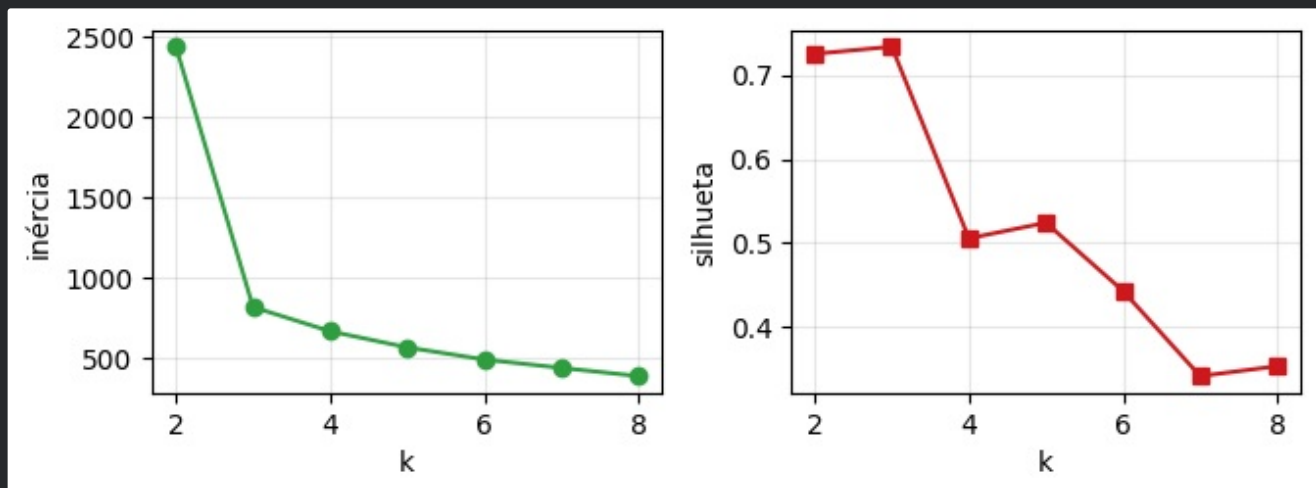
fig, (esq, dir_) = plt.subplots(1, 2, figsize=(6.8, 2.6))
esq.plot(list(ks), inercias, "o-", color="#2f9e41")
esq.set_xlabel("k"); esq.set_ylabel("inércia"); esq.grid(alpha=0.3)
dir_.plot(list(ks), silhuetas, "s-", color="#cd191e")
dir_.set_xlabel("k"); dir_.set_ylabel("silhueta"); dir_.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("p05.png", dpi=100, bbox_inches="tight")

melhor = list(ks)[silhuetas.index(max(silhuetas))]
print("melhor k pela silhueta:", melhor)
for k, s in zip(ks, silhuetas):
    print(f" k={k} silhueta={s:.4f}")
```

--- SAÍDA ---

```
melhor k pela silhueta: 3
k=2 silhueta=0.7248
k=3 silhueta=0.7334
k=4 silhueta=0.5057
k=5 silhueta=0.5242
k=6 silhueta=0.4432
k=7 silhueta=0.3420
k=8 silhueta=0.3538
```

--- FIGURA ---



+-- #CÓDIGO 34.7 DBSCAN: grupos de forma qualquer -----

KMeans só enxerga grupos aproximadamente esféricos. O DBSCAN segue a densidade, descobre o número de grupos sozinho e ainda rotula ruído como -1.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN, KMeans
from sklearn.datasets import make_moons

X, _ = make_moons(n_samples=400, noise=0.06, random_state=0)

kmeans = KMeans(n_clusters=2, n_init=10, random_state=0).fit(X)
dbscan = DBSCAN(eps=0.2, min_samples=5).fit(X)

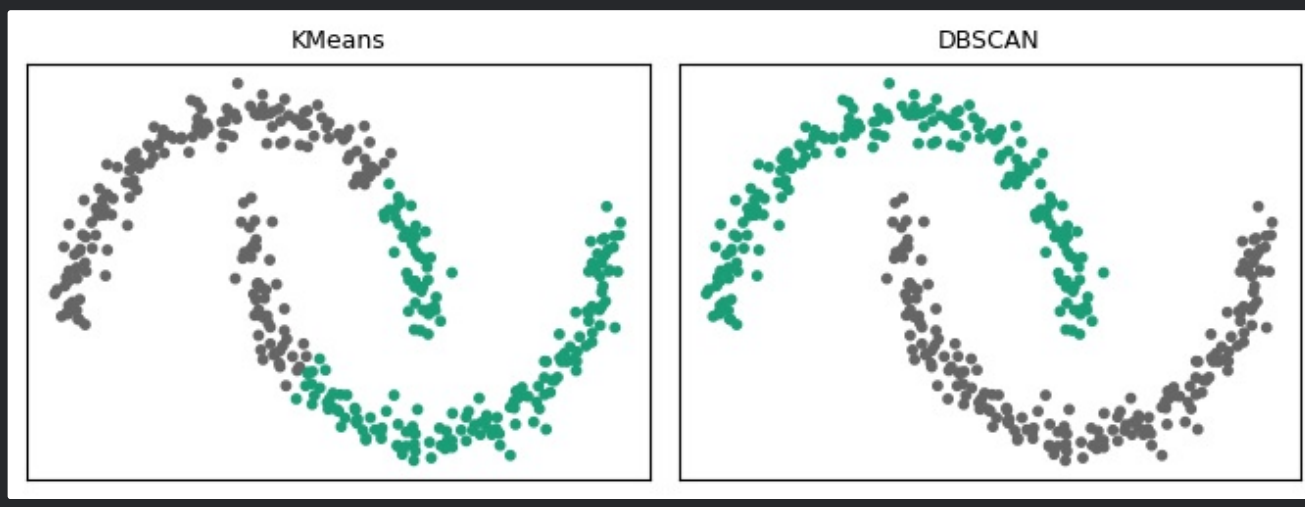
fig, (esq, dir_) = plt.subplots(1, 2, figsize=(6.8, 2.6))
esq.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, s=10, cmap="Dark2")
esq.set_title("KMeans", fontsize=9)
dir_.scatter(X[:, 0], X[:, 1], c=dbscan.labels_, s=10, cmap="Dark2")
dir_.set_title("DBSCAN", fontsize=9)
for eixo in (esq, dir_):
    eixo.set_xticks([]); eixo.set_yticks([])
fig.tight_layout()
fig.savefig("p06.png", dpi=100, bbox_inches="tight")

print("grupos do DBSCAN:", len(set(dbscan.labels_) -
    (1 if -1 in dbscan.labels_ else 0)))
print("pontos marcados como ruído:", int((dbscan.labels_ == -1).sum()))
```

--- SAÍDA ---

grupos do DBSCAN: 2
pontos marcados como ruído: 0

--- FIGURA ---



>> EXPERIMENTOS PROPOSTOS

- . Aplique PCA a um conjunto seu e diga quantos componentes guardam 95% da variância.
- . Compare a acurácia com 2, 10 e todos os componentes.
- . Use a silhueta para escolher o número de grupos em dados sem rótulo.
- . Explique por que t-SNE não deve ser usado como etapa de pré-processamento para um classificador.

[35] DETECÇÃO DE ANOMALIAS

Encontrar o ponto fora da curva sem que ninguém tenha dito o que é normal. Treinamento e avaliação aqui não usam rótulo.

+-- #CÓDIGO 35.1 Linha de base estatística -----

Três desvios padrão é a regra mais simples que existe. Funciona bem quando os dados são aproximadamente normais e há um único atributo.

```
import numpy as np

rng = np.random.default_rng(4)
normal = rng.normal(-60, 3, 200)          # RSSI típico
medidas = np.concatenate([normal, [-25, -95, -20]])

media, desvio = medidas.mean(), medidas.std()
escore = np.abs((medidas - media) / desvio)
suspeitos = np.where(escore > 3)[0]

print(f"média {media:.2f} desvio {desvio:.2f}")
print("índices suspeitos:", suspeitos)
print("valores:", medidas[suspeitos].round(2))
```

--- SAÍDA ---

```
média -59.66 desvio 5.36
índices suspeitos: [200 201 202]
valores: [-25. -95. -20.]
```

+-- #CÓDIGO 35.2 Isolation Forest -----

O algoritmo isola pontos com cortes ao acaso: quem é isolado com poucos cortes é anômalo. Não precisa de rótulo e funciona em várias dimensões.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(2)
normais = rng.normal(0, 1, size=(300, 2))
anomalias = rng.uniform(-6, 6, size=(15, 2))
X = np.vstack([normais, anomalias])

detector = IsolationForest(contamination=0.05, random_state=0)
rotulo = detector.fit_predict(X)      # -1 = anomalia

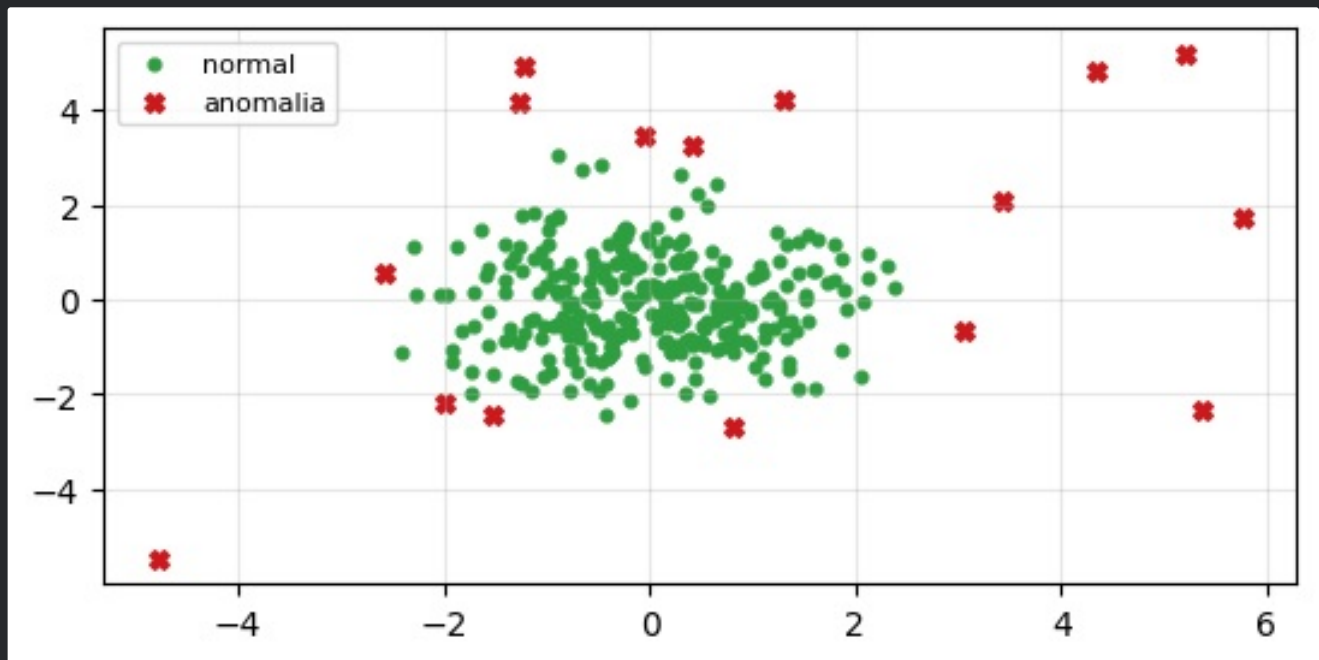
plt.figure(figsize=(6.4, 3.0))
plt.scatter(X[rotulo == 1, 0], X[rotulo == 1, 1], s=12,
            color="#2f9e41", label="normal")
plt.scatter(X[rotulo == -1, 0], X[rotulo == -1, 1], s=30,
            color="#cd191e", marker="X", label="anomalia")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("a01.png", dpi=100, bbox_inches="tight")

print("pontos marcados como anomalia:", int((rotulo == -1).sum()))
print("escore dos cinco mais suspeitos:",
      np.sort(detector.score_samples(X))[:5].round(3))
```

--- SAÍDA ---

pontos marcados como anomalia: 16
escore dos cinco mais suspeitos: [-0.812 -0.765 -0.736 -0.734 -0.734]

--- FIGURA ---



+-- #CÓDIGO 35.3 Comparando detectores -----

`LocalOutlierFactor` compara a densidade local com a dos vizinhos e, por padrão, só rotula o próprio conjunto de treino.

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(2)
X = np.vstack([rng.normal(0, 1, size=(300, 2)),
               rng.uniform(-6, 6, size=(15, 2))])
verdade = np.r_[np.ones(300), -np.ones(15)]

detectores = {
    "isolation forest": IsolationForest(contamination=0.05,
                                         random_state=0),
    "local outlier": LocalOutlierFactor(contamination=0.05),
    "one-class SVM": OneClassSVM(nu=0.05, gamma="scale"),
}

for nome, det in detectores.items():
    previsto = (det.fit_predict(X) if nome == "local outlier"
                else det.fit(X).predict(X))
    detectadas = int(((previsto == -1) & (verdade == -1)).sum())
    falsos = int(((previsto == -1) & (verdade == 1)).sum())
    print(f"{nome:<18}detectou {detectadas:>2}/15 "
          f"falsos alarmes {falsos:>2}")

--- SAÍDA ---
isolation forest detectou 11/15 falsos alarmes 5
local outlier    detectou 11/15 falsos alarmes 5
one-class SVM    detectou 9/15 falsos alarmes 12
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Detecte anomalias em um registro de medidas do seu laboratório com Isolation Forest.
- . Compare o z-score e o Isolation Forest no mesmo conjunto.
- . Varie o parâmetro `contamination` e observe quantos pontos passam a ser marcados.
- . Explique por que um detector treinado sem rótulo ainda precisa ser conferido por uma pessoa.

[36] POUCOS RÓTULOS: SEMI E AUTOSSUPERVISÃO

Rotular dados custa caro. Estas técnicas aproveitam o conjunto não rotulado, que quase sempre é muito maior que o rotulado.

+--- #CÓDIGO 36.1 O problema em números -----

A curva não é linear: os primeiros rótulos valem muito mais que os últimos. Saber onde ela satura evita rotular mais do que o necessário.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
X = X / 16.0
X_pool, X_te, y_pool, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

print(f'{"rótulos usados":>16}{"acurácia":>11}')
for n in [20, 50, 100, 300, len(y_pool)]:
    rng = np.random.default_rng(0)
    idx = rng.choice(len(y_pool), n, replace=False)
    m = SVC().fit(X_pool[idx], y_pool[idx])
    print(f"{n:>16}{m.score(X_te, y_te):>11.4f}")
```

--- SAÍDA ---

rótulos usados	acurácia
20	0.2519
50	0.7796
100	0.9185
300	0.9574
1257	0.9870

+-- #CÓDIGO 36.2 Autotreinamento e propagação de erro -----

O modelo rotula os casos em que tem mais confiança, acrescenta-os ao treino e repete. Repare no resultado: aqui o autotreinamento **piorou** tudo. Com 30 rótulos o classificador ainda erra muito, e cada erro cometido com confiança alta entra no treino como se fosse verdade, contaminando as rodadas seguintes. Limiar alto demais não acrescenta nada (0.99 não moveu uma amostra); limiar baixo acrescenta erro. O método só compensa quando o modelo inicial já é bom, e é por isso que o exemplo seguinte, que não depende da confiança do classificador, se sai muito melhor.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.semi_supervised import SelfTrainingClassifier
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X = X / 16.0
X_pool, X_te, y_pool, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

rng = np.random.default_rng(0)
rotulados = rng.choice(len(y_pool), 30, replace=False)

y_parcial = np.full(len(y_pool), -1)      # -1 significa sem rótulo
y_parcial[rotulados] = y_pool[rotulados]
print("rotuladas:", (y_parcial != -1).sum(),
      "| sem rótulo:", (y_parcial == -1).sum())

base = LogisticRegression(max_iter=2000)
sozinho = base.fit(X_pool[rotulados], y_pool[rotulados]).score(X_te, y_te)
print(f"nsó com os 30 rótulos: {sozinho:.4f}\n")

print(f"{'limiar':>8}{ 'novos rótulos':>15}{ 'rodadas':>9}{ 'acurácia':>11}")
for limiar in [0.70, 0.90, 0.99]:
    m = SelfTrainingClassifier(LogisticRegression(max_iter=2000),
                              threshold=limiar).fit(X_pool, y_parcial)
    novos = int((m.transduction_ != -1).sum()) - len(rotulados)
    print(f"{'limiar':>8.2f}{ 'novos':>15}{ 'm.n_iter':>9}"
          f"{'m.score(X_te, y_te)':>11.4f}")
```

--- SAÍDA ---

rotuladas: 30 | sem rótulo: 1227

só com os 30 rótulos: 0.7074

limiar	novos rótulos	rodadas	acurácia
0.70	1114	10	0.4315
0.90	119	10	0.5815
0.99	0	1	0.7074

+-- #CÓDIGO 36.3 Propagação de rótulos pelo grafo -----

Aqui o rótulo se espalha pela vizinhança: pontos próximos no espaço de atributos tendem a compartilhar a classe. Independe de o classificador ter confiança calibrada.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.semi_supervised import LabelSpreading

X, y = load_digits(return_X_y=True)
X = X / 16.0
X_pool, X_te, y_pool, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

rng = np.random.default_rng(0)
rotulados = rng.choice(len(y_pool), 50, replace=False)
y_parcial = np.full(len(y_pool), -1)
y_parcial[rotulados] = y_pool[rotulados]

modelo = LabelSpreading(kernel="knn", n_neighbors=7)
modelo.fit(X_pool, y_parcial)

nao_rotulados = y_parcial == -1
acerto = (modelo.transduction_[nao_rotulados]
          == y_pool[nao_rotulados]).mean()
print(f"rótulos inferidos corretamente no pool: {acerto:.4f}")
print(f"acurácia no teste: {modelo.score(X_te, y_te):.4f}")
```

--- SAÍDA ---

```
rótulos inferidos corretamente no pool: 0.9006
acurácia no teste: 0.9278
```

+-- #CÓDIGO 36.4 Aprendizado ativo -----

Em vez de sortear o que rotular, o modelo aponta as amostras em que está mais indeciso. Com o mesmo orçamento de rotulação, o resultado costuma ser melhor.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X = X / 16.0
X_pool, X_te, y_pool, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

def rodar(estrategia, rodadas=12, por_rodada=10):
    rng = np.random.default_rng(0)
    tem_rotulo = np.zeros(len(y_pool), bool)
    tem_rotulo[rng.choice(len(y_pool), 20, replace=False)] = True

    for _ in range(rodadas):
        m = LogisticRegression(max_iter=2000)
        m.fit(X_pool[tem_rotulo], y_pool[tem_rotulo])
        restantes = np.where(~tem_rotulo)[0]
        if estrategia == "aleatória":
            escolhidos = rng.choice(restantes, por_rodada,
                                    replace=False)
        else:
            # menor confiança primeiro
            proba = m.predict_proba(X_pool[restantes]).max(axis=1)
            escolhidos = restantes[np.argsort(proba)[:por_rodada]]
        tem_rotulo[escolhidos] = True

    m = LogisticRegression(max_iter=2000)
    m.fit(X_pool[tem_rotulo], y_pool[tem_rotulo])
    return tem_rotulo.sum(), m.score(X_te, y_te)

for estrategia in ["aleatória", "incerteza"]:
    n, acc = rodar(estrategia)
    print(f"{estrategia:<12}{n} rótulos -> acurácia {acc:.4f}")

--- SAÍDA ---
aleatória    140 rótulos -> acurácia 0.9185
incerteza    140 rótulos -> acurácia 0.9389
```

[*] APLICAÇÃO DIRETA NO LABORATÓRIO

Uma campanha de medidas gera milhares de janelas de sinal e ninguém vai classificar todas à mão. Rotule algumas dezenas, use o aprendizado ativo para escolher as próximas e a propagação para aproveitar o resto. É o caminho realista para um TCC com dados próprios.

>> EXPERIMENTOS PROPOSTOS

- Refaça o CÓDIGO 36.2 com 200 rótulos iniciais e verifique se o autotreinamento passa a ajudar.
- Compare propagação por grafo e autotreinamento com 20, 50 e 200 rótulos.
- Implemente o aprendizado ativo escolhendo pela margem entre as duas classes mais prováveis.
- Estime quantos rótulos o seu problema precisa para chegar a 90% do desempenho máximo.

=====

|| BLOCO XIV

APRENDIZADO POR REFORÇO

Etapas 3 e 4 sem conjunto de dados: o ambiente é a fonte de aprendizado.

=====

[37] APRENDIZADO POR REFORÇO

O terceiro tipo da Tabela 25.2, e o único sem conjunto de dados: o agente age, recebe recompensa e aprende com a consequência.

- * **Ambiente:** o mundo que responde às ações.
- * **Estado:** a situação atual, o que o agente observa.
- * **Ação:** o que ele pode fazer.
- * **Recompensa:** o retorno imediato de uma ação.
- * **Política:** a regra que escolhe a ação em cada estado.
- * **Dilema:** explorar o desconhecido ou aproveitar o que já se sabe.

+-- #CÓDIGO 37.1 Explorar contra aproveitar -----

Sem exploração o agente pode travar em um canal medíocre; só explorando, ele nunca aproveita o que descobriu. O equilíbrio fica no meio, e é o problema central da área.

```
import numpy as np

# Seis canais com taxas de sucesso desconhecidas pelo agente
VERDADEIRO = np.array([0.20, 0.35, 0.55, 0.30, 0.72, 0.45])

def simular(epsilon, passos=4000, semente=0):
    rng = np.random.default_rng(semente)
    soma = np.zeros(6)
    usos = np.zeros(6)
    ganho = 0
    for _ in range(passos):
        if rng.random() < epsilon or usos.min() == 0:
            canal = int(rng.integers(6)) # explora
        else:
            canal = int(np.argmax(soma / usos)) # aproveita
        premio = float(rng.random() < VERDADEIRO[canal])
        usos[canal] += 1
        soma[canal] += premio
        ganho += premio
    return ganho / passos, int(np.argmax(soma / usos))

print(f"melhor canal real: {VERDADEIRO.argmax()} "
      f"(taxa {VERDADEIRO.max():.2f})\n")
print(f"{'epsilon':>8}{ 'ganho médio':>14}{ 'canal eleito':>15}")
for eps in [0.0, 0.05, 0.1, 0.3, 1.0]:
    ganho, eleito = simular(eps)
    print(f"{'eps':>8.2f}{ 'ganho':>14.4f}{ 'eleito':>15}")
```

--- SAÍDA ---

melhor canal real: 4 (taxa 0.72)

epsilon	ganho médio	canal eleito
0.00	0.4545	5
0.05	0.7110	4
0.10	0.6613	4
0.30	0.6285	4
1.00	0.4275	4

Q-learning tabular

[>] A REGRA DE ATUALIZAÇÃO

```
Q[s,a] <- Q[s,a] + alfa * (r + gama * max Q[s2,:] - Q[s,a])
```

O agente corrige a estimativa atual na direção do que realmente aconteceu: recompensa imediata mais o melhor futuro possível, descontado por `gama`.

```

import numpy as np

MAPA = [
    "S..#",
    ".#..",
    "...#",
    "#..G",
]

LINHAS, COLUNAS = len(MAPA), len(MAPA[0])
ACOES = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}
SIMBOLO = {0: "^", 1: "v", 2: "<", 3: ">" }

def celula(estado):
    return divmod(estado, COLUNAS)

def passo(estado, acao):
    i, j = celula(estado)
    di, dj = ACOES[acao]
    ni, nj = i + di, j + dj
    if not (0 <= ni < LINHAS and 0 <= nj < COLUNAS):
        return estado, -1.0, False      # bateu na parede
    if MAPA[ni][nj] == "#":
        return estado, -5.0, False      # obstaculo
    novo = ni * COLUNAS + nj
    if MAPA[ni][nj] == "G":
        return novo, 20.0, True         # chegou
    return novo, -0.2, False            # custo do passo

print("mapa: S início, G destino, # obstáculo")
for linha in MAPA:
    print("  ", linha)
print("\nestados:", LINHAS * COLUNAS, "| ações:", len(ACOES))

```

--- SAÍDA ---

```

mapa: S início, G destino, # obstáculo
S..#
.#..
...#
#..G

```

```

estados: 16 | ações: 4

```

+-----

+-- #CÓDIGO 37.3 Treinando o agente -----

O epsilon cai ao longo do treino: o agente explora muito no começo e vai confiando na política aprendida. O retorno médio subindo é o sinal de que está funcionando.

```
import numpy as np

MAPA = ["S..#", ".#..", "...#", "#..G"]
LINHAS, COLUNAS = 4, 4
ACOES = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}
SIMBOLO = {0: "^", 1: "v", 2: "<", 3: ">"}
```

```
def passo(estado, acao):
    i, j = divmod(estado, COLUNAS)
    di, dj = ACOES[acao]
    ni, nj = i + di, j + dj
    if not (0 <= ni < LINHAS and 0 <= nj < COLUNAS):
        return estado, -1.0, False
    if MAPA[ni][nj] == "#":
        return estado, -5.0, False
    novo = ni * COLUNAS + nj
    if MAPA[ni][nj] == "G":
        return novo, 20.0, True
    return novo, -0.2, False

rng = np.random.default_rng(7)
Q = np.zeros((LINHAS * COLUNAS, len(ACOES)))
ALFA, GAMA, EPISODIOS = 0.15, 0.95, 3000

historico = []
for episodio in range(EPISODIOS):
    epsilon = max(0.05, 1.0 - episodio / 1500)
    estado, total, fim = 0, 0.0, False
    for _ in range(60):
        if rng.random() < epsilon:
            acao = int(rng.integers(4))
        else:
            acao = int(np.argmax(Q[estado]))
        novo, premio, fim = passo(estado, acao)
        Q[estado, acao] += ALFA * (premio + GAMA * Q[novo].max()
                                - Q[estado, acao])
        estado, total = novo, total + premio
        if fim:
            break
    historico.append(total)

for faixa in range(0, EPISODIOS, 600):
    media = np.mean(historico[faixa:faixa + 600])
    print(f"episódios {faixa:>5}-{faixa + 600:<5} "
          f"retorno médio {media:>7.2f}")
```

--- SAÍDA ---

episódios	0-600	retorno médio	-18.08
episódios	600-1200	retorno médio	13.87
episódios	1200-1800	retorno médio	18.43
episódios	1800-2400	retorno médio	18.68
episódios	2400-3000	retorno médio	18.57

+-- #CÓDIGO 37.4 A política aprendida, em ASCII -----

A tabela Q vira um mapa de setas. Esse desenho é o equivalente, no reforço, da matriz de confusão no aprendizado supervisionado: ele mostra de relance se o agente entendeu o problema.

```
import numpy as np

MAPA = ["S..#", ".#..", "...#", "#..G"]
LINHAS, COLUNAS = 4, 4
ACOES = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}
SIMBOLO = {0: "^", 1: "v", 2: "<", 3: ">"}
```

```
def passo(estado, acao):
    i, j = divmod(estado, COLUNAS)
    di, dj = ACOES[acao]
```

```

ni, nj = i + di, j + dj
if not (0 <= ni < LINHAS and 0 <= nj < COLUNAS):
    return estado, -1.0, False
if MAPA[ni][nj] == "#":
    return estado, -5.0, False
novo = ni * COLUNAS + nj
if MAPA[ni][nj] == "G":
    return novo, 20.0, True
return novo, -0.2, False

rng = np.random.default_rng(7)
Q = np.zeros((16, 4))
for episodio in range(3000):
    epsilon = max(0.05, 1.0 - episodio / 1500)
    estado, fim = 0, False
    for _ in range(60):
        acao = (int(rng.integers(4)) if rng.random() < epsilon
                else int(np.argmax(Q[estado])))
        novo, premio, fim = passo(estado, acao)
        Q[estado, acao] += 0.15 * (premio + 0.95 * Q[novo].max()
                                - Q[estado, acao])

        estado = novo
        if fim:
            break

print("+" + "-" * (COLUNAS * 4 - 1) + "+")
for i in range(LINHAS):
    linha = "|"
    for j in range(COLUNAS):
        c = MAPA[i][j]
        if c == "#":
            linha += " # |"
        elif c == "G":
            linha += " G |"
        else:
            linha += f" {SIMBOLO[int(np.argmax(Q[i * COLUNAS + j]))]} |"
    print(linha)
print("+" + "-" * (COLUNAS * 4 - 1) + "+")

estado, caminho = 0, [0]
for _ in range(20):
    estado, _, fim = passo(estado, int(np.argmax(Q[estado])))
    caminho.append(estado)
    if fim:
        break
print("\ncaminho ótimo em células:",
      " -> ".join(str(divmod(e, COLUNAS)) for e in caminho))

--- SAÍDA ---
+-----+
| v | > | v | # |
+-----+
| v | # | v | < |
+-----+
| > | v | v | # |
+-----+
| # | > | > | G |
+-----+

caminho ótimo em células: (0, 0) -> (1, 0) -> (2, 0) -> (2, 1) -> (3, 1) -> (3, 2) -> (3, 3)

```

[i] DO TABULAR AO PROFUNDO

A tabela só funciona com poucos estados. Quando o estado é uma imagem ou um vetor contínuo, troca-se a tabela por uma rede que estima $Q(s,a)$: é o DQN. A regra de atualização continua a mesma; muda apenas quem guarda os valores. Bibliotecas como Gymnasium e Stable-Baselines3 cuidam do resto.

.....
>> EXPERIMENTOS PROPOSTOS

- . Aumente o mapa para 6x6 com mais obstáculos e verifique se o agente ainda converge.
- . Troque o epsilon decrescente por um fixo em 0.1 e compare a curva de retorno.
- . Varie `gama` entre 0.5 e 0.99 e explique o efeito no caminho escolhido.
- . Modele a escolha de canal do CÓDIGO 37.1 como um problema com estados, por exemplo o nível de interferência.

=====

|| BLOCO XV

APRENDIZADO PROFUNDO

As mesmas etapas 3 e 4, agora com TensorFlow, Keras e PyTorch.

=====

[38] REDE NEURAL DO ZERO: O TREINAMENTO POR DENTRO

Antes de chamar uma biblioteca de aprendizado profundo, vale escrever o mecanismo inteiro em trinta linhas. Depois disso, `model.fit()` deixa de ser mágica.

Uma rede é composição de operações simples: multiplicar por pesos, somar viés, aplicar uma não linearidade. O treino ajusta os pesos na direção que reduz a perda, e essa direção vem da derivada.

+-- #CÓDIGO 38.1 O neurônio -----

```
import numpy as np

def sigmoide(z):
    return 1 / (1 + np.exp(-z))

entradas = np.array([0.7, -1.2, 0.3])
pesos = np.array([0.5, -0.4, 0.9])
vies = 0.1

z = entradas @ pesos + vies
saida = sigmoide(z)

print("soma ponderada:", round(z, 4))
print("saída do neurônio:", round(saida, 4))
print("decisão:", 1 if saida >= 0.5 else 0)
```

--- SAÍDA ---

```
soma ponderada: 1.2
saída do neurônio: 0.7685
decisão: 1
```

+-----

+-- #CÓDIGO 38.2 Funções de ativação -----

A ReLU domina o aprendizado profundo moderno por ser barata e não saturar para valores positivos.

```
import numpy as np
import matplotlib.pyplot as plt

z = np.linspace(-6, 6, 400)
funcoes = {
    "sigmoide": 1 / (1 + np.exp(-z)),
    "tanh": np.tanh(z),
    "ReLU": np.maximum(0, z),
}

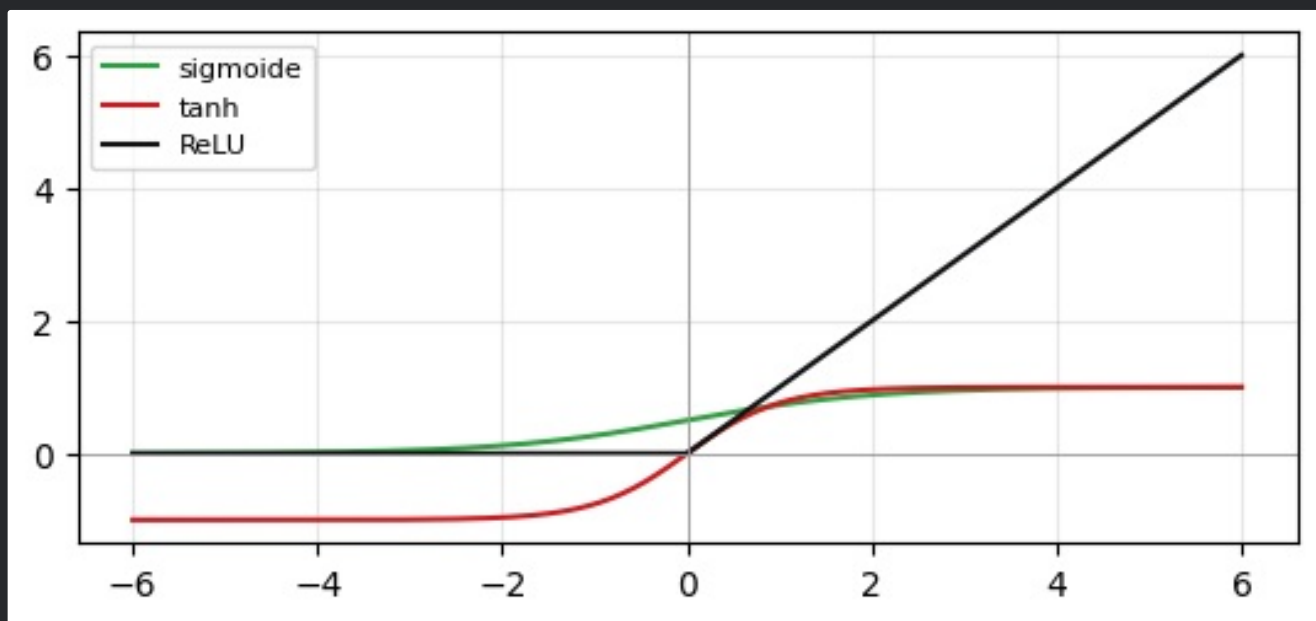
plt.figure(figsize=(6.4, 2.7))
for (nome, valores), cor in zip(funcoes.items(),
                                ["#2f9e41", "#cd191e", "#101214"]):
    plt.plot(z, valores, label=nome, color=cor)
plt.axhline(0, color="#999", linewidth=0.6)
plt.axvline(0, color="#999", linewidth=0.6)
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("m03.png", dpi=100, bbox_inches="tight")

for nome, valores in funcoes.items():
    print(f"{nome:<9} em z=-2: {valores[np.argmin(abs(z + 2))]:+.4f}")
```

--- SAÍDA ---

```
sigmoide  em z=-2: +0.1192
tanh      em z=-2: -0.9640
ReLU      em z=-2: +0.0000
```

--- FIGURA ---



+-- #CÓDIGO 38.3 Descida do gradiente em uma dimensão -----

Todo treino de rede neural é esta ideia repetida em milhares de dimensões ao mesmo tempo.

```
import numpy as np

def perda(w):
    return (w - 3) ** 2 + 1          # mínimo em w = 3

def derivada(w):
    return 2 * (w - 3)

w = -4.0
taxa = 0.2

for passo in range(1, 13):
    w -= taxa * derivada(w)
    if passo % 3 == 0:
        print(f"passo {passo:>2}: w = {w:+.4f}  perda = {perda(w):.4f}")

print("\nmínimo encontrado em w =", round(w, 4))
```

```
--- SAÍDA ---

passo  3: w = +1.4880  perda = 3.2861
passo  6: w = +2.6734  perda = 1.1067
passo  9: w = +2.9295  perda = 1.0050
passo 12: w = +2.9848  perda = 1.0002

mínimo encontrado em w = 2.9848
```

+-- #CÓDIGO 38.4 Rede completa resolvendo o XOR -----

O XOR não é separável por uma reta: é o exemplo clássico que exige camada oculta. Aqui está a retropropagação inteira, sem biblioteca.

```
import numpy as np

rng = np.random.default_rng(1)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float) # XOR

def sigmoide(z):
    return 1 / (1 + np.exp(-z))

# camada oculta com 4 neuronios
W1 = rng.normal(0, 1, (2, 4))
b1 = np.zeros((1, 4))
W2 = rng.normal(0, 1, (4, 1))
b2 = np.zeros((1, 1))
taxa = 0.5

for epoca in range(1, 8001):
    # propagacao direta
    h = sigmoide(X @ W1 + b1)
    saida = sigmoide(h @ W2 + b2)

    # retropropagacao
    erro = saida - y
    d_saida = erro * saida * (1 - saida)
    d_h = (d_saida @ W2.T) * h * (1 - h)

    W2 -= taxa * h.T @ d_saida
    b2 -= taxa * d_saida.sum(axis=0, keepdims=True)
    W1 -= taxa * X.T @ d_h
    b1 -= taxa * d_h.sum(axis=0, keepdims=True)

    if epoca % 2000 == 0:
        print(f"época {epoca}>5} perda = {(erro ** 2).mean():.6f}")

print("\nentrada -> saída da rede (esperado)")
for entrada, previsto, esperado in zip(X, saida.ravel(), y.ravel()):
    print(f"{entrada} -> {previsto:.4f} ({esperado:.0f})")

--- SAÍDA ---
época 2000 perda = 0.002318
época 4000 perda = 0.000787
época 6000 perda = 0.000461
época 8000 perda = 0.000323

entrada -> saída da rede (esperado)
[0. 0.] -> 0.0139 (0)
[0. 1.] -> 0.9817 (1)
[1. 0.] -> 0.9835 (1)
[1. 1.] -> 0.0221 (0)
```

--- #CÓDIGO 38.5 A curva de perda -----

A curva de perda é o primeiro gráfico a olhar em qualquer treino. Platô no início e queda brusca é o padrão típico do XOR.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(1)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

sig = lambda z: 1 / (1 + np.exp(-z))
W1, b1 = rng.normal(0, 1, (2, 4)), np.zeros((1, 4))
W2, b2 = rng.normal(0, 1, (4, 1)), np.zeros((1, 1))

historico = []
for epoca in range(6000):
    h = sig(X @ W1 + b1)
    saida = sig(h @ W2 + b2)
    erro = saida - y
    historico.append((erro ** 2).mean())

    d_s = erro * saida * (1 - saida)
    d_h = (d_s @ W2.T) * h * (1 - h)
    W2 -= 0.5 * h.T @ d_s
    b2 -= 0.5 * d_s.sum(axis=0, keepdims=True)
    W1 -= 0.5 * X.T @ d_h
    b1 -= 0.5 * d_h.sum(axis=0, keepdims=True)

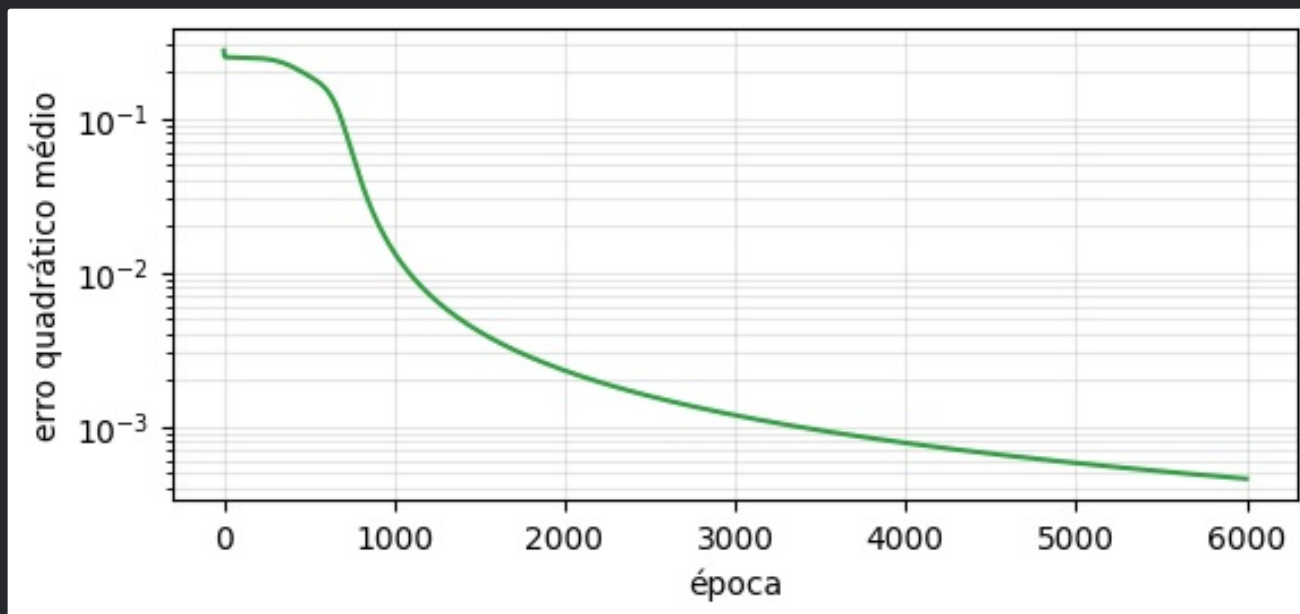
plt.figure(figsize=(6.4, 2.7))
plt.plot(historico, color="#2f9e41")
plt.xlabel("época")
plt.ylabel("erro quadrático médio")
plt.yscale("log")
plt.grid(alpha=0.3, which="both")
plt.savefig("m04.png", dpi=100, bbox_inches="tight")

print("perda inicial:", round(historico[0], 5))
print("perda final:", round(historico[-1], 8))
```

--- SAÍDA ---

perda inicial: 0.27486
perda final: 0.00046125

--- FIGURA ---



[*] O QUE A BIBLIOTECA FAZ POR VOCÊ

Keras e PyTorch automatizam três coisas deste exemplo: o cálculo das derivadas, a organização em camadas e o uso eficiente de memória e processador. A matemática continua sendo exatamente a mesma.

.....

>> EXPERIMENTOS PROPOSTOS

- . Troque a sigmoide por tanh na camada oculta e compare a velocidade de convergência.
- . Altere a taxa de aprendizado para 0.01 e para 5 e observe o efeito.
- . Treine a mesma rede para a função AND e explique por que ela converge mais rápido.
- . Acrescente uma segunda camada oculta e verifique se a perda final melhora.

[39] TENSORFLOW E KERAS

Keras é a interface de alto nível do TensorFlow: as mesmas contas da seção 38, com derivadas automáticas, camadas prontas e treino em lotes.

[i] VERSÕES USADAS

Este volume foi gerado com TensorFlow 2.21 e Keras 3.15, em CPU. Instale com `pip install tensorflow-cpu` (cerca de 300 MB) ou use o Google Colab, onde já vem pronto e com GPU disponível.

+--- #CÓDIGO 39.1 Tensores -----

Tensor é o array do NumPy com duas adições: roda em GPU e guarda o histórico de operações para derivar.

```
import tensorflow as tf

escalar = tf.constant(3.0)
vetor = tf.constant([1.0, 2.0, 3.0])
matriz = tf.constant([[1.0, 2.0], [3.0, 4.0]])

print("formato do vetor:", vetor.shape, "tipo:", vetor.dtype)
print("soma:", tf.reduce_sum(vetor).numpy())
print("produto matricial:\n", tf.matmul(matriz, matriz).numpy())
print("de tensor para NumPy:", vetor.numpy())
```

```
--- SAÍDA ---
formato do vetor: (3,) tipo: <dtype: 'float32'>
soma: 6.0
produto matricial:
[[ 7. 10.]
 [15. 22.]]
de tensor para NumPy: [1. 2. 3.]
```

+--- #CÓDIGO 39.2 Derivada automática -----

A *fita* grava as operações e devolve o gradiente. É isso que substitui a retropropagação escrita à mão da seção 38.

```
import tensorflow as tf

x = tf.Variable(3.0)

with tf.GradientTape() as fita:
    y = x ** 2 + 2 * x + 1      # y = (x+1)^2

derivada = fita.gradient(y, x)

print("y em x=3:", y.numpy())
print("dy/dx em x=3:", derivada.numpy(), "(esperado 2x+2 = 8)")
```

```
--- SAÍDA ---
y em x=3: 16.0
dy/dx em x=3: 8.0 (esperado 2x+2 = 8)
```

+-- #CÓDIGO 39.3 Primeiro modelo Sequential -----

Três decisões definem o modelo: as camadas, a função de perda e o otimizador.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(42)

modelo = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(8, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])

modelo.compile(optimizer="adam", loss="binary_crossentropy",
               metrics=["accuracy"])
modelo.summary()
```

--- SAÍDA ---

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8)	24
dense_1 (Dense)	(None, 8)	72
dense_2 (Dense)	(None, 1)	9

Total params: 105 (420.00 B)

Trainable params: 105 (420.00 B)

Non-trainable params: 0 (0.00 B)

+-- #CÓDIGO 39.4 Treinando e avaliando -----

`verbose=0` silencia a barra de progresso, útil em script. `validation_split` separa parte do treino para acompanhar o sobreajuste.

```
import numpy as np
import keras
from keras import layers
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(42)
X, y = make_moons(n_samples=1200, noise=0.2, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=42)

modelo = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
modelo.compile("adam", "binary_crossentropy", metrics=["accuracy"])

historico = modelo.fit(X_tr, y_tr, epochs=40, batch_size=32,
                      validation_split=0.2, verbose=0)

perda, acuracia = modelo.evaluate(X_te, y_te, verbose=0)
print(f"perda no teste: {perda:.4f}")
print(f"acurácia no teste: {acuracia:.4f}")
print("épocas registradas:", len(historico.history["loss"]))
print("acurácia na última época de treino:",
      round(historico.history["accuracy"][-1], 4))
```

--- SAÍDA ---

```
perda no teste:    0.2596
acurácia no teste: 0.8767
épocas registradas: 40
acurácia na última época de treino: 0.8944
```

+-- #CÓDIGO 39.5 Curvas de treino -----

Se a curva de validação sobe enquanto a de treino desce, o modelo começou a decorar. É o momento de parar.

```
import keras
import matplotlib.pyplot as plt
from keras import layers
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(42)
X, y = make_moons(n_samples=1200, noise=0.2, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=42)

modelo = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
modelo.compile("adam", "binary_crossentropy", metrics=["accuracy"])
h = modelo.fit(X_tr, y_tr, epochs=60, batch_size=32,
              validation_split=0.2, verbose=0)

fig, (esq, dir_) = plt.subplots(1, 2, figsize=(6.8, 2.7))
esq.plot(h.history["loss"], color="#2f9e41", label="treino")
esq.plot(h.history["val_loss"], color="#cd191e", label="validação")
esq.set_xlabel("época"); esq.set_ylabel("perda")
esq.legend(fontsize=8); esq.grid(alpha=0.3)

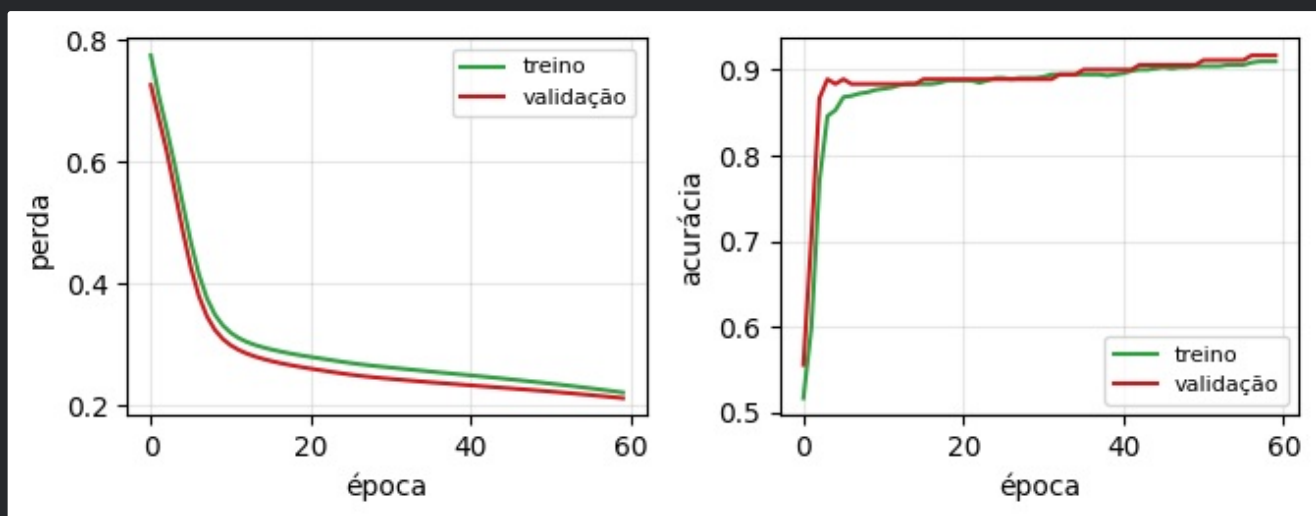
dir_.plot(h.history["accuracy"], color="#2f9e41", label="treino")
dir_.plot(h.history["val_accuracy"], color="#cd191e",
          label="validação")
dir_.set_xlabel("época"); dir_.set_ylabel("acurácia")
dir_.legend(fontsize=8); dir_.grid(alpha=0.3)

fig.tight_layout()
fig.savefig("k01.png", dpi=100, bbox_inches="tight")
print("perda final de treino:", round(h.history["loss"][-1], 4))
print("perda final de validação:",
      round(h.history["val_loss"][-1], 4))
```

--- SAÍDA ---

perda final de treino: 0.2214
perda final de validação: 0.2122

--- FIGURA ---



+-- #CÓDIGO 39.6 Classificação multiclasse -----

Para várias classes: última camada com `softmax` e perda `sparse_categorical_crossentropy` quando os rótulos são inteiros.

```
import numpy as np
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = X / 16.0 # escala 0 a 1
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=30, batch_size=32, verbose=0)

perda, acc = modelo.evaluate(X_te, y_te, verbose=0)
print(f"acurácia no teste: {acc:.4f}")

probabilidades = modelo.predict(X_te[:3], verbose=0)
for i, p in enumerate(probabilidades):
    print(f"amostra {i}: prevista {p.argmax()} "
          f"com confiança {p.max():.3f}, verdadeira {y_te[i]}")
```

--- SAÍDA ---

```
acurácia no teste: 0.9667
amostra 0: prevista 3 com confiança 0.567, verdadeira 2
amostra 1: prevista 0 com confiança 0.998, verdadeira 0
amostra 2: prevista 4 com confiança 1.000, verdadeira 4
```

+-- #CÓDIGO 39.7 Regularização e dropout -----

Dropout desliga neurônios ao acaso durante o treino e força a rede a não depender de poucos caminhos. A diferença entre treino e validação é o termômetro.

```
import keras
from keras import layers, regularizers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = X / 16.0
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

def construir(com_regularizacao):
    if com_regularizacao:
        meio = [layers.Dense(128, activation="relu",
                             kernel_regularizer=regularizers.l2(1e-3)),
                layers.Dropout(0.3)]
    else:
        meio = [layers.Dense(128, activation="relu")]
    modelo = keras.Sequential(
        [layers.Input(shape=(64,))] + meio
        + [layers.Dense(10, activation="softmax")])
    modelo.compile("adam", "sparse_categorical_crossentropy",
                   metrics=["accuracy"])
    return modelo

for rotulo, flag in [("sem regularização", False),
                    ("com dropout e L2", True)]:
    m = construir(flag)
    h = m.fit(X_tr, y_tr, epochs=40, batch_size=32,
              validation_split=0.2, verbose=0)
    treino = h.history["accuracy"][-1]
    validacao = h.history["val_accuracy"][-1]
    print(f"{rotulo:<20} treino {treino:.4f} "
          f"validação {validacao:.4f} "
          f"diferença {treino - validacao:+.4f}")
```

--- SAÍDA ---

```
sem regularização      treino 0.9972 validação 0.9630 diferença +0.0343
com dropout e L2       treino 0.9824 validação 0.9704 diferença +0.0120
```

+-- #CÓDIGO 39.8 Parada antecipada e melhor modelo -----

`EarlyStopping` interrompe quando a validação para de melhorar e devolve os melhores pesos. Deixe as épocas com folga e confie no callback.

```
import keras
from keras import layers, callbacks
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = X / 16.0
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])

parada = callbacks.EarlyStopping(monitor="val_loss", patience=5,
                                restore_best_weights=True)
guarda = callbacks.ModelCheckpoint("melhor.keras",
                                  monitor="val_loss",
                                  save_best_only=True)

h = modelo.fit(X_tr, y_tr, epochs=200, batch_size=32,
               validation_split=0.2, verbose=0,
               callbacks=[parada, guarda])

print("épocas executadas:", len(h.history["loss"]), "de 200")
print("melhor época:", parada.best_epoch + 1)
print(f"acurácia no teste: "
      f"{modelo.evaluate(X_te, y_te, verbose=0)[1]:.4f}")
```

--- SAÍDA ---

```
épocas executadas: 78 de 200
melhor época: 73
acurácia no teste: 0.9778
```

+-- #CÓDIGO 39.9 Salvando e recarregando -----

O formato `.keras` guarda arquitetura, pesos e configuração de treino em um único arquivo.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(1)
modelo = keras.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy")

modelo.save("modelo.keras")
recarregado = keras.models.load_model("modelo.keras")

entrada = np.random.default_rng(0).normal(size=(2, 4))
iguais = np.allclose(modelo.predict(entrada, verbose=0),
                      recarregado.predict(entrada, verbose=0))
print("previsões idênticas após recarregar?", iguais)

import os
print("tamanho do arquivo:",
      os.path.getsize("modelo.keras") // 1024, "KB")
```

```
--- SAÍDA ---
previsões idênticas após recarregar? True
tamanho do arquivo: 18 KB
```

+-----

TAREFA	ÚLTIMA CAMADA	FUNÇÃO DE PERDA
classificação binária	Dense(1, "sigmoid")	binary_crossentropy
multiclasse com rótulo inteiro	Dense(n, "softmax")	sparse_categorical_crossentropy
multiclasse com rótulo one-hot	Dense(n, "softmax")	categorical_crossentropy
regressão	Dense(1)	mse ou mae

+-- Tabela 39.1: como terminar a rede.

.....

>> EXPERIMENTOS PROPOSTOS

- . Treine um MLP para o conjunto `load_wine` e compare com a floresta aleatória da seção 31.
- . Aumente a rede para 3 camadas de 128 neurônios e observe treino contra validação.
- . Acrescente `EarlyStopping` e verifique em que época o treino para.
- . Salve o modelo, recarregue em outro script e faça uma previsão.

[40] REDES CONVOLUCIONAIS E SINAIS

Convolução aprende padrões locais: bordas em imagens, formas de onda em sinais. É a arquitetura que domina visão computacional e boa parte do processamento de sinais moderno.

Uma camada densa liga tudo a tudo e ignora a vizinhança. A convolução desliza um pequeno filtro pelo dado e reaproveita os mesmos pesos em todas as posições. Resultado: menos parâmetros e sensibilidade a padrões locais, onde eles estiverem.

+-- #CÓDIGO 40.1 A imagem como tensor -----

O formato é (amostras, altura, largura, canais). Um canal significa escala de cinza; três significam RGB.

```
import numpy as np
from sklearn.datasets import load_digits

X, y = load_digits(return_X_y=True)
imagens = X.reshape(-1, 8, 8, 1) / 16.0

print("formato do lote:", imagens.shape)
print("uma imagem:", imagens[0].shape, "rótulo:", y[0])
print("\nprimeiro dígito em texto:")
for linha in imagens[0, :, :, 0]:
    print("".join("#" if v > 0.5 else "+" if v > 0.2 else ".")
          for v in linha))
```

--- SAÍDA ---

```
formato do lote: (1797, 8, 8, 1)
uma imagem: (8, 8, 1) rótulo: 0
```

```
primeiro dígito em texto:
```

```
..+##...
..####+.
..#..#+.
.+#...++.
.++..#+.
.+#..#+.
..#+##..
..+##...
```

O filtro responde onde há transição. A rede aprende os valores do filtro em vez de recebê-los prontos.

```
import numpy as np

imagem = np.array([
    [0, 0, 1, 1, 0, 0],
    [0, 0, 1, 1, 0, 0],
    [0, 0, 1, 1, 0, 0],
    [0, 0, 1, 1, 0, 0],
], dtype=float)

filtro = np.array([[-1, 1]], dtype=float) # detector de borda

altura = imagem.shape[0]
largura = imagem.shape[1] - filtro.shape[1] + 1
saida = np.zeros((altura, largura))
for i in range(altura):
    for j in range(largura):
        janela = imagem[i, j:j + filtro.shape[1]]
        saida[i, j] = (janela * filtro).sum()

print("entrada:\n", imagem.astype(int))
print("\nresposta do filtro (bordas verticais):\n",
      saida.astype(int))
```

--- SAÍDA ---

entrada:

```
[[0 0 1 1 0 0]
 [0 0 1 1 0 0]
 [0 0 1 1 0 0]
 [0 0 1 1 0 0]]
```

resposta do filtro (bordas verticais):

```
[[ 0  1  0 -1  0]
 [ 0  1  0 -1  0]
 [ 0  1  0 -1  0]
 [ 0  1  0 -1  0]]
```

+-- #CÓDIGO 40.3 CNN para dígitos -----

```
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = X.reshape(-1, 8, 8, 1) / 16.0
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(8, 8, 1)),
    layers.Conv2D(16, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(2),
    layers.Conv2D(32, 3, activation="relu", padding="same"),
    layers.Flatten(),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.summary()

modelo.fit(X_tr, y_tr, epochs=15, batch_size=32, verbose=0)
print(f"\nacurácia no teste: "
      f"{modelo.evaluate(X_te, y_te, verbose=0)[1]:.4f}")
```

--- SAÍDA ---

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 8, 8, 16)	160
max_pooling2d (MaxPooling2D)	(None, 4, 4, 16)	0
conv2d_1 (Conv2D)	(None, 4, 4, 32)	4,640
flatten (Flatten)	(None, 512)	0
dropout (Dropout)	(None, 512)	0
dense (Dense)	(None, 10)	5,130

Total params: 9,930 (38.79 KB)

Trainable params: 9,930 (38.79 KB)

Non-trainable params: 0 (0.00 B)

acurácia no teste: 0.9733

+-- #CÓDIGO 40.4 Onde a CNN erra -----

A diagonal concentra os acertos. Fora dela estão os pares que o modelo confunde, e é ali que mora a próxima melhoria.

```
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = X.reshape(-1, 8, 8, 1) / 16.0
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(8, 8, 1)),
```

```

layers.Conv2D(16, 3, activation="relu", padding="same"),
layers.MaxPooling2D(2),
layers.Flatten(),
layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=15, batch_size=32, verbose=0)

previsto = modelo.predict(X_te, verbose=0).argmax(axis=1)
matriz = confusion_matrix(y_te, previsto)

plt.figure(figsize=(4.6, 4.0))
plt.imshow(matriz, cmap="Greens")
plt.colorbar(shrink=0.8)
plt.xlabel("previsto"); plt.ylabel("verdadeiro")
plt.xticks(range(10), fontsize=7); plt.yticks(range(10), fontsize=7)
for i in range(10):
    for j in range(10):
        if matriz[i, j]:
            plt.text(j, i, matriz[i, j], ha="center", va="center",
                     fontsize=6,
                     color="white" if i == j else "#cd191e")
plt.savefig("k02.png", dpi=100, bbox_inches="tight")

erros = np.where(previsto != y_te)[0]
print("erros no teste:", len(erros), "de", len(y_te))
for i in erros[:5]:
    print(f" amostra {i}: previu {previsto[i]}, era {y_te[i]}")

```

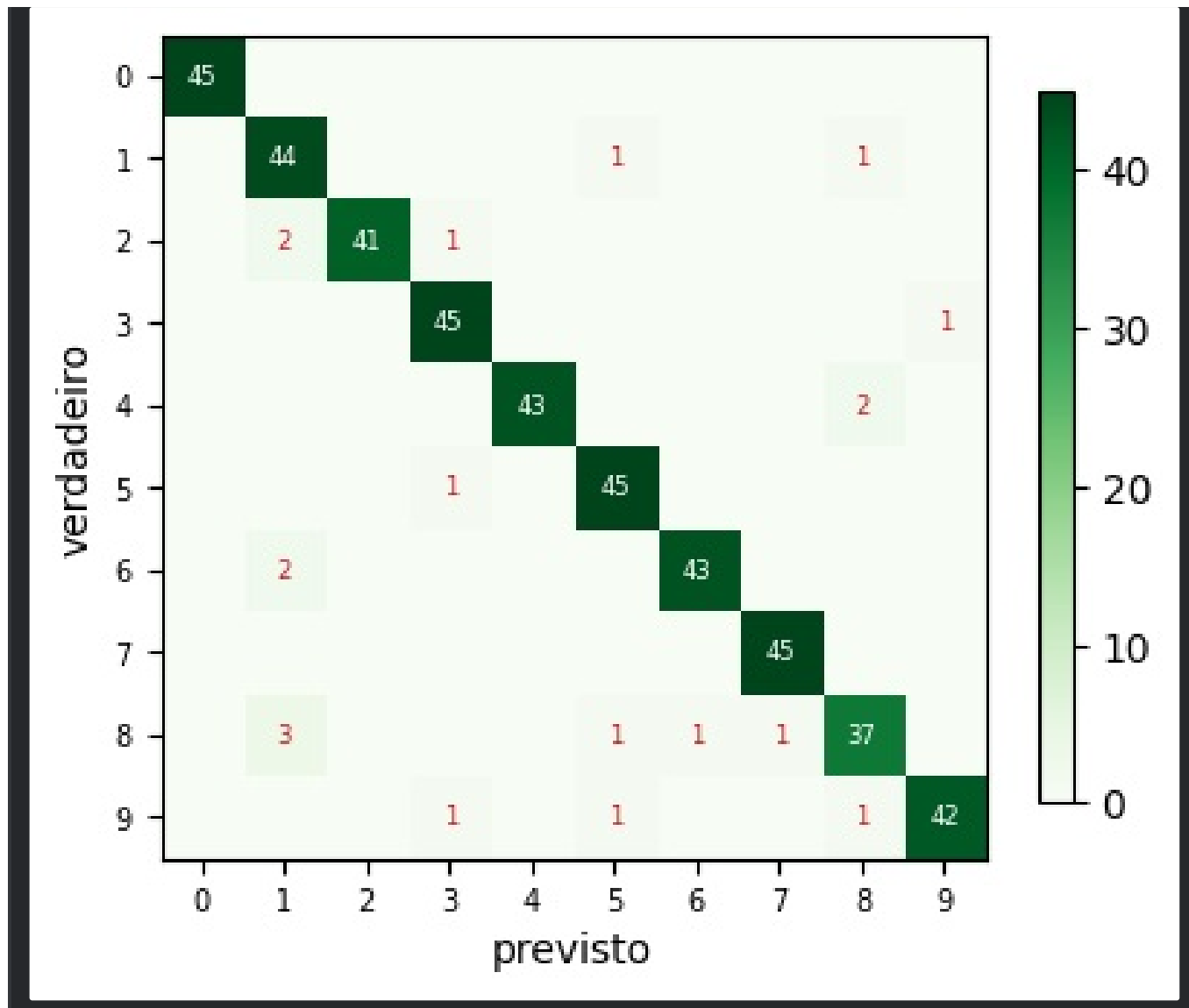
--- SAÍDA ---

```

erros no teste: 20 de 450
 amostra 0: previu 3, era 2
 amostra 38: previu 8, era 4
 amostra 39: previu 1, era 6
 amostra 46: previu 3, era 5
 amostra 54: previu 8, era 4

```

--- FIGURA ---



Convolução em uma dimensão

+-- #CÓDIGO 40.5 Classificando formas de onda -----

Mesma ideia da imagem, agora deslizando o filtro no tempo. É o ponto de partida para classificação de modulação e detecção de eventos em sinais.

```
import numpy as np
import keras
from keras import layers
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(3)
rng = np.random.default_rng(3)

def gerar(n_por_classe=400, tamanho=128):
    t = np.linspace(0, 1, tamanho, endpoint=False)
    X, y = [], []
    for classe in range(3):
        for _ in range(n_por_classe):
            f = rng.uniform(3, 8)
            fase = rng.uniform(0, 2 * np.pi)
            if classe == 0:          # senoide
                onda = np.sin(2 * np.pi * f * t + fase)
            elif classe == 1:        # quadrada
                onda = np.sign(np.sin(2 * np.pi * f * t + fase))
            else:                    # ruído
                onda = rng.normal(0, 1, tamanho)
            X.append(onda + rng.normal(0, 0.25, tamanho))
            y.append(classe)
    return np.array(X)[..., None], np.array(y)

X, y = gerar()
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=3, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(128, 1)),
    layers.Conv1D(16, 9, activation="relu", padding="same"),
    layers.MaxPooling1D(4),
    layers.Conv1D(32, 5, activation="relu", padding="same"),
    layers.GlobalAveragePooling1D(),
    layers.Dense(3, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=12, batch_size=32, verbose=0)

acc = modelo.evaluate(X_te, y_te, verbose=0)[1]
print("classes: 0 senoide, 1 quadrada, 2 ruído")
print(f"acurácia no teste: {acc:.4f}")

--- SAÍDA ---
classes: 0 senoide, 1 quadrada, 2 ruído
acurácia no teste: 1.0000
```

+-----

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(3)
t = np.linspace(0, 1, 128, endpoint=False)

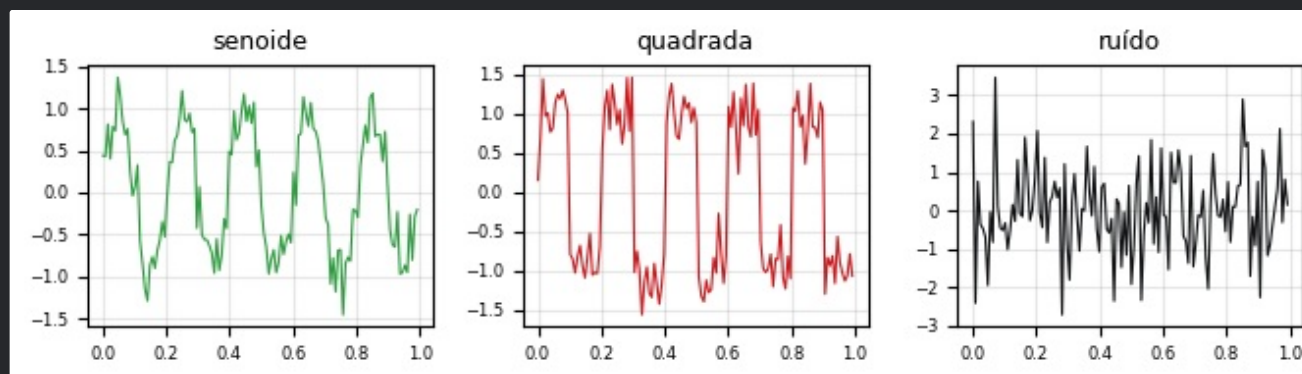
exemplos = {
    "senoide": np.sin(2 * np.pi * 5 * t),
    "quadrada": np.sign(np.sin(2 * np.pi * 5 * t)),
    "ruído": rng.normal(0, 1, 128),
}

fig, eixos = plt.subplots(1, 3, figsize=(6.8, 2.0))
cores = ["#2f9e41", "#cd191e", "#101214"]
for eixo, (nome, onda), cor in zip(eixos, exemplos.items(), cores):
    eixo.plot(t, onda + rng.normal(0, 0.25, 128), color=cor,
              linewidth=0.8)
    eixo.set_title(nome, fontsize=9)
    eixo.tick_params(labelsize=6)
    eixo.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("k03.png", dpi=100, bbox_inches="tight")
print("três classes com ruído aditivo")
```

--- SAÍDA ---

três classes com ruído aditivo

--- FIGURA ---



[*] TRANSFERÊNCIA DE APRENDIZADO

Para imagens de verdade, ninguém treina do zero. Carrega-se uma rede já treinada em milhões de fotos (`keras.applications.MobileNetV2`, por exemplo), congelam-se as camadas iniciais e treina-se apenas a última. Isso exige baixar os pesos, o que não foi feito na geração deste manual, mas é uma linha de código no Colab.

>> EXPERIMENTOS PROPOSTOS

- . Aumente o número de filtros da CNN de dígitos e veja o efeito na acurácia e no número de parâmetros.
- . Substitua a CNN por um MLP com o mesmo número de parâmetros e compare.
- . Acrescente uma quarta classe de sinal, como dente de serra, e retreine.
- . Aplique a CNN 1D a um sinal real capturado no laboratório.

[41] ARQUITETURAS E TREINO AVANÇADO

API funcional, autoencoders, redes recorrentes, normalização em lote, agendamento da taxa e transferência de aprendizado: o repertório que separa o primeiro modelo do modelo bom.

Além da pilha linear

+-- #CÓDIGO 41.1 API funcional do Keras -----

Cada camada é chamada como função sobre a anterior. É assim que se constrói rede com várias entradas, várias saídas ou conexões que pulam camadas.

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

# Duas entradas: espectro e metadados do enlace
espectro = keras.Input(shape=(128,), name="espectro")
contexto = keras.Input(shape=(3,), name="contexto")

x = layers.Dense(32, activation="relu")(espectro)
x = layers.Dense(16, activation="relu")(x)
juntos = layers.Concatenate()([x, contexto])
saida = layers.Dense(1, activation="sigmoid")(juntos)

modelo = keras.Model([espectro, contexto], saida)
modelo.compile("adam", "binary_crossentropy", metrics=["accuracy"])

print("entradas:", [e.shape for e in modelo.inputs])
print("parâmetros:", modelo.count_params())

--- SAÍDA ---
entradas: [(None, 128), (None, 3)]
parâmetros: 4676
```

+-- #CÓDIGO 41.2 Conexão residual -----

O atalho soma a entrada à saída do bloco. É a ideia da ResNet, e é o que torna possível treinar redes muito profundas.

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

entrada = keras.Input(shape=(64,))
x = layers.Dense(64, activation="relu")(entrada)
x = layers.Dense(64)(x)
somado = layers.Add()([entrada, x])      # atalho
x = layers.Activation("relu")(somado)
saida = layers.Dense(10, activation="softmax")(x)

modelo = keras.Model(entrada, saida)
print("camadas:", len(modelo.layers))
print("parâmetros:", modelo.count_params())

--- SAÍDA ---
camadas: 6
parâmetros: 8970
```

Autoencoders

+-- #CÓDIGO 41.3 Compressão sem rótulo -----

O autoencoder aprende a reconstruir a própria entrada passando por um gargalo. O código do meio é uma representação compacta, aprendida sem rótulo nenhum.

```
import numpy as np
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(1)
X, _ = load_digits(return_X_y=True)
X = X / 16.0
X_tr, X_te = train_test_split(X, test_size=0.25, random_state=1)

codificador = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(8, activation="relu", name="codigo"),
])
decodificador = keras.Sequential([
    layers.Input(shape=(8,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="sigmoid"),
])
auto = keras.Sequential([codificador, decodificador])
auto.compile("adam", "mse")
auto.fit(X_tr, X_tr, epochs=60, batch_size=32, verbose=0)

reconstruido = auto.predict(X_te, verbose=0)
erro = np.mean((X_te - reconstruido) ** 2)
print("dimensão original:", X.shape[1], "-> código:", 8)
print(f"erro quadrático médio da reconstrução: {erro:.5f}")
print("compressão de", round(64 / 8, 1), "vezes")
```

--- SAÍDA ---

```
dimensão original: 64 -> código: 8
erro quadrático médio da reconstrução: 0.02573
compressão de 8.0 vezes
```

+--- #CÓDIGO 41.4 Antes e depois da reconstrução -----

```
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers
from sklearn.datasets import load_digits

keras.utils.set_random_seed(1)
X, y = load_digits(return_X_y=True)
X = X / 16.0

auto = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="sigmoid"),
])
auto.compile("adam", "mse")
auto.fit(X, X, epochs=60, batch_size=32, verbose=0)

amostras = X[:8]
saida = auto.predict(amostras, verbose=0)

fig, eixos = plt.subplots(2, 8, figsize=(6.8, 2.0))
for i in range(8):
    eixos[0, i].imshow(amostras[i].reshape(8, 8), cmap="Greys")
    eixos[1, i].imshow(saida[i].reshape(8, 8), cmap="Greys")
    for linha in (0, 1):
        eixos[linha, i].set_xticks([]); eixos[linha, i].set_yticks([])
eixos[0, 0].set_ylabel("original", fontsize=7)
eixos[1, 0].set_ylabel("recons.", fontsize=7)
fig.tight_layout()
fig.savefig("d01.png", dpi=100, bbox_inches="tight")
print("oito dígitos reconstruídos a partir de 8 números cada")
```

--- SAÍDA ---
oito dígitos reconstruídos a partir de 8 números cada

--- FIGURA ---



+--- #CÓDIGO 41.5 Autoencoder para remover ruído de sinal -----

Treinado com pares ruidoso e limpo, o autoencoder aprende a filtrar. É uma alternativa aprendida ao filtro clássico, útil quando o ruído não é bem comportado.

```
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers

keras.utils.set_random_seed(2)
rng = np.random.default_rng(2)

def gerar(n, tamanho=128):
    t = np.linspace(0, 1, tamanho, endpoint=False)
    limpo = np.array([np.sin(2 * np.pi * rng.uniform(2, 6) * t
        + rng.uniform(0, 6.28)) for _ in range(n)])
    sujo = limpo + rng.normal(0, 0.5, limpo.shape)
```

```

return sujo, limpo

X_sujo, X_limpo = gerar(3000)
T_sujo, T_limpo = gerar(200)

modelo = keras.Sequential([
    layers.Input(shape=(128,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(128),
])
modelo.compile("adam", "mse")
modelo.fit(X_sujo, X_limpo, epochs=40, batch_size=64, verbose=0)

recuperado = modelo.predict(T_sujo, verbose=0)
antes = np.mean((T_sujo - T_limpo) ** 2)
depois = np.mean((recuperado - T_limpo) ** 2)
print(f"erro antes: {antes:.5f}")
print(f"erro depois: {depois:.5f}")
print(f"redução de ruído: {10 * np.log10(antes / depois):.2f} dB")

plt.figure(figsize=(6.6, 2.6))
plt.plot(T_sujo[0], color="#cccccc", linewidth=0.9, label="ruidoso")
plt.plot(T_limpo[0], color="#101214", linewidth=1.0, label="limpo")
plt.plot(recuperado[0], color="#2f9e41", linewidth=1.0,
         label="recuperado")
plt.legend(fontsize=8, ncol=3)
plt.grid(alpha=0.3)
plt.savefig("d02.png", dpi=100, bbox_inches="tight")

```

```

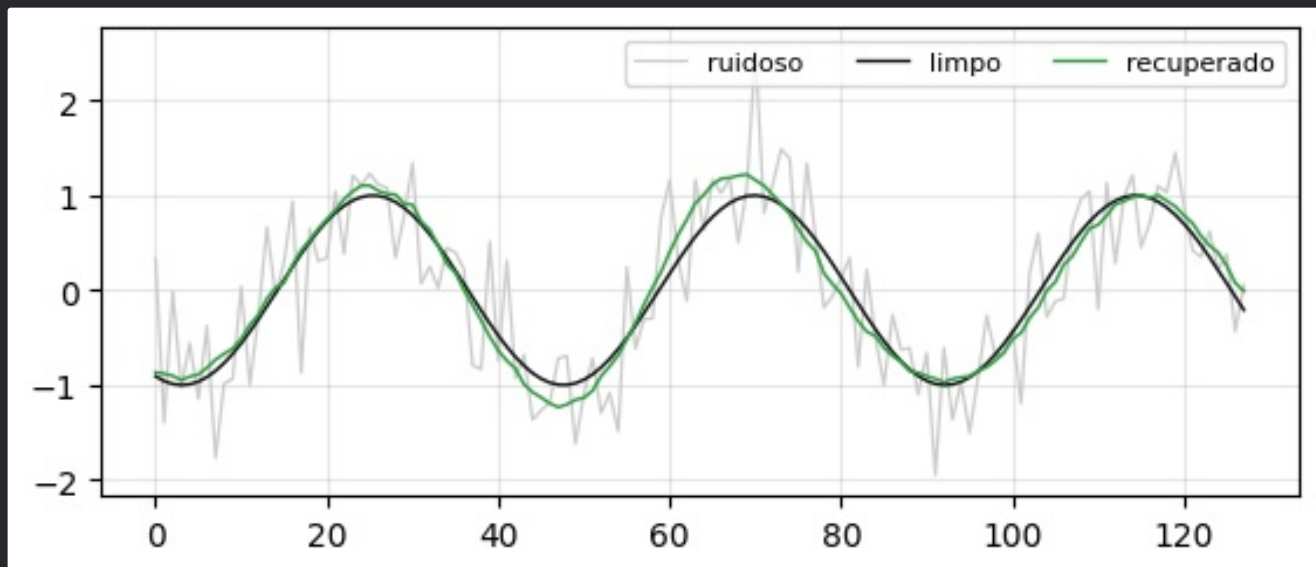
--- SAÍDA ---
erro antes: 0.24846
erro depois: 0.01985
redução de ruído: 10.98 dB

```

```

--- FIGURA ---

```



Redes recorrentes

--- #CÓDIGO 41.6 LSTM para previsão de série ---

A LSTM guarda estado ao longo da sequência. Para séries curtas e regulares, uma CNN 1D ou até uma floresta podem empatar com ela e treinar bem mais rápido.

```

import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers

keras.utils.set_random_seed(3)

```

```

rng = np.random.default_rng(3)

t = np.arange(1200)
serie = (20 + 5 * np.sin(2 * np.pi * t / 60)
        + 2 * np.sin(2 * np.pi * t / 17)
        + rng.normal(0, 0.4, t.size))

janela = 40
X = np.array([serie[i:i + janela] for i in range(len(serie) - janela)])
y = serie[janela:]
X = X[:, None] # (amostras, tempo, canais)

corte = int(len(X) * 0.8)
X_tr, X_te, y_tr, y_te = X[:corte], X[corte:], y[:corte], y[corte:]

modelo = keras.Sequential([
    layers.Input(shape=(janela, 1)),
    layers.LSTM(32),
    layers.Dense(1),
])
modelo.compile("adam", "mse", metrics=["mae"])
modelo.fit(X_tr, y_tr, epochs=15, batch_size=32, verbose=0)

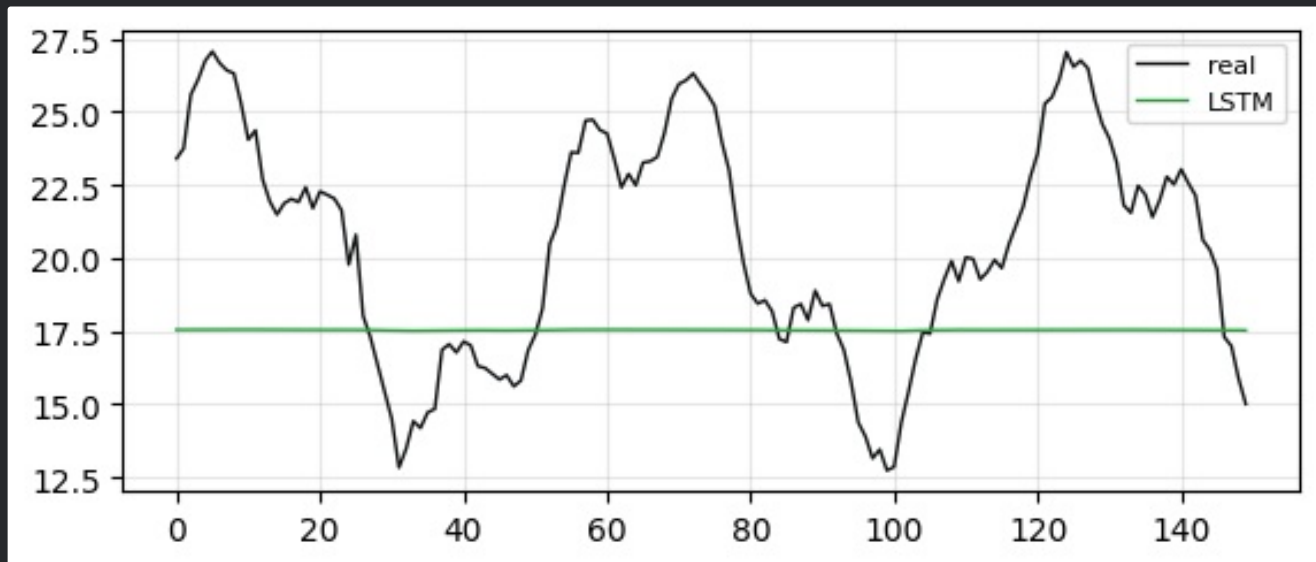
previsto = modelo.predict(X_te, verbose=0).ravel()
persistencia = X_te[:, -1, 0]
print(f"erro absoluto do modelo:      "
      f"{np.abs(previsto - y_te).mean():.4f}")
print(f"erro absoluto da persistência: "
      f"{np.abs(persistencia - y_te).mean():.4f}")

plt.figure(figsize=(6.6, 2.6))
plt.plot(y_te[:150], color="#101214", linewidth=1, label="real")
plt.plot(previsto[:150], color="#2f9e41", linewidth=1,
         label="LSTM")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("d03.png", dpi=100, bbox_inches="tight")

--- SAÍDA ---
erro absoluto do modelo:      3.7796
erro absoluto da persistência: 0.6731

--- FIGURA ---

```



Treino: o que realmente muda o resultado

+-- #CÓDIGO 41.7 Comparando otimizadores -----

Adam é o padrão razoável para começar. SGD puro precisa de ajuste fino da taxa, mas às vezes generaliza melhor.

```
import keras
import matplotlib.pyplot as plt
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X = X / 16.0
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

plt.figure(figsize=(6.4, 2.8))
cores = {"sgd": "#101214", "rmsprop": "#cd191e", "adam": "#2f9e41"}

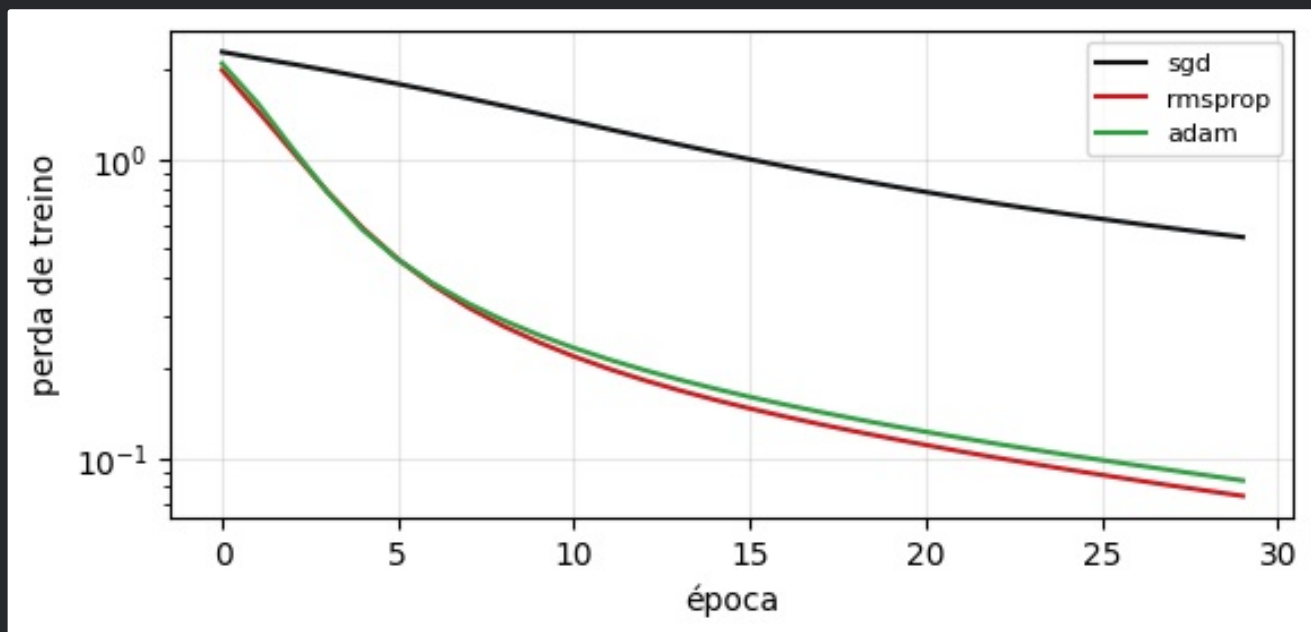
for nome, cor in cores.items():
    keras.utils.set_random_seed(0)
    modelo = keras.Sequential([
        layers.Input(shape=(64,)),
        layers.Dense(64, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ])
    modelo.compile(nome, "sparse_categorical_crossentropy",
        metrics=["accuracy"])
    h = modelo.fit(X_tr, y_tr, epochs=30, batch_size=32, verbose=0)
    plt.plot(h.history["loss"], color=cor, label=nome)
    print(f"{nome:<9}perda final {h.history['loss'][-1]:.4f} "
          f"teste {modelo.evaluate(X_te, y_te, verbose=0)[1]:.4f}")

plt.xlabel("época"); plt.ylabel("perda de treino")
plt.yscale("log"); plt.legend(fontsize=8); plt.grid(alpha=0.3)
plt.savefig("d04.png", dpi=100, bbox_inches="tight")
```

--- SAÍDA ---

```
sgd      perda final 0.5487  teste 0.9133
rmsprop  perda final 0.0745  teste 0.9711
adam     perda final 0.0839  teste 0.9756
```

--- FIGURA ---



+-- #CÓDIGO 41.8 Normalização em lote e agendamento da taxa -----

`BatchNormalization` estabiliza o treino de redes profundas. `ReduceLRonPlateau` reduz a taxa quando a perda para de cair, o que costuma render um ajuste fino no fim.

```
import keras
from keras import layers, callbacks
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X = X / 16.0
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

def treinar(com_bn, com_agenda):
    keras.utils.set_random_seed(0)
    camadas = [layers.Input(shape=(64,))]
    for _ in range(3):
        camadas.append(layers.Dense(128, activation="relu"))
        if com_bn:
            camadas.append(layers.BatchNormalization())
    camadas.append(layers.Dense(10, activation="softmax"))

    modelo = keras.Sequential(camadas)
    modelo.compile("adam", "sparse_categorical_crossentropy",
                   metrics=["accuracy"])

    cb = []
    if com_agenda:
        cb.append(callbacks.ReduceLRonPlateau(monitor="loss",
                                              factor=0.5,
                                              patience=3))

    modelo.fit(X_tr, y_tr, epochs=30, batch_size=32, verbose=0,
              callbacks=cb)
    return modelo.evaluate(X_te, y_te, verbose=0)[1]

for rotulo, bn, agenda in [("base", False, False),
                           ("com batchnorm", True, False),
                           ("com agenda de taxa", False, True),
                           ("com os dois", True, True)]:
    print(f"{rotulo:<22}{treinar(bn, agenda):.4f}")
```

--- SAÍDA ---

base	0.9756
com batchnorm	0.9689
com agenda de taxa	0.9756
com os dois	0.9689

Transferência de aprendizado

+-- #CÓDIGO 41.9 Reaproveitando um modelo já treinado -----

As camadas da tarefa A viram extrator de características na tarefa B. Repare no resultado: aqui a transferência **não** venceu o treino do zero, porque a tarefa B ainda tem 150 amostras e é muito parecida com a A. O ganho aparece quando os dados novos são escassos de verdade ou quando o modelo de origem foi treinado em um conjunto muito maior, como as redes de `keras.applications`. Repita o exemplo com `train_size=40` e compare.

```
import numpy as np
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(4)
X, y = load_digits(return_X_y=True)
X = X.reshape(-1, 8, 8, 1) / 16.0

# Tarefa A: muitos dados, dígitos de 0 a 4
a = y < 5
modelo_a = keras.Sequential([
    layers.Input(shape=(8, 8, 1)),
    layers.Conv2D(16, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(2),
```

```

        layers.Conv2D(32, 3, activation="relu", padding="same"),
        layers.Flatten(),
        layers.Dense(5, activation="softmax"),
    ])
    modelo_a.compile("adam", "sparse_categorical_crossentropy",
                     metrics=["accuracy"])
    modelo_a.fit(X[a], y[a], epochs=20, batch_size=32, verbose=0)
    print(f"tarefa A (dígitos 0-4): "
          f"{modelo_a.evaluate(X[a], y[a], verbose=0)[1]:.4f}")

# Tarefa B: poucos dados, dígitos de 5 a 9
b = y >= 5
Xb, yb = X[b], y[b] - 5
Xb_tr, Xb_te, yb_tr, yb_te = train_test_split(
    Xb, yb, train_size=150, random_state=0, stratify=yb)

def do_zero():
    keras.utils.set_random_seed(4)
    m = keras.Sequential([
        layers.Input(shape=(8, 8, 1)),
        layers.Conv2D(16, 3, activation="relu", padding="same"),
        layers.MaxPooling2D(2),
        layers.Conv2D(32, 3, activation="relu", padding="same"),
        layers.Flatten(),
        layers.Dense(5, activation="softmax"),
    ])
    m.compile("adam", "sparse_categorical_crossentropy",
              metrics=["accuracy"])
    return m

def transferido():
    # reaproveita as camadas já treinadas, menos a última
    corpo = keras.Sequential([layers.Input(shape=(8, 8, 1))]
                             + modelo_a.layers[:-1])
    corpo.trainable = False # congela o que já aprendeu
    m = keras.Sequential([layers.Input(shape=(8, 8, 1)), corpo,
                          layers.Dense(5, activation="softmax")])
    m.compile("adam", "sparse_categorical_crossentropy",
              metrics=["accuracy"])
    return m

for rotulo, construir in [("do zero", do_zero),
                          ("transferido", transferido)]:
    m = construir()
    m.fit(Xb_tr, yb_tr, epochs=20, batch_size=16, verbose=0)
    print(f"tarefa B com 150 amostras, {rotulo:<12}"
          f"{m.evaluate(Xb_te, yb_te, verbose=0)[1]:.4f}")

--- SAÍDA ---
tarefa A (dígitos 0-4): 1.0000
tarefa B com 150 amostras, do zero      0.9665
tarefa B com 150 amostras, transferido 0.9504

```

+-----

SITUAÇÃO	ARQUITETURA INDICADA
tabela de atributos independentes	MLP denso, ou florestas
imagem	CNN 2D
sinal ou série curta	CNN 1D
série com dependência longa	LSTM, GRU
sem rótulo, comprimir ou limpar	autoencoder
poucos dados, domínio parecido	transferência de aprendizado
texto ou sequências longas	transformer

+-- Tabela 41.1: por onde começar.

.....
>> EXPERIMENTOS PROPOSTOS

- . Troque a LSTM por uma CNN 1D no mesmo problema de série e compare erro e tempo de treino.
- . Use o autoencoder para reduzir a dimensão e alimente um classificador com o código de 8 números.
- . Treine com e sem `BatchNormalization` uma rede de cinco camadas e compare.
- . Aplique transferência de aprendizado a um problema seu com poucos dados rotulados.

[42] PYTORCH

A biblioteca preferida em pesquisa. O treino é um laço Python explícito, o que dá controle total e deixa cada etapa visível.

[!] ÚNICA SEÇÃO NÃO EXECUTADA DESTE MANUAL

Todos os demais códigos das partes I, II e III foram executados na geração do arquivo e mostram a saída real. Os desta seção não: o ambiente de composição não tinha GPU nem as bibliotecas CUDA que o pacote do PyTorch exige para carregar. O código segue a API oficial da versão 2.x e deve ser executado no seu computador ou no Colab, onde o PyTorch já vem instalado. Onde a saída é previsível, ela aparece descrita no texto ao lado.

Tensores e autograd

+-- #CÓDIGO 42.1 Tensores -----

A API é deliberadamente próxima do NumPy. A diferença principal é o método `.to(dispositivo)`, que move o tensor para a GPU.

```
import torch

escalar = torch.tensor(3.0)
vetor = torch.tensor([1.0, 2.0, 3.0])
matriz = torch.tensor([[1.0, 2.0], [3.0, 4.0]])

print(vetor.shape, vetor.dtype)
print(vetor.sum(), vetor.mean())
print(matriz @ matriz)
print(torch.zeros(2, 3), torch.ones(2), torch.arange(0, 5))

# conversao com NumPy
import numpy as np
print(torch.from_numpy(np.array([1.0, 2.0])))
print(vetor.numpy())

# dispositivo
dispositivo = "cuda" if torch.cuda.is_available() else "cpu"
print("rodando em:", dispositivo)
vetor = vetor.to(dispositivo)
```

+-- #CÓDIGO 42.2 Derivadas automáticas -----

`requires_grad=True` marca o tensor para rastreamento. `backward()` acumula as derivadas em `.grad`.

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x + 1

y.backward()           # calcula dy/dx

print("y =", y.item())   # 16.0
print("dy/dx =", x.grad.item()) # 8.0

# gradiente de uma funcao de varias variaveis
w = torch.tensor([1.0, 2.0], requires_grad=True)
perda = (w ** 2).sum()
perda.backward()
print("gradiente:", w.grad) # tensor([2., 4.] )
```

Um modelo completo

+-- #CÓDIGO 42.3 Definindo a rede -----

Em `nn.Module`, o método `forward` descreve o caminho dos dados. É onde entram conexões residuais, múltiplas entradas e tudo que não cabe em uma pilha.

```
import torch
from torch import nn

torch.manual_seed(42)

# Forma rapida, equivalente ao Sequential do Keras
modelo = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 16),
    nn.ReLU(),
    nn.Linear(16, 1),
)

# Forma explicita, usada quando a rede nao e uma pilha simples
class Rede(nn.Module):
    def __init__(self):
        super().__init__()
        self.oculta = nn.Linear(2, 16)
        self.saida = nn.Linear(16, 1)

    def forward(self, x):
        x = torch.relu(self.oculta(x))
        return self.saida(x)

rede = Rede()
print(rede)
print("parâmetros:", sum(p.numel() for p in rede.parameters()))
```

+-----

+-- #CÓDIGO 42.4 O laço de treino -----

Os cinco passos do laço são sempre os mesmos e valem a pena decorar. [BCEWithLogitsLoss](#) já inclui a sigmoide, por isso a última camada não a aplica.

```
import torch
from torch import nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

torch.manual_seed(42)
X, y = make_moons(n_samples=1200, noise=0.2, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=42)

X_tr = torch.tensor(X_tr, dtype=torch.float32)
y_tr = torch.tensor(y_tr, dtype=torch.float32).unsqueeze(1)
X_te = torch.tensor(X_te, dtype=torch.float32)
y_te = torch.tensor(y_te, dtype=torch.float32).unsqueeze(1)

modelo = nn.Sequential(nn.Linear(2, 16), nn.ReLU(),
                      nn.Linear(16, 16), nn.ReLU(),
                      nn.Linear(16, 1))
criterio = nn.BCEWithLogitsLoss()
otimizador = torch.optim.Adam(modelo.parameters(), lr=0.01)

for epoca in range(1, 201):
    modelo.train()
    otimizador.zero_grad()           # 1. zera gradientes antigos
    saida = modelo(X_tr)             # 2. propagacao direta
    perda = criterio(saida, y_tr)     # 3. calcula a perda
    perda.backward()                 # 4. retropropaga
    otimizador.step()                 # 5. atualiza os pesos

    if epoca % 50 == 0:
        print(f"época {epoca:>3}  perda {perda.item():.4f}")

modelo.eval()
with torch.no_grad():
    previsto = (torch.sigmoid(modelo(X_te)) >= 0.5).float()
    acuracia = (previsto == y_te).float().mean()
print(f"acurácia no teste: {acuracia.item():.4f}")
```

+-- #CÓDIGO 42.5 Lotes com DataLoader -----

Para conjuntos que não cabem na memória, o [DataLoader](#) entrega um lote por vez e ainda embaralha a cada época.

```
import torch
from torch.utils.data import TensorDataset, DataLoader

X = torch.randn(1000, 4)
y = torch.randint(0, 2, (1000, 1)).float()

conjunto = TensorDataset(X, y)
carregador = DataLoader(conjunto, batch_size=32, shuffle=True)

print("lotes por época:", len(carregador))
for entradas, alvos in carregador:
    print("formato do lote:", entradas.shape, alvos.shape)
    break

# dentro do treino:
# for entradas, alvos in carregador:
#     otimizador.zero_grad()
#     perda = criterio(modelo(entradas), alvos)
#     perda.backward()
#     otimizador.step()
```

+-- #CÓDIGO 42.6 Salvando e recarregando -----

Guarde o `state_dict` e mantenha a definição da classe no código. Salvar o objeto inteiro amarra o arquivo à versão da biblioteca.

```
import torch
from torch import nn

modelo = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 3))

# Recomendado: salvar apenas os pesos
torch.save(modelo.state_dict(), "modelo.pt")

novo = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 3))
novo.load_state_dict(torch.load("modelo.pt"))
novo.eval()

print("pesos carregados")
```

+-----

Escolhendo entre os dois

ASPECTO	KERAS / TENSORFLOW	PYTORCH
treino	<code>model.fit()</code> em uma linha	laço explícito de cinco passos
curva de aprendizado	mais suave para começar	exige entender cada etapa
depuração	abstraída	tensor a tensor, em Python puro
pesquisa	menos comum hoje	padrão na área
produção	TF Serving, TF Lite, TF.js maduros	TorchScript, ONNX, ExecuTorch
dispositivo	automático	explícito com <code>.to(device)</code>

+-- Tabela 42.1: as duas bibliotecas lado a lado.

[*] QUAL USAR NO TCC

Se o objetivo é entregar um resultado com arquitetura conhecida, Keras chega mais rápido. Se o trabalho exige modificar o treino, a função de perda ou a arquitetura, PyTorch dá menos atrito. Saber ler os dois é mais útil do que defender um deles.

.....

>> EXPERIMENTOS PROPOSTOS

- . Reescreva o modelo do CÓDIGO 39.6 em PyTorch e compare a acurácia.
- . Acrescente ao laço de treino o registro da perda por época e plote a curva.
- . Implemente uma parada antecipada manual dentro do laço.
- . Explique o papel de `zero_grad()` e o que acontece se ele for esquecido.

=====

|| BLOCO XVI

PREDIÇÃO E EXPORTAÇÃO

Etapas 5 e 6: levar o modelo ao mundo e fechar o ciclo com honestidade.

=====

[43] PREDIÇÃO, IMPLANTAÇÃO E EXPORTAÇÃO

Etapas 5 e 6 do processo. Um modelo só serve quando cabe no destino, responde no tempo exigido e continua sendo o mesmo depois de salvo.

+-- #CÓDIGO 43.1 Salvando e recarregando o modelo -----

Guarde junto a versão do scikit-learn: modelos salvos não têm compatibilidade garantida entre versões.

```
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

X, y = load_iris(return_X_y=True)
modelo = RandomForestClassifier(random_state=0).fit(X, y)

joblib.dump(modelo, "modelo.joblib")
recarregado = joblib.load("modelo.joblib")

print("previsões idênticas?",
      (modelo.predict(X) == recarregado.predict(X)).all())
import os
print("tamanho do arquivo:",
      os.path.getsize("modelo.joblib") // 1024, "KB")
```

--- SAÍDA ---

```
previsões idênticas? True
tamanho do arquivo: 184 KB
```

+-- #CÓDIGO 43.2 Medindo o tempo de inferência -----

Repare no custo por amostra isolada contra o lote: chamar o modelo mil vezes é muito mais lento que uma chamada com mil linhas. Em serviço de tempo real, agrupe as requisições.

```
import time
import numpy as np
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

X, y = load_digits(return_X_y=True)
X = X / 16.0
amostra = X[:1]

print(f"{'modelo':<14}{'treino (s)':>12}{'1 amostra (ms)':>17}"
      f"{'1000 juntas (ms)':>18}")
for nome, modelo in [
    ("logística", LogisticRegression(max_iter=2000)),
    ("k vizinhos", KNeighborsClassifier()),
    ("floresta", RandomForestClassifier(n_estimators=200,
                                       random_state=0))]:
    t0 = time.perf_counter()
    modelo.fit(X, y)
    treino = time.perf_counter() - t0

    t0 = time.perf_counter()
    for _ in range(200):
        modelo.predict(amostra)
    uma = (time.perf_counter() - t0) / 200 * 1000

    t0 = time.perf_counter()
    modelo.predict(X[:1000])
    lote = (time.perf_counter() - t0) * 1000

    print(f"{'nome':<14}{'treino:>12.3f'}{'uma:>17.3f'}{'lote:>18.2f}")
```

--- SAÍDA ---

modelo	treino (s)	1 amostra (ms)	1000 juntas (ms)
logística	0.076	0.058	0.47
k vizinhos	0.001	0.427	12.79
floresta	0.714	12.750	41.74

+-- #CÓDIGO 43.3 Tamanho do modelo em disco -----

Uma floresta guarda todas as árvores. Em microcontrolador ou navegador, esse número decide o projeto antes da acurácia.

```
import os
import joblib
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

X, y = load_digits(return_X_y=True)

for nome, modelo in [
    ("logística", LogisticRegression(max_iter=2000)),
    ("árvore", DecisionTreeClassifier(random_state=0)),
    ("floresta 200", RandomForestClassifier(n_estimators=200,
                                          random_state=0))]:

    modelo.fit(X, y)
    arquivo = f"{nome}.joblib".replace(" ", "_")
    joblib.dump(modelo, arquivo, compress=3)
    tamanho = os.path.getsize(arquivo) / 1024
    print(f"{nome:<14}{tamanho:>9.1f} KB    "
          f"acurácia {modelo.score(X, y):.4f}")
```

--- SAÍDA ---

logística	5.4 KB	acurácia 1.0000
árvore	8.5 KB	acurácia 1.0000
floresta 200	1866.3 KB	acurácia 1.0000

Do Keras para o dispositivo

+-- #CÓDIGO 43.4 Convertendo para TensorFlow Lite -----

A quantização troca os pesos de 32 bits por inteiros de 8, encolhendo o arquivo e acelerando a execução. É o caminho para rodar em Raspberry Pi, celular ou microcontrolador.

```
import os
import numpy as np
import keras
import tensorflow as tf
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=25, batch_size=32, verbose=0)
acuracia = modelo.evaluate(X_te, y_te, verbose=0)[1]

modelo.save("modelo.keras")
modelo.export("modelo_exportado", verbose=0)

conversor = tf.lite.TFLiteConverter.from_saved_model(
    "modelo_exportado")
comum = conversor.convert()
open("modelo.tflite", "wb").write(comum)

conversor.optimizations = [tf.lite.Optimize.DEFAULT]
quantizado = conversor.convert()
open("modelo_q.tflite", "wb").write(quantizado)

print(f"acurácia do modelo treinado: {acuracia:.4f}\n")
print(f"{'formato':<22}{'KB':>9}")
print(f"{'keras':<22}{os.path.getsize('modelo.keras')/1024:>9.1f}")
print(f"{'tflite':<22}{len(comum)/1024:>9.1f}")
print(f"{'tflite quantizado':<22}{len(quantizado)/1024:>9.1f}")
print(f"\nredução pela quantização: "
      f"{100 * (1 - len(quantizado) / len(comum)):.1f}%")
```

--- SAÍDA ---

acurácia do modelo treinado: 0.9644

formato	KB
keras	102.1
tflite	27.8
tflite quantizado	11.0

redução pela quantização: 60.4%

+-- #CÓDIGO 43.5 O modelo quantizado ainda acerta? -----

Sempre reavalie depois de converter. A perda costuma ser de frações de ponto percentual, mas precisa ser medida e registrada, não presumida.

```
import numpy as np
import keras
import tensorflow as tf
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=25, batch_size=32, verbose=0)
modelo.export("exp", verbose=0)

conversor = tf.lite.TFLiteConverter.from_saved_model("exp")
conversor.optimizations = [tf.lite.Optimize.DEFAULT]
open("q.tflite", "wb").write(conversor.convert())

interprete = tf.lite.Interpreter(model_path="q.tflite")
interprete.allocate_tensors()
entrada = interprete.get_input_details()[0]
saida = interprete.get_output_details()[0]

previstos = []
for amostra in X_te:
    interprete.set_tensor(entrada["index"], amostra[None, :])
    interprete.invoke()
    previstos.append(interprete.get_tensor(saida["index"]).argmax())

original = modelo.evaluate(X_te, y_te, verbose=0)[1]
convertido = (np.array(previstos) == y_te).mean()
print(f"acurácia no Keras: {original:.4f}")
print(f"acurácia no TFLite: {convertido:.4f}")
print(f"diferença: {abs(original - convertido):.4f}")
```

--- SAÍDA ---

```
acurácia no Keras:  0.9756
acurácia no TFLite: 0.9756
diferença: 0.0000
```

+-----
Depois de publicar

+-- #CÓDIGO 43.6 Detectando desvio nos dados -----

O teste de Kolmogorov-Smirnov compara a distribuição atual com a do treino. Quando ela muda, o modelo continua respondendo, mas sobre um mundo que já não é o que ele aprendeu.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(0)

referencia = rng.normal(-60, 3, 2000)      # medidas do treino
cenarios = {
    "sem mudança": rng.normal(-60, 3, 500),
    "deslocamento": rng.normal(-56, 3, 500),
    "mais dispersa": rng.normal(-60, 7, 500),
}

print(f"{'cenário':<16}{'média':>9}{'desvio':>9}"
      f"{'p (KS)':>10}  alerta")
for nome, atual in cenarios.items():
    _, p = stats.ks_2samp(referencia, atual)
    alerta = "SIM" if p < 0.01 else "não"
    print(f"{'nome':<16}{atual.mean():>9.2f}{atual.std():>9.2f}"
          f"{'p':>10.2e}  {alerta}")
```

--- SAÍDA ---

cenário	média	desvio	p (KS)	alerta
sem mudança	-60.10	2.96	6.02e-01	não
deslocamento	-56.18	2.93	2.90e-93	SIM
mais dispersa	-59.66	7.38	4.88e-17	SIM

+-----

+-- #CÓDIGO 43.7 Uma função de inferência para produção -----

Validar a entrada, devolver a confiança, marcar os casos duvidosos para revisão humana e carimbar a versão do modelo. Quatro cuidados que evitam a maior parte dos problemas em operação.

```
import json
import numpy as np

class Preditor:
    """Encapsula validação, inferência e registro."""

    def __init__(self, modelo, nomes, versao):
        self.modelo = modelo
        self.nomes = nomes
        self.versao = versao
        self.chamadas = 0

    def prever(self, entrada):
        if len(entrada) != len(self.nomes):
            raise ValueError(
                f"esperava {len(self.nomes)} atributos, "
                f"recebi {len(entrada)}")
        vetor = np.asarray(entrada, dtype=float).reshape(1, -1)
        if not np.isfinite(vetor).all():
            raise ValueError("entrada com valor ausente ou infinito")

        prob = self.modelo.predict_proba(vetor)[0]
        self.chamadas += 1
        return {
            "classe": int(prob.argmax()),
            "confianca": round(float(prob.max()), 4),
            "versao_modelo": self.versao,
            "revisar": bool(prob.max() < 0.60),
        }

from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

dados = load_iris()
modelo = RandomForestClassifier(random_state=0).fit(dados.data,
                                                    dados.target)
preditor = Preditor(modelo, dados.feature_names, "2026.07.1")

print(json.dumps(preditor.prever([5.1, 3.5, 1.4, 0.2]), indent=2))
try:
    preditor.prever([5.1, 3.5])
except ValueError as erro:
    print("\nrecusado:", erro)
print("chamadas atendidas:", preditor.chamadas)
```

--- SAÍDA ---

```
{
  "classe": 0,
  "confianca": 1.0,
  "versao_modelo": "2026.07.1",
  "revisar": false
}
```

```
recusado: esperava 4 atributos, recebi 2
chamadas atendidas: 1
```

[*] LISTA DE VERIFICAÇÃO ANTES DE PUBLICAR

O modelo salvo reproduz as previsões do modelo treinado? O tempo de resposta cabe no requisito? O tamanho cabe no destino? Existe versão registrada junto com as previsões? Existe monitor de desvio nos dados de entrada? Existe um caminho para desligar o modelo e voltar à regra anterior?

.....
>> EXPERIMENTOS PROPOSTOS

- . Meça o tempo de inferência do seu modelo com lote de 1 e de 1000 amostras.
- . Converta uma rede sua para TFLite e compare tamanho e acurácia.
- . Implemente o monitor de desvio sobre duas campanhas de medidas diferentes.
- . Escreva a classe `Predictor` para o seu problema, com as validações que ele exige.

[44] FECHANDO O CICLO: AVALIAÇÃO HONESTA

A diferença entre um modelo que funciona no notebook e um que funciona no mundo está quase sempre em como ele foi avaliado.

+-- #CÓDIGO 44.1 Reprodutibilidade -----

Sem semente fixa não existe comparação justa entre dois modelos: parte da diferença observada é sorteio.

```
import random
import numpy as np

def fixar_sementes(mente=42):
    """Fixa as sementes de todas as fontes de aleatoriedade."""
    random.seed(mente)
    np.random.seed(mente)
    # keras.utils.set_random_seed(mente)
    # torch.manual_seed(mente)

fixar_sementes(42)
primeiro = np.random.rand(3)

fixar_sementes(42)
segundo = np.random.rand(3)

print("mesma sequência?", np.allclose(primeiro, segundo))
print(primeiro.round(4))
```

```
--- SAÍDA ---
mesma sequência? True
[0.3745 0.9507 0.732 ]
```

+-- #CÓDIGO 44.2 Vazamento de dados -----

Qualquer decisão tomada olhando o conjunto inteiro, inclusive escolher atributos, contamina a avaliação. Tudo o que aprende com os dados vai dentro do pipeline.

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.svm import SVC

X, y = load_wine(return_X_y=True)

# ERRADO: seleciona atributos usando o conjunto inteiro
X_vazado = SelectKBest(f_classif, k=5).fit_transform(X, y)
errado = cross_val_score(SVC(), StandardScaler().fit_transform(X_vazado),
                          y, cv=5).mean()

# CERTO: seleção e escala dentro do pipeline, refeitas a cada dobra
fluxo = make_pipeline(StandardScaler(),
                       SelectKBest(f_classif, k=5), SVC())
certo = cross_val_score(fluxo, X, y, cv=5).mean()

print(f"com vazamento: {errado:.4f}")
print(f"sem vazamento: {certo:.4f}")
print(f"otimismo indevido: {errado - certo:+.4f}")
```

```
--- SAÍDA ---
com vazamento: 0.9776
sem vazamento: 0.9663
otimismo indevido: +0.0113
```

+-- #CÓDIGO 44.3 Treino, validação e teste -----

A validação escolhe o modelo, o teste apenas relata o resultado. Cada olhada no teste para tomar decisão gasta um pouco da confiança do número final.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True)

X_resto, X_teste, y_resto, y_teste = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)
X_treino, X_val, y_treino, y_val = train_test_split(
    X_resto, y_resto, test_size=0.25, random_state=0,
    stratify=y_resto)

print(f"treino {len(y_treino)} | validação {len(y_val)} | "
      f"teste {len(y_teste)}")

melhor, melhor_n = -1, None
for n in [5, 25, 100, 300]:
    m = RandomForestClassifier(n_estimators=n, random_state=0)
    m.fit(X_treino, y_treino)
    nota = m.score(X_val, y_val)
    print(f"  n_estimators={n:>3}  validação {nota:.4f}")
    if nota > melhor:
        melhor, melhor_n = nota, n

final = RandomForestClassifier(n_estimators=melhor_n,
                              random_state=0)
final.fit(X_resto, y_resto)
print(f"\nescolhido n={melhor_n}")
print(f"acurácia no teste (usado uma única vez): "
      f"{final.score(X_teste, y_teste):.4f}")
```

--- SAÍDA ---

```
treino 1077 | validação 360 | teste 360
n_estimators= 5  validação 0.9139
n_estimators= 25  validação 0.9667
n_estimators=100  validação 0.9806
n_estimators=300  validação 0.9722
```

```
escolhido n=100
acurácia no teste (usado uma única vez): 0.9806
```

+-----

+-- #CÓDIGO 44.4 Classes desbalanceadas -----

Com 97% de uma classe, um modelo que responde sempre a majoritária acerta 97%. Nesses casos, acurácia não é métrica: olhe revocação, F1 e a matriz.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, recall_score,
                             f1_score, confusion_matrix)
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=2000, weights=[0.97, 0.03],
                          n_informative=5, random_state=7)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=7, stratify=y)

print("proporção da classe rara:", round(y.mean(), 4))

for rotulo, peso in [("sem ajuste", None),
                    ("com class_weight", "balanced")]:
    m = LogisticRegression(max_iter=2000, class_weight=peso)
    m.fit(X_tr, y_tr)
    p = m.predict(X_te)
    print(f"\n{rotulo}")
    print(f"  acurácia  {accuracy_score(y_te, p):.4f}")
    print(f"  revocação {recall_score(y_te, p):.4f}")
    print(f"  F1        {f1_score(y_te, p):.4f}")
    print(f"  matriz:", confusion_matrix(y_te, p).tolist())
```

--- SAÍDA ---

proporção da classe rara: 0.034

sem ajuste

acurácia 0.9800
revocação 0.4500
F1 0.6000
matriz: [[579, 1], [11, 9]]

com class_weight

acurácia 0.7900
revocação 0.8000
F1 0.2025
matriz: [[458, 122], [4, 16]]

+--- #CÓDIGO 44.5 Curva de aprendizado -----

Se a curva de validação ainda sobe no fim, mais dados ajudam. Se as duas estacionaram longe uma da outra, o problema é o modelo, não a quantidade de dados.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import learning_curve
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True)

tamanhos, treino, validacao = learning_curve(
    RandomForestClassifier(n_estimators=60, random_state=0),
    X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 8),
    random_state=0)

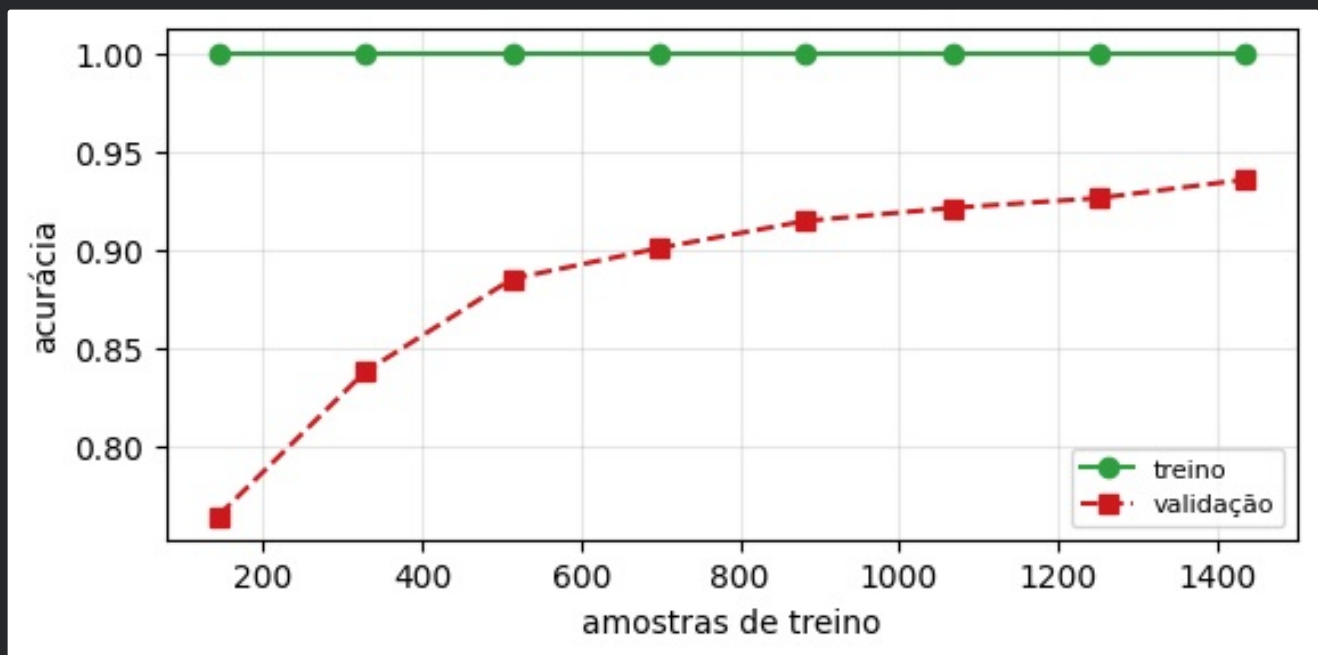
plt.figure(figsize=(6.4, 2.9))
plt.plot(tamanhos, treino.mean(axis=1), "o-", color="#2f9e41",
        label="treino")
plt.plot(tamanhos, validacao.mean(axis=1), "s--", color="#cd191e",
        label="validação")
plt.xlabel("amostras de treino")
plt.ylabel("acurácia")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("m05.png", dpi=100, bbox_inches="tight")

print("com 10% dos dados:", round(validacao.mean(axis=1)[0], 4))
print("com 100% dos dados:", round(validacao.mean(axis=1)[-1], 4))
```

--- SAÍDA ---

com 10% dos dados: 0.7641
com 100% dos dados: 0.9355

--- FIGURA ---



--- #CÓDIGO 44.6 Registrando o experimento -----

Um arquivo por experimento, com versões, hiperparâmetros e resultado. É o que permite comparar a tentativa de hoje com a de três semanas atrás.

```
import json
import platform
from datetime import date

import sklearn
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

X, y = load_digits(return_X_y=True)
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
notas = cross_val_score(modelo, X, y, cv=5)

experimento = {
    "data": date(2026, 7, 22).isoformat(),
    "python": platform.python_version(),
    "scikit_learn": sklearn.__version__,
    "numpy": np.__version__,
    "modelo": type(modelo).__name__,
    "hiperparametros": {"n_estimators": 100, "random_state": 42},
    "metrica": "acuracia",
    "media": round(float(notas.mean()), 4),
    "desvio": round(float(notas.std()), 4),
}

with open("experimento.json", "w", encoding="utf-8") as f:
    json.dump(experimento, f, indent=2, ensure_ascii=False)

print(json.dumps(experimento, indent=2, ensure_ascii=False))
```

--- SAÍDA ---

```
{
  "data": "2026-07-22",
  "python": "3.12.3",
  "scikit_learn": "1.8.0",
  "numpy": "2.4.4",
  "modelo": "RandomForestClassifier",
  "hiperparametros": {
    "n_estimators": 100,
    "random_state": 42
  },
  "metrica": "acuracia",
  "media": 0.9394,
  "desvio": 0.0213
}
```

Checklist antes de apresentar um modelo

1. A linha de base trivial foi calculada e o modelo a supera?
2. Treino, validação e teste foram separados antes de qualquer ajuste?
3. Toda transformação que aprende com os dados está dentro do pipeline?
4. A métrica escolhida corresponde ao custo real do erro?
5. As sementes e as versões estão registradas?
5. O modelo foi avaliado em dados que ele nunca viu?
7. Os erros do modelo foram inspecionados um a um?
3. Os limites do modelo estão escritos no relatório?

[!] LIMITES E RESPONSABILIDADE

Um modelo aprende as regularidades dos dados que recebeu, inclusive os vieses. Ele não sabe explicar suas decisões por conta própria, não generaliza para condições que não apareceram no treino e degrada quando o mundo muda. Em aplicações que afetam pessoas, isso deixa de ser detalhe técnico. Documente a origem dos dados, o desempenho por subgrupo e as condições em que o modelo não deve ser usado.

.....

>> EXPERIMENTOS PROPOSTOS

- . Refaça um experimento seu com e sem pipeline e compare os resultados.
- . Calcule a linha de base majoritária de um conjunto desbalanceado e compare com o seu modelo.
- . Gere a curva de aprendizado do seu modelo e decida se vale coletar mais dados.
- . Escreva o registro em JSON de três variações do seu modelo e compare-as.

=====

|| BLOCO XVII

APÊNDICES

Referência rápida de aprendizado de máquina.

=====

[F] CARTÃO DE CONSULTA E GLOSSÁRIO

As chamadas mais usadas das três bibliotecas e o vocabulário em inglês que você vai encontrar na documentação.

+-- #CÓDIGO F.1 Resumo das três bibliotecas -----

```
# ---- scikit-learn ----
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
modelo = make_pipeline(StandardScaler(), RandomForestClassifier())
modelo.fit(X_tr, y_tr)
modelo.predict(X_te); modelo.score(X_te, y_te)
cross_val_score(modelo, X, y, cv=5).mean()
print(classification_report(y_te, modelo.predict(X_te)))

# ---- keras ----
import keras
from keras import layers
modelo = keras.Sequential([
    layers.Input(shape=(n_atributos,)),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(n_classes, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
h = modelo.fit(X_tr, y_tr, epochs=50, batch_size=32,
               validation_split=0.2, verbose=0)
modelo.evaluate(X_te, y_te); modelo.save("modelo.keras")

# ---- pytorch ----
import torch
from torch import nn
modelo = nn.Sequential(nn.Linear(n, 64), nn.ReLU(), nn.Linear(64, c))
criterio = nn.CrossEntropyLoss()
otim = torch.optim.Adam(modelo.parameters(), lr=1e-3)
for epoca in range(epocas):
    otim.zero_grad()
    perda = criterio(modelo(X_tr), y_tr)
    perda.backward()
    otim.step()
torch.save(modelo.state_dict(), "modelo.pt")
```

Glossário

INGLÊS	PORTUGUÊS	O QUE É
feature	atributo	coluna de entrada
label / target	rótulo / alvo	resposta desejada
training set	conjunto de treino	usado para ajustar
validation set	conjunto de validação	usado para escolher o modelo
test set	conjunto de teste	usado uma única vez
loss	perda	quanto o modelo erra
epoch	época	uma passagem pelos dados
batch	lote	grupo de amostras por atualização
learning rate	taxa de aprendizado	tamanho do passo
overfitting	sobreajuste	decorar o treino
underfitting	subajuste	modelo simples demais
gradient descent	descida do gradiente	método de otimização
backpropagation	retropropagação	cálculo das derivadas na rede
dropout	abandono	desligar neurônios durante o treino
inference	inferência	usar o modelo treinado
data leakage	vazamento de dados	informação do teste no treino

+-- Tabela F.1: vocabulário essencial.

Para continuar

- * Documentação do scikit-learn, seção *User Guide*, com explicação teórica de cada algoritmo.
- * Guia do Keras em keras.io/guides e os tutoriais oficiais do PyTorch.
- * Conjuntos de dados para praticar: sklearn.datasets, Kaggle, UCI e, para sinais de rádio, o RadioML.
- * Ambiente sem instalar nada: Google Colab, com GPU gratuita por sessão.
- * Próximos temas: a Parte IV deste manual, com atenção, transformers, modelos generativos, RAG e ajuste fino.

[*] SOBRE AS TRÊS PARTES DESTA MANUAL

A Parte I ensina a linguagem, a Parte II o ferramental numérico e a Parte III o aprendizado de máquina. A ordem não é obrigatória, mas cada volume assume o anterior: as seções de NumPy e Matplotlib aparecem em quase todo código desta Parte III.